

From Pixels to Threads:

Understanding GPUs, from the Rendering Pipeline to General-Purpose Computing

Mahesha S
Compiler Engineer

mahesha.shivamallappa@gmail.com

Abstract

The graphics processing unit (GPU) began life as a chip whose only job was to draw images on a screen. Within about two decades it became the engine of scientific computing and, more recently, of artificial intelligence. This paper explains how that happened and how a GPU works, for a reader who knows basic programming but nothing about computer graphics or GPUs; every term is explained when it first appears. After a short warm-up on computer basics, we show how a computer draws a 3D scene and why that work splits naturally into millions of independent pieces. We then show how the stages of this drawing work were first built as fixed-function hardware, a separate circuit for each stage that could be adjusted with a few settings but not programmed, and how by 1999 all these circuits fitted on a single chip, the first GPU. Next we show how two of the GPU's stages became programmable, turning it into a parallel machine that runs the programmer's own code, and how researchers then used it for general calculations by disguising them as drawing, until unified hardware and the CUDA language turned the GPU into a general-purpose parallel processor. The core of the paper explains the three design ideas behind every GPU (many simple cores, one instruction driving many arithmetic units, and many threads kept ready to hide memory delays), how they make a GPU different from a CPU, how GPU programs are written, what is inside a modern NVIDIA GPU, the H100, and how groups of 32 threads execute together, including what happens when they disagree at an `if`. We then derive the main rules for writing fast GPU code (occupancy, memory access and the amount of computation) and apply them, step by step, to a complete example. We then show how GPU design has changed to keep up with artificial intelligence. A dedicated section compares NVIDIA and AMD GPUs: their history, the design of their cores, the way each handles threads that disagree at an `if`, shown on the machine code both compilers produce for the same kernel, their units for values shared by a whole group of threads, and their matrix engines. The paper is a tutorial survey; it claims no novelty, and aims to leave a newcomer with a clear, connected picture of what GPUs are, how they work and why they look the way they do.

Keywords: GPU architecture, GPGPU, graphics rendering pipeline, programmable shaders, SPMD, SIMT execution, branch divergence, latency hiding, occupancy, CUDA, streaming multi-processor, NVIDIA, AMD, tensor cores, parallel reduction.

Contents

1	Introduction	4
2	Warm-up: A Few Computer Basics	6
2.1	Programs, instructions and processors	6
2.2	Registers, memory and caches	6
2.3	Latency and bandwidth	6
2.4	Threads	7
2.5	Parallelism, and data parallelism in particular	7
2.6	Vector instructions	7
2.7	Numbers: floating point and FLOP/s	8
3	How a Computer Draws a 3D Scene	9
3.1	The goal: a screenful of coloured dots, again and again	10
3.2	The input: a 3D world built from triangles	10
3.3	The recipe: three stages	11
3.4	Stage 1: where does each corner land on the screen?	11
3.5	Stage 2: which pixels does each triangle cover?	12
3.6	Stage 3: what colour is each pixel?	12
3.7	The key observation: the same small program, millions of times	13
4	Fixed-Function Hardware: The Graphics Pipeline in Silicon	14
4.1	Why special hardware?	14
4.2	How a fixed-function chip works	15
4.3	The first GPU: NVIDIA's GeForce 256	15
4.4	The one thing still missing: programmability	16
5	Programmable GPU: Shaders Replace Fixed-Function Stages	17
5.1	From settings to programs	17
5.2	How a programmable GPU works	17
5.3	Floating point and shading languages	18
5.4	What the GPU had become	18
6	The Birth of GPGPU	19
6.1	Computing disguised as drawing	19
6.2	Hiding the graphics	20
6.3	The limits of computing through graphics	20
7	Unification: The GPU Becomes a General-Purpose Processor	21
7.1	One kind of processor for everything	21
7.2	CUDA, and the end of the graphics disguise	22
7.3	Maturing for science, then for AI	22
7.4	One programming model, many languages	23
8	Three Design Ideas Behind Every GPU	24
8.1	Starting point: a "fat" CPU core	24
8.2	Idea #1: many slim cores	25
8.3	Idea #2: one instruction, many ALUs (SIMD)	25
8.4	Idea #3: hide memory delays by switching between threads	26
8.5	Putting the ideas together	28
8.6	The result: a GPU next to a CPU	28

9	How a GPU Program Is Written	29
9.1	Host, device and kernels	30
9.2	Organising threads: grids and blocks	30
9.3	A first kernel	32
9.4	Where data lives: memory spaces	32
9.5	One model, many names	34
10	Inside a Modern NVIDIA GPU	34
10.1	The whole chip	35
10.2	Handing out thread blocks	36
10.3	Inside one SM	38
10.4	Choosing what to run, every cycle	42
10.5	Values shared by a whole warp	46
11	How a Warp Executes: the SIMT Model	46
11.1	Thirty-two threads, one instruction	46
11.2	SIMD versus SIMT	47
11.3	When threads disagree: branch divergence	47
12	What Makes GPU Code Fast	49
12.1	Keeping enough warps on each SM: occupancy	50
12.2	Accessing memory the right way	52
12.3	Making the computation efficient	56
12.4	A worked example: summing a million numbers into one	59
13	GPUs in the Age of AI	69
13.1	Why AI means matrix multiplication	70
13.2	Tensor cores: matrix units inside the SM	70
13.3	Feeding the tensor cores	70
13.4	Smaller numbers	71
13.5	Beyond one piece of silicon	71
14	NVIDIA and AMD: Two Implementations of the Same Design Ideas	71
14.1	Two family trees	72
14.2	The core: SM versus compute unit	74
14.3	Handling divergence	79
14.4	Computing a value once for the whole group	83
14.5	Matrix engines	83
15	Conclusion	84
A	Glossary	91

1 Introduction

A **graphics processing unit**, or GPU, is a chip that was originally built to do one job: draw the images you see on a computer screen. Every time a video game shows a new frame, the GPU calculates the colour of each of the millions of dots (pixels) on the screen, and it does this dozens of times every second. To manage that, GPUs were designed very differently from the main processor of a computer, the CPU. Instead of a few powerful processing cores, a GPU has about a hundred simpler cores, and together they contain thousands of small arithmetic units that all work at the same time.

Over the last twenty years, people discovered that this design is good at much more than drawing pictures. Any task that consists of doing the same calculation on a very large amount of data, such as simulating weather, analysing medical images or training a neural network, can be split across those thousands of cores. In 2012, two ordinary gaming GPUs were used to train a neural network that recognised objects in photographs far better than anything before it [32], and that result helped start today’s boom in artificial intelligence. Today many of the largest supercomputers and large AI systems are built around GPUs, and the word “graphics” in the name has become something of a historical accident.

This paper explains how that happened and how a GPU actually works. It is written for a reader who knows basic programming (simple C code is enough) but *nothing* about computer graphics or GPUs. Every term is explained when it first appears, and Section 2 gives the few pieces of general computer background the rest of the paper relies on. By the end, the reader should be able to answer four questions with confidence:

1. **Where did GPUs come from?** How a chip for drawing pictures turned into a general-purpose parallel computer.
2. **Why are GPUs built the way they are?** The three design ideas that set a GPU apart from a CPU.
3. **How does a GPU run a program?** How work is divided into threads, how the hardware schedules them, and what happens inside one GPU core, using a current NVIDIA GPU as the example.
4. **What makes GPU code fast or slow?** The handful of rules that follow from the design, shown on a complete worked example.

The one idea to remember.

The whole story rests on a single observation: *drawing an image means doing the same small calculation for millions of independent pieces of data*. Each pixel’s colour can be computed without waiting for any other pixel. Hardware built to exploit that kind of independence turns out to be useful for a great deal of other work too. The rest of the paper is about how engineers discovered this, exploited it and finally rebuilt the GPU around it.

Why another GPU tutorial?

There are excellent textbooks on GPU programming [44, 72] and excellent architecture papers on individual GPU generations [1, 38, 59]. What is harder to find is a single, beginner-friendly narrative that connects the *history* (where the ideas came from), the *mechanism* (how the hardware actually runs a program) and the *practice* (why some code runs fast and other code does not). Engineers who join a GPU compiler, runtime or performance team usually assemble this picture piece by piece;

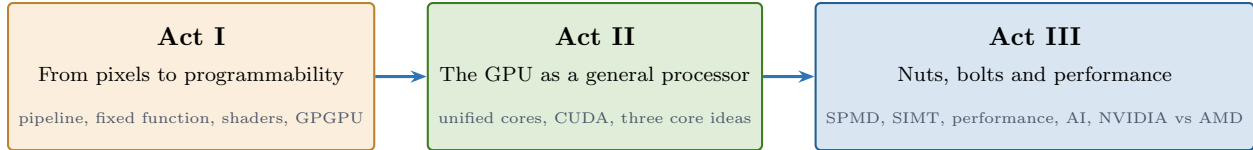


Figure 1: The structure of the paper as a three-act story.

this paper tries to give it in one place. Throughout, it follows one simple rule: build intuition first, add detail second. It also serves as background for the author’s more specialised surveys on branch divergence [68], systolic arrays and matrix engines [70], and low-precision number formats [69].

Roadmap.

After a short warm-up on computer basics (Section 2), the paper is organised as a story in three acts, summarised in Figure 1.

- **Act I: from pixels to programmability.** Section 3 explains, step by step, how a computer turns a 3D scene into an image, and why that work is so easy to split up. Section 4 shows how the stages of that work were first built as fixed-function hardware, a separate circuit for each stage that could be adjusted but not programmed, until in 1999 one chip, the first to be called a GPU, held all of them. Section 5 shows how two of the pipeline’s stages became programmable, and Section 6 how researchers then tricked GPUs into doing non-graphics calculations.
- **Act II: the GPU as a general-purpose processor.** Section 7 describes how GPUs were redesigned for general computing and how CUDA made them easy to program. Section 8 explains the three design ideas behind every GPU, and how they make a GPU so different from a CPU.
- **Act III: nuts and bolts.** Section 9 shows how a GPU program is written and how its work is divided into threads. Section 10 opens up a current NVIDIA GPU, our running example for the rest of the paper. Section 11 explains how groups of threads execute together and what happens when they disagree at an `if` statement. Section 12 explains the factors that make GPU code fast or slow (occupancy, memory access and the amount of computation) and applies them to a complete example, the sum of a million numbers. Section 13 looks at how GPU design has changed to keep up with artificial intelligence. Section 14 compares NVIDIA and AMD GPUs on five points: their history, the core, the handling of divergence, the handling of values shared by a whole group of threads, and the matrix engines. Section 15 gives our concluding remarks.

A note on vocabulary.

GPU makers often use different names for the same thing. Because our running example is an NVIDIA GPU, we use NVIDIA’s terms throughout and switch to AMD’s only in Section 14. Table 4 (Section 9) translates between vendors, and the glossary in Appendix A collects every technical term with a one-line definition, so a reader who meets a forgotten word can look it up quickly.

2 Warm-up: A Few Computer Basics

This section collects the handful of general computer ideas the rest of the paper builds on. A reader who already knows what a register, a cache, a thread and a vector instruction are can safely skip to Section 3.

2.1 Programs, instructions and processors

A program written in a language such as C is translated by a **compiler** into **machine instructions**: very small steps such as “load a number from memory”, “add two numbers”, “compare two numbers” or “jump to another instruction if the comparison was true”. A **processor** executes these instructions one after another. For each instruction it *fetches* the instruction from memory, *decodes* it to work out what it asks for, and *executes* it, usually in an **arithmetic logic unit** (ALU), the circuit that actually adds, multiplies or compares. A special register called the **program counter** (PC) remembers which instruction comes next.

The processor works in steps paced by a **clock**. A clock rate of 2 GHz means two billion steps per second; one step is called a **cycle**. A modern chip contains several complete processors, called **cores**, each able to run its own sequence of instructions.

2.2 Registers, memory and caches

Instructions work on data, and data must be stored somewhere. There are three main places (Figure 2).

- **Registers** are a few dozen small storage slots inside the core itself. An ALU can read and write them in about one cycle.
- **Main memory** (also called DRAM) is the large memory of the computer, measured in gigabytes. It is built from separate memory chips, outside the processor chip and connected to it by wires on the circuit board, so fetching one value from it takes on the order of a hundred nanoseconds, which is hundreds of cycles of waiting.
- **Caches** sit in between: small, fast memories on the processor chip that keep copies of recently used data. If the data a program needs is already in the cache, the long trip to main memory is avoided.

2.3 Latency and bandwidth

Two different numbers describe how fast memory is, and the difference between them is central to this paper.

- **Latency** is how long one request takes: the time from asking for a value until it arrives.
- **Bandwidth** is how much data can be delivered per second when many requests are in flight at once. It is measured in bytes per second for every kind of processor: a desktop CPU’s main memory delivers tens of gigabytes per second (GB/s), a large server CPU’s a few hundred GB/s, and a data-centre GPU’s several terabytes per second (TB/s).

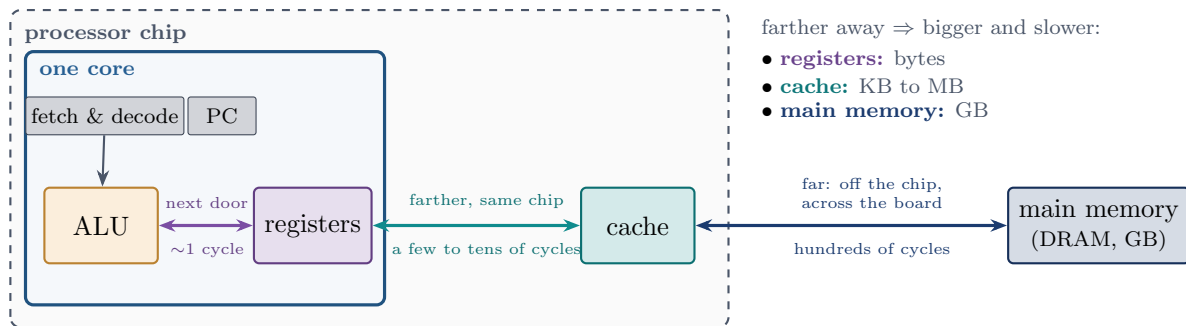


Figure 2: Where data lives, drawn by distance from the ALU. Registers sit right next to the ALU inside the core; the cache is on the same chip but farther away; main memory is on separate chips across the circuit board. The farther away the storage, the larger and slower it is. Waiting for main memory is one of the biggest costs in any computer, and CPUs and GPUs deal with it in completely different ways (Section 8).

A motorway is a useful picture. Latency is how long one car takes to drive from one city to the other. Bandwidth is how many cars arrive per hour. Adding more lanes does not make any single car arrive sooner, but it lets far more cars arrive per hour. As we will see, CPUs are designed to make each individual request finish quickly (low latency), whereas GPUs are designed to keep a huge number of requests moving at once (high bandwidth).

2.4 Threads

A **thread** is one independent sequence of instructions being executed, together with its own program counter and its own register values. A program can create many threads that run at the same time on different cores, or take turns on the same core. Switching a CPU core from one thread to another is normally done by the operating system and takes thousands of cycles, because the registers of the old thread must be saved and those of the new thread loaded. GPUs, we shall see, are built so that this switch costs nothing at all.

2.5 Parallelism, and data parallelism in particular

Parallelism means doing several pieces of work at the same time. The kind that matters most for GPUs is **data parallelism**: performing the *same* operation on many *different* data items, where no item depends on any other. Listing 1 shows a simple example that brightens a photograph.

Listing 1: A data-parallel loop: every iteration is independent of every other.

```
1 // brighten every pixel of an image by 10%
2 for (int i = 0; i < num_pixels; i++)
3     pixel[i] = min(pixel[i] * 1.1f, 1.0f);
```

Iteration i only touches `pixel[i]`. Nothing stops us from running all iterations at the same time on different ALUs: the final image would be exactly the same as if the loop had run one iteration after another. Loops like this are the natural food of a GPU. A program with a long chain of steps, each needing the result of the one before, cannot be split this way and remains the job of the CPU.

2.6 Vector instructions

One way to exploit data parallelism inside a single core is a **vector** or **SIMD** (single instruction, multiple data) instruction: one instruction that adds, say, eight pairs of numbers at once using

eight ALUs side by side (Figure 3). Modern CPUs have vector units 4 to 16 numbers wide. GPUs take this idea much further, as Section 8 explains.

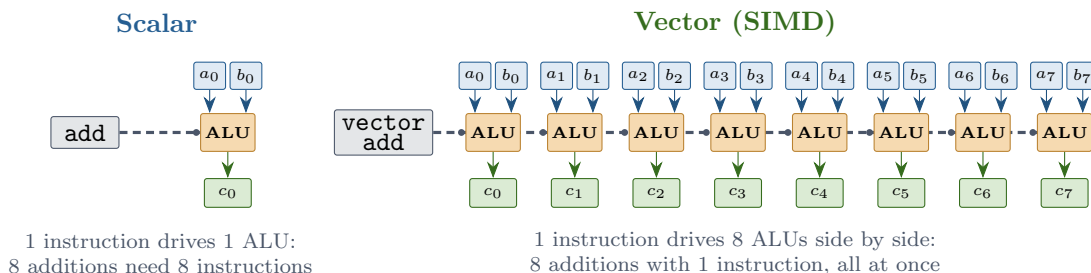


Figure 3: Scalar versus vector (SIMD) execution. A scalar `add` drives one ALU and produces one result, so eight additions need eight instructions, one after another. A vector add is decoded once and drives eight ALUs side by side, each working on its own pair of numbers, so all eight results are produced at once.

2.7 Numbers: floating point and FLOP/s

Why not just integers?

Integers ($\dots, -2, -1, 0, 1, 2, \dots$) are enough for counting, but most calculations a GPU performs, such as the position of a point in 3D space, the brightness of a pixel or the weight in a neural network, need fractional values like 3.14 or -0.001 , and sometimes very large or very small ones. For these, computers use **floating-point** numbers.

How a floating-point number is stored.

A floating-point number is the computer’s version of scientific notation. Just as 6,020,000 can be written 6.02×10^6 , a floating-point number is stored as three pieces packed into a fixed number of bits:

- a **sign** bit, saying whether the number is positive or negative;
- an **exponent**, saying how far the point “floats” (the power of two the number is scaled by), which sets the *range* of values that can be represented, from tiny to huge; and
- a **fraction** (or mantissa), holding the significant digits, which sets the *precision*, that is, how many digits are exact.

Because the number of bits is fixed, most real numbers cannot be stored exactly and are rounded to the nearest representable value. More bits mean smaller rounding errors and a wider range; fewer bits mean larger errors, but also less storage and faster arithmetic.

Common sizes.

Table 1 lists the formats that appear in this paper. The standard workhorse is the 32-bit format, **FP32** (“single precision”). Scientific simulations that need many exact digits use **FP64** (“double precision”). Artificial intelligence, which turns out to tolerate large rounding errors, increasingly uses 16-, 8- and even 4-bit formats, because smaller numbers are cheaper to store, to move and to compute with [69]. This trade-off between accuracy and speed returns in Section 13.

Table 1: Floating-point formats used in this paper. “Digits” is the approximate number of correct decimal digits; the bits are split into sign, exponent and fraction.

Format	Bits	Sign / exponent / fraction	Digits	Typical use
FP64 (double)	64	1 / 11 / 52	about 16	scientific simulation
FP32 (single)	32	1 / 8 / 23	about 7	graphics, general computing
FP16 (half)	16	1 / 5 / 10	about 3	graphics, AI
BF16 (bfloat16)	16	1 / 8 / 7	about 2	AI training
FP8	8	1 / 4 / 3 or 1 / 5 / 2	about 1	AI training and inference
FP4	4	1 / 2 / 1	under 1	AI inference

Measuring arithmetic speed: FLOP/s.

One floating-point addition or multiplication is one **floating-point operation**, or FLOP. The arithmetic speed of a processor is measured in **FLOP/s**, FLOPs per second, with the usual prefixes: a gigaFLOP/s (GFLOP/s) is 10^9 , a teraFLOP/s (TFLOP/s) 10^{12} and a petaFLOP/s (PFLOP/s) 10^{15} operations per second. Most processors can perform a **fused multiply-add** ($a \times b + c$) as a single instruction, which counts as two FLOPs.

The *peak* speed of a processor is simple arithmetic: the number of ALUs, times the clock rate, times the FLOPs each ALU completes per cycle. For example, an NVIDIA H100 GPU (in its SXM form) has $132 \times 128 = 16,896$ ALUs for 32-bit floating-point arithmetic; each can complete one fused multiply-add (2 FLOPs) per cycle. NVIDIA quotes a peak of 67 TFLOP/s in FP32 [62], which corresponds to a clock rate of about 1.98 GHz:

$$16,896 \times 2 \times 1.98 \times 10^9 \approx 67 \times 10^{12} \text{ FLOP/s} = 67 \text{ TFLOP/s.}$$

A typical desktop CPU reaches roughly one to a few TFLOP/s in FP32. Real programs rarely reach peak speed, for reasons that Section 12 explains; the narrow formats used for AI run many times faster still on the GPU’s specialised matrix units (Section 13).

Takeaway: what to carry forward

Five ideas from this warm-up will appear again and again: *instructions and ALUs* (the work, and the units that do it), *registers and main memory* (where data lives, and how far away it is), *latency versus bandwidth* (the time for one request versus the volume of many), *threads* (independent streams of instructions), and *data parallelism* (the same operation on many independent items). A GPU is, in one sentence, a machine designed to run an enormous number of threads doing data-parallel work, while hiding the long latency of memory.

3 How a Computer Draws a 3D Scene

GPUs were invented to draw pictures, so the natural place to start is with the job itself. This section says nothing about hardware. It explains, step by step, the *work* a computer must do to turn a description of a 3D world into the picture on a screen, much as a cookbook explains a recipe before anyone builds a kitchen. At the end we step back and notice one property of that work. That property is the single most important idea in this paper: every machine described in the later sections was built to exploit it.

3.1 The goal: a screenful of coloured dots, again and again

A screen is a grid of tiny coloured dots called **pixels**. A common “4K” screen has 3840×2160 pixels, about 8.3 million of them. Every picture you see on it, whether a photograph, a spreadsheet or a video game, is nothing more than a choice of colour for each of those dots.

A moving image is a rapid sequence of still pictures, called **frames**, shown one after another so quickly that the eye sees smooth motion. Games typically show 60 or more frames every second, so each frame must be ready in just $1/60$ of a second. Producing a frame, that is, computing the colour of every pixel in it, is called **rendering**.

So the goal is easy to state and hard to meet: about 8.3 million pixel colours, 60 times a second, or roughly 5×10^8 colours every second. But colours computed *from what*? The next subsection answers that.

3.2 The input: a 3D world built from triangles

A world of shapes, not pictures.

Picture a video game in which a knight walks through a castle courtyard, past stone walls and a fountain. What you see on the screen is the courtyard as viewed from one particular spot, say just behind the knight’s shoulder. Graphics calls that viewpoint the **camera**. It is not an object in the game and not a real device: it is simply an imaginary eye placed somewhere in the 3D world, with a position and a direction in which it looks, and each frame shows what that eye would see. In this game, the camera follows the knight, so it moves whenever he does. The game therefore cannot keep a ready-made picture of this 3D world to copy onto the screen, because every time the knight or the camera moves, the view changes. Instead it keeps a description of the *shapes* in this 3D world, called the **scene**, and for every single frame it works out, from scratch, what the camera would see from where it now stands. Rendering is therefore the journey from a 3D scene to a 2D grid of coloured pixels, repeated at 60 or more frames per second.

Triangles.

How can a computer describe a curved shape such as the knight’s helmet? It cheats, cleverly. The surface of every object is approximated by a large number of small, flat **triangles**, joined edge to edge like the panels of a football or the tiles of a mosaic. A few triangles give a rough, faceted shape; thousands give a surface that looks perfectly smooth on the screen (Figure 4a). A detailed game character may use hundreds of thousands of triangles, and a whole scene many millions. Triangles are chosen because they are the simplest flat shape there is: any three points always lie on one flat surface, so a triangle can never be bent or twisted, and every triangle can be handled in exactly the same way.

Vertices.

Each corner of a triangle is called a **vertex** (plural **vertices**), and the vertices are what the scene actually stores (Figure 4b). A vertex is a point in 3D space, given by its (x, y, z) coordinates, together with a little extra information: a colour, the direction the surface faces at that point (its **normal**, needed for lighting), and a pair of texture coordinates, explained next. Neighbouring triangles share corners, so each object is stored as one long list of vertices plus a note of which three of them form each triangle. The scene is then simply the collection of all its objects, together with the textures, camera and lights described below.

Textures.

Triangles alone would give a world of plain, flat-coloured surfaces. To make a wall look like rough stone, a picture of stone, called a **texture**, is “glued” onto its triangles, much like wallpaper onto a wall (Figure 4c). Each vertex’s **texture coordinates** say which point of the picture belongs to that corner; the points in between are filled in from the picture automatically.

The camera and the lights.

Finally, the scene records the camera, that is, where the viewpoint is and which way it looks, and where the lights are. With that, the input to rendering is complete: a long list of vertices, a set of texture pictures, a camera and some lights. The output is a frame of coloured pixels.

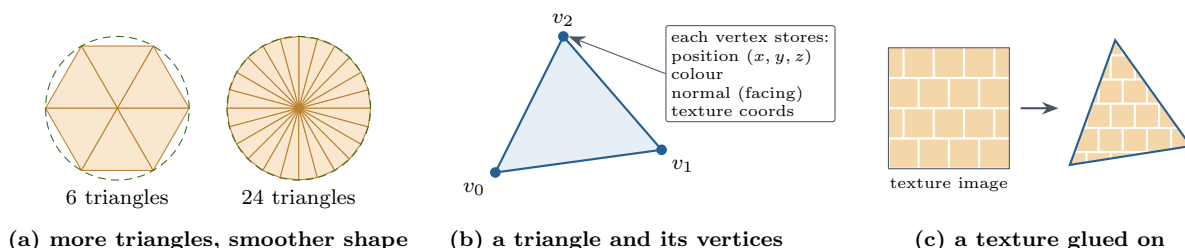


Figure 4: The raw material of 3D graphics. (a) Curved surfaces are approximated by flat triangles; the more triangles, the smoother the shape (the dashed circle is the ideal shape). (b) Each triangle is defined by its three corners, the vertices, and each vertex carries its position and a few other values. (c) An image, the texture, is glued onto the triangles to add detail that the geometry alone does not have.

3.3 The recipe: three stages

The work of turning the scene into a frame is always organised in the same order, as a production line of three stages called the **rendering pipeline** (Figure 5). Each stage answers one question:

1. **Stage 1, vertex processing:** where on the screen does each corner of each triangle land?
2. **Stage 2, rasterisation:** which pixels does each triangle cover?
3. **Stage 3, fragment processing:** what colour should each of those pixels be?

Game programs do not perform these stages themselves. They describe the scene through a standard software library, called a **graphics API** (application programming interface), such as OpenGL, Direct3D, Vulkan or Metal, and the library has the stages carried out. Real graphics APIs split the work into more steps than we describe here, but they all reduce to these three. We describe each stage only as far as we need to see one thing: how its work splits into a huge number of small, separate pieces. The next three subsections take the stages one at a time, and Figure 6, at the end of Section 3.6, then follows a single triangle through all three: Stage 1 finds where its three corners land on the screen, Stage 2 finds the pixels it covers, and Stage 3 gives each of those pixels its colour.

3.4 Stage 1: where does each corner land on the screen?

Every vertex is stored as a point in 3D space. Stage 1 works out where on the screen the camera would see it. This is one small, fixed calculation: the vertex’s (x, y, z) coordinates are multiplied by

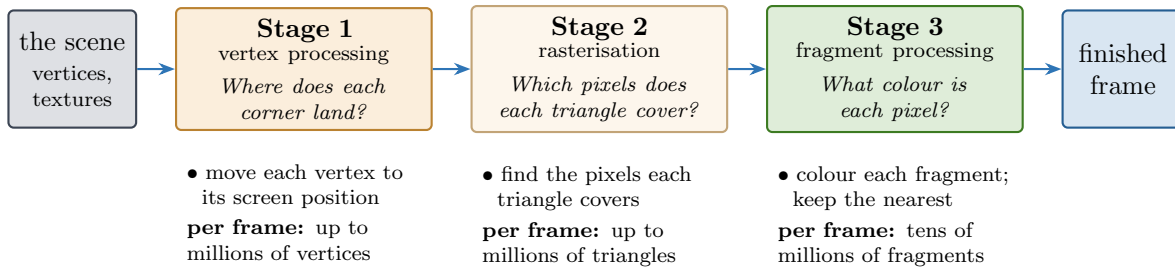


Figure 5: The rendering pipeline: three stages, each answering one question. The scene enters on the left and a finished frame leaves on the right. Every stage handles a very large number of separate items per frame (vertices, triangles, fragments), and handles each of them in the same way.

a few small tables of numbers (4×4 **matrices**) that describe where the vertex’s object is placed in the world, where the camera stands, and how perspective makes distant things look smaller. These tables are the same for every vertex of an object; only the vertex’s own coordinates differ.

The parallel work. A frame can involve up to millions of vertices. Every one of them goes through exactly the same calculation, and moving one vertex needs nothing from any other: vertex 5,001 can be done before, after or at the same time as vertex 5,000.

3.5 Stage 2: which pixels does each triangle cover?

Once its three corners have screen positions, each triangle covers some patch of the screen’s pixels (Figure 6b). Stage 2, called **rasterisation**, finds exactly which pixels those are (Figure 6c); triangles that lie entirely outside the camera’s view are simply skipped. For every pixel a triangle covers, rasterisation produces a **fragment**: a “candidate pixel” that records which pixel it belongs to, how far this bit of the triangle is from the camera (its **depth**), and the values needed to colour it, blended from the triangle’s three corners. Because triangles can overlap, one in front of another, a single pixel may receive several fragments; Stage 3 decides which one is seen.

The parallel work. A frame can involve up to millions of triangles. Every triangle is tested against the pixels in exactly the same way, and finding the pixels one triangle covers needs nothing from any other triangle.

3.6 Stage 3: what colour is each pixel?

Finally, every fragment is given its colour, a step called **shading** (Figure 6d). Typically the colour is looked up in the texture picture at the fragment’s texture coordinates, and then made brighter or darker according to how directly the surface faces the light. Where several fragments land on the same pixel, only the one nearest the camera is kept (the **depth test**); that is how a wall in front hides the fountain behind it. The surviving colour becomes the pixel’s colour in the finished frame.

The parallel work. This is the biggest stage. A 4K screen at 60 frames per second already needs 5×10^8 pixel colours every second, before counting fragments that are coloured and then hidden, and each fragment can take dozens of arithmetic operations. Yet every fragment follows the same recipe, and colouring one fragment needs nothing from its neighbours.

Figure 6 puts the three stages together for a single triangle, from its three corners in the 3D scene to its finished, coloured pixels.

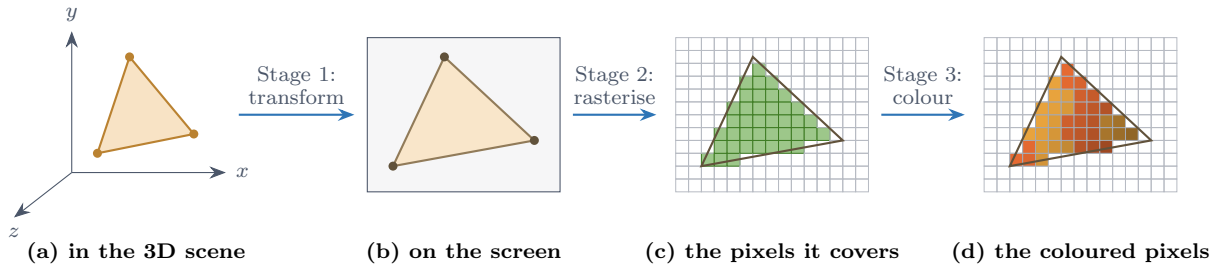


Figure 6: One triangle’s journey through all three stages. (a) Its three vertices, somewhere in the 3D scene. (b) Stage 1 moves each vertex, separately, to where it appears on the screen. (c) Stage 2 finds every pixel whose centre lies inside the triangle and creates one fragment (green) for each. (d) Stage 3 colours each fragment separately: here a brick texture is looked up for every fragment, and lighting makes the side nearer the light (upper left) brighter.

3.7 The key observation: the same small program, millions of times

Now put the three “parallel work” paragraphs side by side, as a designer of computers would. The same three properties appear in every stage.

1. **Independence.** Every vertex, every triangle and every fragment can be processed without waiting for, or talking to, any other.
2. **Uniformity.** Within a stage, every item goes through exactly the same sequence of steps; only the data differ.
3. **Volume.** Millions of items must be processed for every frame, and a new frame is due every sixtieth of a second.

Together, these say that rendering is *data parallelism* (Section 2) on an enormous scale. This pattern has a name: **single program, multiple data**, or SPMD. One small program is written to handle *one* item, and it is run separately, and in any order, for every item. In graphics, such a program is called a **shader**, because its first job was to compute shading (colour).

Listing 2: The small program of Stage 3, written for *one* fragment (in GLSL, the shading language of OpenGL). It is run once for every fragment.

```
uniform sampler2D albedoMap; // texture: same for all fragments
uniform vec3 lightDir; // light direction: same for all fragments
in vec3 normal; // this fragment's facing direction
in vec2 uv; // this fragment's texture coordinates
out vec4 fragColor; // this fragment's result

void main() {
    vec3 base = texture(albedoMap, uv).rgb; // read texture colour
    float diffuse = max(dot(normalize(normal), -lightDir), 0.0);
    fragColor = vec4(base * diffuse, 1.0); // this fragment's colour
}
```

Listing 2 shows what such a program looks like for Stage 3. Read it as the recipe for colouring *one* fragment. It looks up the surface’s base colour in a texture picture (`texture(...)`), works out how directly the surface faces the light (the `dot` product of the facing direction and the light direction, never allowed below zero), and multiplies the two. The word `uniform` marks inputs that

are the same for every fragment, such as the light direction, while `in` marks inputs that differ from fragment to fragment, such as the facing direction. Notice what is *missing*: there is no loop over the pixels. The program describes one fragment, and it is simply run once for every fragment, as many at a time as the machine allows.

Takeaway: rendering is SPMD

Drawing a frame means running one small program independently on millions of items (vertices, triangles, fragments), dozens of times every second. A machine that does this well must be able to run the same program on a huge number of items at once. That is exactly what the graphics hardware of the following sections was built to do, and why the same hardware later turned out to be so good at problems that have nothing to do with pictures.

4 Fixed-Function Hardware: The Graphics Pipeline in Silicon

Section 3 described the work of rendering: a pipeline of three stages, each doing the same small calculation on millions of independent items, many times every second. For about two decades, from the 1980s to 1999, special graphics hardware did this work by building the pipeline directly into silicon: one block of circuits for each stage, each block wired to do its stage's calculation and nothing else. This section explains how such hardware worked, how it used the three properties of Section 3.7, and how all three stages came to sit on a single chip, the first GPU.

4.1 Why special hardware?

A general-purpose processor, the CPU, can certainly do this work, but it handles the items of a frame one after another, and that is far too slow for rendering. A screen of a million pixels redrawn 60 times per second, with a few dozen operations per pixel, already needs billions of operations per second, and every increase in screen size or visual quality multiplies that number. Throughout the 1980s and 1990s, CPUs became faster every year, yet they never caught up with rendering, because its demand for computation grew even faster. But the three properties of the work found in Section 3.7 point to a different kind of hardware.

First, every item in a stage goes through exactly the same calculation (*uniformity*). In Stage 1, for example, every vertex is multiplied by the same matrices. A CPU is built to run *any* program: for every instruction, it reads the instruction from memory, works out what it means, and only then carries it out (Section 2). Rendering does not need this flexibility. If a stage only ever does one calculation, that calculation can be wired directly into a circuit, which then has no program to read at all. Such a circuit is called **fixed-function** hardware: its function is fixed when the chip is made, and it cannot be programmed to do anything else. Because it does nothing else, it can be much smaller and faster than a CPU doing the same calculation.

Second, the items do not depend on each other (*independence*), and there are millions of them in every frame (*volume*). So there is no need to process them one after another: many fixed-function circuits can work on different items at the same time, and there is always enough work to keep all of them busy.

Graphics hardware was built on exactly these observations: build each stage of the rendering pipeline as fixed-function circuits, and use many of them at once. The next subsection shows what such a chip looks like.

4.2 How a fixed-function chip works

Figure 7 shows the plan of a fixed-function graphics chip of the late 1990s. It is simply the pipeline of Figure 5, built in hardware: a block of circuits for each stage, connected in the same order.

- **Stage 1 block (vertex processing).** Circuits that multiply each vertex by the 4×4 matrices and compute its lighting.
- **Stage 2 block (rasterisation).** A circuit that, given the three corners of a triangle, produces a fragment for every pixel the triangle covers.
- **Stage 3 blocks (fragment processing).** Circuits that look up the texture and compute the final colour of each fragment, and write it into the image stored in the graphics memory.

Each of the three properties of the work shows up in this plan.

- **Uniformity gives specialised blocks.** Each block is fixed-function hardware, built for its stage's calculation alone. It reads no instructions; it offers only a few settings to choose from, such as which of several lighting formulas to use.
- **Independence gives an assembly line.** Items flow from block to block like cars along an assembly line. While Stage 3 colours the fragments of triangle 1, Stage 2 is already rasterising triangle 2 and Stage 1 is processing the vertices of triangle 3 (Figure 7, bottom). No block waits for a triangle to leave the chip before starting on the next one, because no triangle needs the result of another. So all three blocks are busy at the same time. This way of working is called **pipelining**, and it is where the rendering pipeline gets its name.
- **Volume decides how many copies.** Every stage has many independent items, but not equally many. A single triangle covers many pixels, so a frame has far more fragments than vertices or triangles. The chip therefore spends its circuits where the work is. Stages 1 and 2 each get a single block, while Stage 3 gets several identical copies, called **pixel pipelines**, placed side by side, each colouring a different fragment at the same moment (Figure 7, right). Four pixel pipelines colour four fragments at a time. Later chips gave Stage 1 several copies too, but always fewer than Stage 3.

4.3 The first GPU: NVIDIA's GeForce 256

Before 1999, the graphics cards sold for ordinary PCs did only part of this work. They took over Stages 2 and 3, while Stage 1, transforming and lighting the vertices, was still done by the PC's CPU. Hardware for the whole pipeline existed, but only in expensive professional machines.

In 1999, NVIDIA released the GeForce 256, the first chip for ordinary PCs to hold all three stages [58]. Its plan is the one in Figure 7:

- **Stage 1:** a *transform-and-lighting engine*, fixed-function circuits that multiply every vertex by the matrices and compute its lighting;
- **Stage 2:** a *setup and rasterisation engine*, which takes each triangle and produces a fragment for every pixel it covers;
- **Stage 3:** four identical pixel pipelines, each with its own unit for looking up textures, together colouring four fragments in every clock cycle; at the chip's clock rate of 120 MHz, that is 480 million fragments per second;

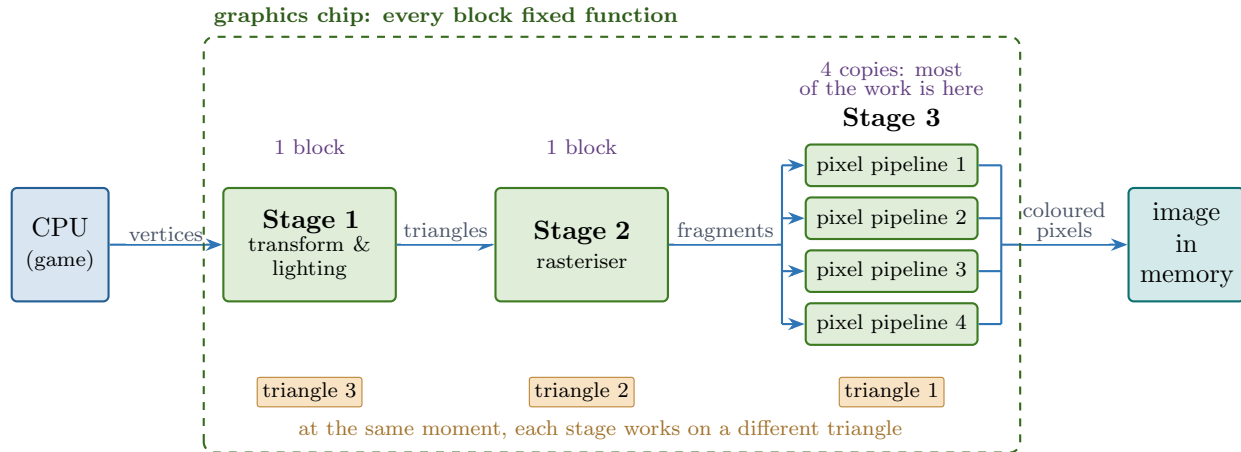


Figure 7: Plan of a fixed-function graphics chip of the late 1990s: the rendering pipeline of Figure 5 built in hardware. Each block is wired for its own stage’s calculation (uniformity). Each stage works on a different triangle at the same moment, like stations on an assembly line (independence). And the stage with the most work, Stage 3, has four identical copies side by side (volume).

- **memory:** its own fast memory, next to the chip, holding the textures and the image being drawn.

All of this fitted on one piece of silicon of about 17 million transistors, able to process at least 10 million triangles per second.

NVIDIA marketed the GeForce 256 as the first **Graphics Processing Unit**, or **GPU**, and defined the term as a single chip with integrated transform, lighting, triangle setup and rendering engines, able to process at least 10 million polygons per second [58]. The name reflects what had changed. Just as the CPU is the Central Processing Unit of a computer, the GPU was now a processing unit of its own, dedicated to graphics: the CPU only had to send it the description of the scene (vertices, textures, camera and lights), and the GPU did all three stages of rendering by itself and produced the finished image. The name stuck.

4.4 The one thing still missing: programmability

By 1999 the whole pipeline lived on one chip, but every block in it was fixed function. In the language of Section 3, the “small program” that each stage runs millions of times was wired into the hardware, and nobody could change it. If a game wanted a lighting effect the chip’s designers had not foreseen, it simply could not have it. Section 5 tells how that program became something programmers could write themselves.

Takeaway: the pipeline built in silicon

Fixed-function graphics hardware built the three-stage pipeline of Section 3 directly into silicon, and used all three properties of the work: a specialised block for each stage (uniformity), items flowing through the blocks like an assembly line (independence), and several copies of the busiest block side by side (volume). In 1999, NVIDIA’s GeForce 256 put the whole pipeline on a single chip, the first to be called a GPU. What it could not do was let anyone change the calculation each block performed.

5 Programmable GPU: Shaders Replace Fixed-Function Stages

Section 4 ended with a GPU in which every block was fixed function: the calculation of each stage was wired into the chip, and a programmer could only choose among a few settings. This section shows how, in the early 2000s, two of the three stages became programmable, so that the “small program” of Section 3.7 could at last be written by the programmer instead of by the chip’s designers.

5.1 From settings to programs

Fixed-function hardware can only produce the effects its designers foresaw. As games demanded more realistic materials, shadows and lighting, the number of settings kept growing, until the natural next step was to let the programmer write the calculation themselves.

In 2001, NVIDIA’s GeForce 3, together with version 8 of Microsoft’s DirectX graphics API, took this step for two of the three stages [37].

- **Stage 1** became fully programmable. The fixed transform-and-lighting engine was replaced by a small processor that runs a program supplied by the game: a **vertex shader**, run once for every vertex.
- **Stage 3** became programmable too, but at first only in a limited way. It runs a **fragment shader** (also called a pixel shader) once for every fragment, but early fragment shaders could be only a handful of instructions long.
- **Stage 2** stayed fixed function, as it still is today: finding the pixels that a triangle covers is the same job in every game, so there was little reason to make it programmable.

These first shaders were short. They were written in an assembly-like language, could be only about a dozen to at most 128 instructions long, and could hardly use `if` statements or loops. But they were real programs, chosen by the programmer.

5.2 How a programmable GPU works

Figure 8 shows the plan of a programmable GPU of the early 2000s. It is the plan of Figure 7 with two changes.

- **The Stage 1 and Stage 3 blocks are now processors.** Like a small CPU, each one reads instructions and carries them out, but it is built only to run shaders. The Stage 1 blocks, called **vertex processors**, run the vertex shader; the Stage 3 blocks, called **fragment processors**, run the fragment shader.
- **The game now sends programs as well as data.** Besides the scene (vertices, textures, camera and lights), the CPU sends the GPU the two shader programs to run.

The three properties of the work still shape the chip, exactly as in Section 4.2:

- **Uniformity.** Each programmable stage has its own shader, but *within* a stage every item runs the same one: every vertex runs the vertex shader, and every fragment runs the fragment shader. So each program is loaded once and then run millions of times.

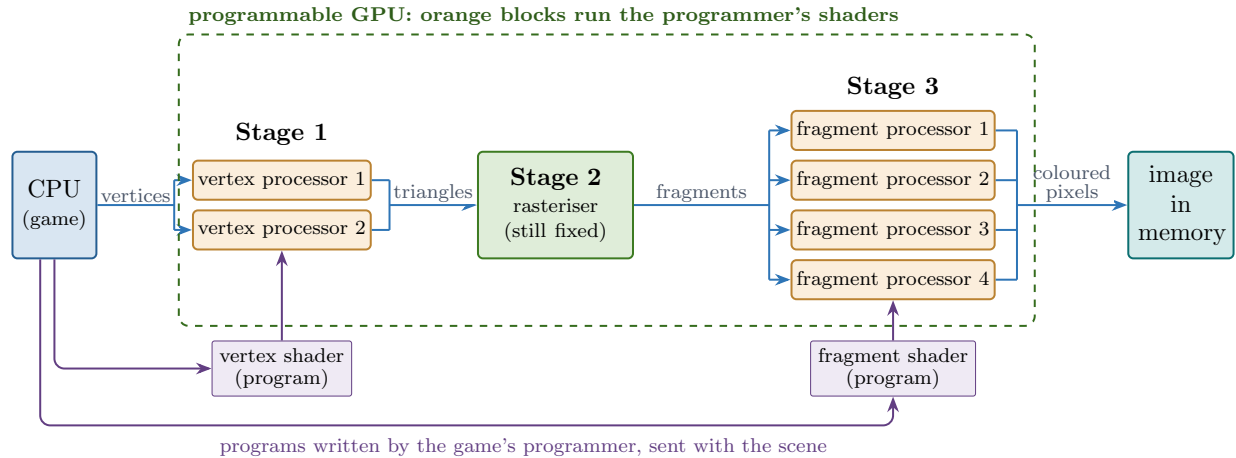


Figure 8: Plan of a programmable GPU of the early 2000s. It is the fixed-function plan of Figure 7 with two changes: the Stage 1 and Stage 3 blocks (orange) are now processors that run programs, the vertex shader and the fragment shader, which the game sends along with the scene; Stage 2 stays fixed function. As before, Stage 3, with the most work, has the most copies.

- **Independence.** No vertex or fragment needs the result of another. So the stages still work on different triangles at the same moment, and the processors of one stage can each run the shader on a different item at the same time.
- **Volume.** There are more fragments than vertices, so there are more fragment processors than vertex processors: ATI’s Radeon 9700 of 2002, for example, had four vertex processors and eight fragment processors.

5.3 Floating point and shading languages

The decisive step came a year later. Until then, fragment shaders worked with colours stored as 8-bit fixed-point numbers, 256 steps between 0 and 1 for each colour channel. That is enough to store a final colour, but too coarse for calculating realistic lighting, where small differences add up over many steps. In 2002, ATI’s Radeon 9700 and DirectX 9 brought *floating-point* arithmetic (Section 2) to fragment shaders, and NVIDIA’s GeForce FX followed with full 32-bit floating point [66]. A fragment shader could now compute with real numbers and store real numbers in a texture.

At about the same time, C-like **shading languages** appeared: NVIDIA’s Cg [42], Microsoft’s HLSL and OpenGL’s GLSL (Listing 2 is written in GLSL). Shaders now looked like small C functions, and the limits on their length rose with every generation.

5.4 What the GPU had become

Put these changes together and look at the GPU of the mid-2000s from the outside. It was a chip that

- ran a program written by the programmer, not by the chip’s designers;
- ran that program on millions of independent items at the same time, as in the SPMD model of Section 3.7;
- computed in floating point;

- had very fast memory of its own; and
- was cheap: a graphics card cost a few hundred dollars, far less than a parallel computer built for science, because the large market of PC games paid for its design and allowed it to be made in the millions.

These are the ingredients of a powerful parallel computer. Its one limitation was that it understood nothing but graphics: its input had to be vertices and textures, and its output was an image. Section 6 tells how people began to use it anyway, for problems that had nothing to do with pictures.

Takeaway: the pipeline opens up

In the early 2000s, the Stage 1 and Stage 3 blocks of the GPU became processors that run programs written by the programmer: vertex shaders and fragment shaders. Stage 2 stayed fixed function. With floating-point arithmetic and C-like shading languages, the GPU became a programmable machine that runs one program on millions of independent items at once.

6 The Birth of GPGPU

Section 5 ended with a GPU that was, in effect, a powerful parallel computer, but one that understood only graphics. This section tells how researchers began to use it for general calculations anyway, by disguising them as drawing. The practice became known as **GPGPU**, general-purpose computing on GPUs. The section ends with the limits of that disguise, which led to the redesign described in Section 7.

6.1 Computing disguised as drawing

Researchers noticed the machine described in Section 5.4 almost as soon as it appeared. As early as 2001, Larsen and McAllister multiplied matrices on a graphics card [36]. In 2002, Mark Harris coined the term **GPGPU** and later started a community web site under that name. Over the next few years, graphics chips were used to solve large systems of equations [16, 33], simulate fluids, process images, search databases and even, in simulation, trace rays of light [67]; Owens et al. surveyed this work in 2007 [66].

All of it relied on the same trick, sketched in Figure 9: general computation was disguised as drawing a picture. Suppose we have an input array A and want to compute an $N \times M$ output array C , where each element is $C[i][j] = f(A, i, j)$: some calculation f that may read any elements of A .

- **Arrays become textures.** The input array A is uploaded as a texture image, so that element $A[i][j]$ is stored as the “colour” of the texture pixel at position (i, j) .
- **The loop body becomes a fragment shader.** The calculation f is written as a fragment shader, which may read any element of A by looking up the texture.
- **The loop becomes a drawing command.** To run the shader for every (i, j) , the program asks the chip to draw one rectangle exactly $N \times M$ pixels in size. The rasteriser produces one fragment per pixel, so the hardware runs the shader once for every element, in parallel.

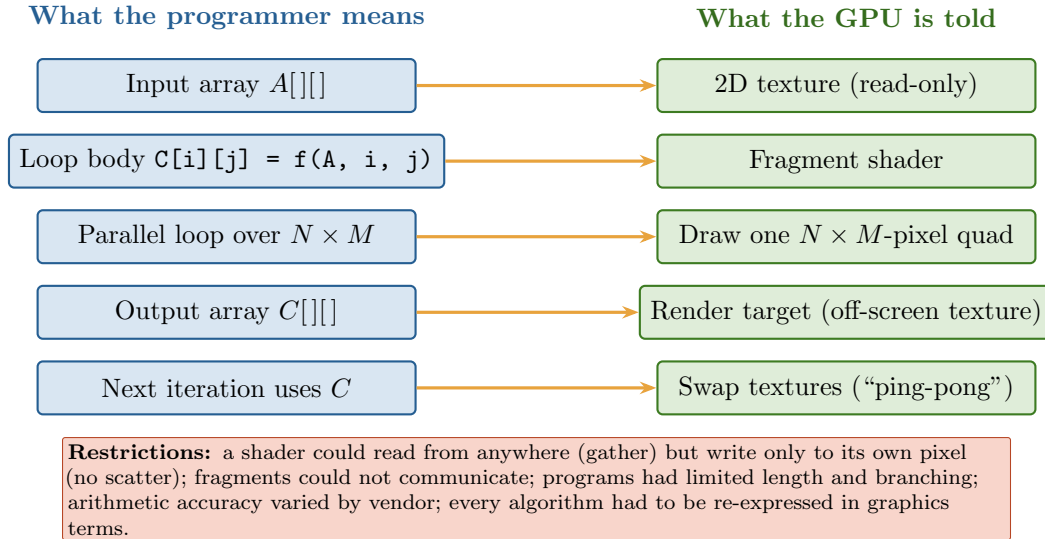


Figure 9: The “GPGPU through graphics” recipe used from about 2001 to 2006. Each general-purpose concept on the left was mapped onto a graphics concept on the right.

- **The output array becomes an image.** Each fragment’s “colour” is the computed value $C[i][j]$, written into an off-screen image instead of the display (**render to texture**). Algorithms that repeat a step many times alternate between two such images, reading from one and writing to the other (**ping-pong**).

6.2 Hiding the graphics

The trick worked, and it often delivered large speed-ups over the CPUs of the day, but it demanded fluency in graphics programming: a scientist who wanted to multiply matrices first had to learn about textures, shaders and render targets. Two research projects tried to hide this graphics machinery. Sh, from the University of Waterloo, let programmers write shaders from inside C++ [43]. Brook for GPUs, from Stanford, was more influential [17]. Brook let the programmer think in terms of *streams* of data and *kernels*, functions applied to every element of a stream, and its compiler translated these into textures and shaders automatically, so the programmer never saw a triangle. Brook’s ideas, and some of its people, went on to shape CUDA (Section 7).

6.3 The limits of computing through graphics

The disguise worked, but it had hard limits, listed at the bottom of Figure 9, and every algorithm had to be squeezed into the shape of a drawing. The hardware added one more problem. The GPUs of this period still mirrored the pipeline: one group of processors reserved for vertex shaders and a separate group reserved for fragment shaders, in a fixed ratio chosen by the designers (Figure 8). Even in graphics this caused a balancing problem. A scene made of very many tiny triangles keeps the vertex processors busy while the fragment processors sit partly idle; a full-screen effect does the opposite (Figure 10, left). For GPGPU it was worse still: computation ran only in fragment shaders, so all the vertex processors were wasted. The solution, described next, was to make all the processors identical, and to give them a language of their own.

Takeaway: the seed of GPGPU

GPGPU did not start with new hardware. It started when people realised that a fragment shader was really an SPMD program running on a massively parallel, floating-point machine, and that drawing a rectangle was really a way to launch that program once for every element of an array. Everything that came later made this hidden computer directly accessible.

7 Unification: The GPU Becomes a General-Purpose Processor

Between 2005 and 2010, GPUs were redesigned so that they could be programmed directly for general computing, without any graphics disguise. Two changes made this possible: identical processors for all kinds of work (Section 7.1), and a programming language built for computing rather than drawing (Section 7.2).

7.1 One kind of processor for everything

Section 6 ended with a problem: the GPU had two separate pools of processors, one for vertex shaders and one for fragment shaders, and whichever pool had less work sat partly idle (Figure 10, left).

The fix was to stop building two kinds of processor. A vertex shader and a fragment shader are both small programs, each run on many independent items; the only real difference between them is the program itself. So a single kind of processor, able to run *any* shader, can do both jobs. A GPU built this way has one pool of identical processors, and the hardware hands each piece of work, whether a vertex or a fragment, to whichever processor is free. This is called a **unified shader** design. As Figure 10 (right) shows, every processor is now useful, whatever the mix of work in the scene.

The first unified design appeared in a games console, Microsoft's Xbox 360 of 2005, whose graphics chip was designed by ATI. A year later, in November 2006, NVIDIA brought the idea to PCs with its G80 chip, sold as the GeForce 8800 [38].

NVIDIA designed the G80 with a second goal in mind: general-purpose computing. If every processor can run any program, it can also run a program that is neither a vertex shader nor a fragment shader, but an ordinary calculation. A GPGPU program no longer had to leave the vertex processors idle: the whole chip was available to it.

The G80 had 128 of these simple processors, organised in 16 groups of 8. Each group is called a **streaming multiprocessor**, or **SM**. The eight processors of an SM share its control logic and a small, fast memory, and together they work through the items given to the SM.

Whatever the program is, a vertex shader, a fragment shader or a GPGPU program, it is written for *one* item: one vertex, one fragment, or one element of the calculation. To process a million items, the GPU runs the program a million times, once for each item. Each of these runs has its own item being processed, its own current position in the program, and its own temporary values computed along the way, so it is exactly what Section 2.4 calls a **thread**. The mapping is one to one: *one item, one thread*.

An SM of the G80 could keep up to 768 threads in progress at the same time, many more than its eight processors could execute at any one moment. Section 8 explains why keeping so many threads ready is the key to a GPU's speed. The SM is still the basic building block of NVIDIA GPUs today, and Section 10 looks inside a modern one.

This change of view is captured in the title of this paper. The GPU stopped being a *pixel processor* and became a *thread processor*: a machine whose basic unit of work is a thread that may

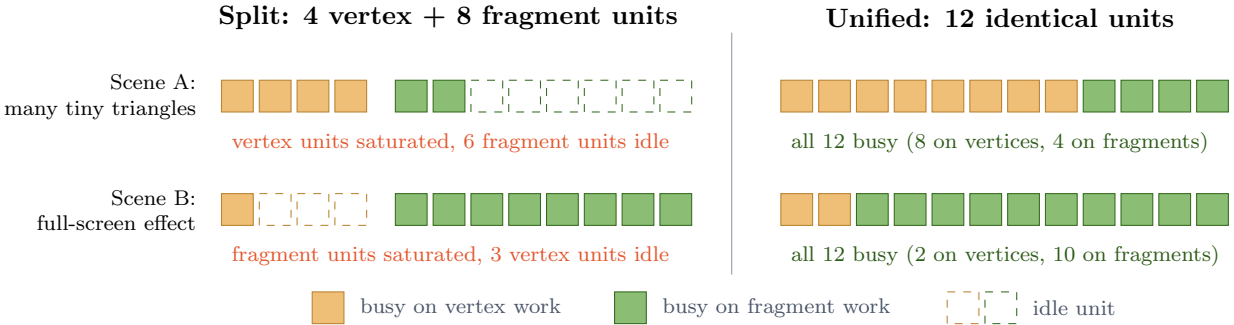


Figure 10: Why unification matters. With separate processor pools (left), whichever pool is not the bottleneck sits partly idle, and the ratio that suits one scene is wrong for another. With one pool of identical processors (right), the hardware assigns work dynamically and every unit stays busy. For general-purpose computing, unification also meant that *all* of the chip’s arithmetic was available to a general-purpose program.

perform any computation at all, colouring a pixel being just one example. To do that well, it was built to keep thousands of threads in progress at once; today’s GPUs keep hundreds of thousands.

7.2 CUDA, and the end of the graphics disguise

The hardware was now a thread processor, but programmers could still reach it only through a graphics API, using the disguise of Section 6. In 2007, NVIDIA removed that disguise by releasing **CUDA** [45]: an extension of the C language for writing general-purpose programs that run on the GPU, with no graphics concepts at all. In CUDA, the programmer writes an ordinary-looking C function called a **kernel**, which describes the work of *one* thread, exactly like a shader describes the work for one pixel. The program then asks the GPU to run that kernel on, say, a million threads at once, each thread working on its own data item, one item per thread as in Section 7.1. Unlike a fragment shader, which could write only the one pixel it was working on, a CUDA thread can write its result to any location in the GPU’s own memory. And threads can cooperate, by sharing data and by waiting for each other. (Section 9 explains all of this with a complete example.)

Other vendors followed. ATI (by then part of AMD) offered its own interfaces from 2006, and in 2008 the Khronos Group, the industry body behind OpenGL, published **OpenCL** [30], a vendor-neutral standard built on essentially the same ideas. In 2009, Microsoft added “compute shaders” to its Direct3D graphics API.

7.3 Maturing for science, then for AI

The next few generations of GPUs removed the remaining obstacles to serious scientific use. NVIDIA’s Fermi chips (2010) added proper caches (Section 2), memory that detects and corrects bit errors (essential when a simulation runs for days), and fast, standards-compliant 64-bit floating-point arithmetic. AMD, NVIDIA’s main competitor, redesigned its GPUs in 2012 around an architecture called **Graphics Core Next** (GCN), which made general-purpose code easier to compile [1]; Section 14 compares AMD’s designs with NVIDIA’s.

Also in 2012, Krizhevsky, Sutskever and Hinton used two gaming GPUs to train a neural network for recognising objects in photographs, and beat all previous methods by a wide margin [32]. Training a neural network consists overwhelmingly of multiplying large matrices, which is exactly the kind of uniform, data-parallel arithmetic GPUs excel at. Deep learning grew explosively, and

within a few years it became the most important use of GPUs; since about 2017, GPU designs have been shaped as much by neural networks as by graphics (Section 13).

7.4 One programming model, many languages

CUDA runs only on NVIDIA GPUs, and OpenCL, although supported by every vendor, never became as widely used. So each major GPU maker now offers its own language, and some languages aim to work across vendors:

- **CUDA** (NVIDIA, 2007), for NVIDIA GPUs only; still the most widely used, and the language used in this paper.
- **OpenCL** (Khronos Group, 2008), a vendor-neutral standard supported by NVIDIA, AMD, Intel and others [30].
- **HIP** (AMD, 2016), part of AMD’s ROCm software. HIP is deliberately almost identical to CUDA, so that CUDA programs can be converted to it with little effort, and HIP programs can also be compiled for NVIDIA GPUs [12].
- **SYCL** (Khronos Group, 2015), a vendor-neutral standard based on modern C++ [31]. It is the main language of Intel’s oneAPI software for Intel GPUs [26].
- **Metal** (Apple, 2014), for the GPUs in Apple’s phones, tablets and computers [15].

The important point is that all the languages listed above share one programming model: the programmer writes a kernel for one thread, one item per thread, and launches it on a very large number of threads. They differ mainly in their names for things. Section 9 explains the model using CUDA, and Table 4 translates its terms into several of the others.

These languages are *low level*: they stay close to the hardware. The programmer decides how the work is split into threads, how the threads are grouped, and where each piece of data is kept in the GPU’s memory. This gives full control over performance, but it takes expertise.

Higher-level languages take some of these decisions away from the programmer. For example, OpenAI’s Triton lets AI researchers write GPU kernels in Python [73, 74]. The programmer describes the calculation on whole blocks of data, and Triton’s compiler decides how to split it into threads and how to use the GPU’s memory. The same Triton program can be compiled for both NVIDIA and AMD GPUs. NVIDIA has since brought the same approach into CUDA itself: **cu-Tile**, introduced with CUDA 13.1 in December 2025, lets programmers write such block-level, or *tile*, kernels in Python for NVIDIA GPUs, while the compiler handles the division of the work into threads [52].

Table 2 sums up Sections 4 to 7: how the programmer’s view of the GPU changed, from a chip that could not be programmed at all to one that runs general-purpose programs written in CUDA and similar languages.

Table 2: How the programmer’s view of the GPU changed, from Section 4 to Section 7.

Period	Interface	What a programmer could do	Section
1999–2000	fixed function	Nothing: only built-in settings, no programs of their own.	Section 4
2001–2002	assembly shaders	Short vertex shaders; very short fragment shaders on 8-bit colours.	Section 5
2002–2006	Cg, HLSL, GLSL	Floating-point shaders written in C-like shading languages.	Section 5
2001–2006	shaders, Brook	General calculations, disguised as drawing; each result written only to its own pixel.	Section 6
2007–2009	CUDA, OpenCL	General-purpose kernels in a C-like language; one thread per item; write anywhere in GPU memory; threads can cooperate.	Section 7.2
2010 onwards	CUDA, OpenCL	The same, plus caches, error-correcting memory and fast 64-bit arithmetic for science.	Section 7.3
2014 onwards	HIP, SYCL, Metal	The same programming model as CUDA, now offered for the GPUs of the other major vendors: AMD, Intel and Apple.	Section 7.4
2021 onwards	Triton, cuTile	Higher level: kernels in Python on whole blocks (tiles) of data; the compiler decides threads and memory use.	Section 7.4

8 Three Design Ideas Behind Every GPU

This is the most important section of the paper. GPUs from different companies and different years differ in many details, but their overall shape follows from just three design ideas, first laid out clearly by Fatahalian and Houston [22]. Each idea gives up what a CPU is built for, finishing one thread as fast as possible (low *latency*), in exchange for what a GPU is built for, finishing as many threads as possible per second (high *throughput*). To see why, we start from an ordinary CPU core and change it one step at a time.

8.1 Starting point: a “fat” CPU core

A modern CPU core spends most of its transistors, not on doing arithmetic, but on making *one* thread run as fast as possible. It contains many kinds of clever machinery for this purpose; the four most important are:

- **Out-of-order execution.** The core looks ahead in the instruction stream for instructions that do not depend on the one currently waiting, and runs them early.
- **Branch prediction.** At an if statement, the core guesses which way the program will go and starts working on that path before the condition is even known. When the condition is finally computed, the core checks its guess. If the guess was wrong, it throws away the work done on the wrong path and restarts on the correct one.
- **Prefetching.** The core guesses which memory locations the program will need soon and fetches them in advance.

- **Large caches.** Several levels of cache, often many megabytes, keep recently used data close to the ALUs so that main memory is visited as rarely as possible.

Real CPU cores add many more techniques of the same kind, such as register renaming and running several instructions per cycle, but these four are enough to see what they have in common: every one of them exists to reduce *latency* (Section 2), the time to complete one instruction or one memory access. All this machinery takes up so much of the core that the ALUs themselves end up being only a small part of it (Figure 11, left). This is the right design when there is only one thread to run, or a few. It is the wrong design when there are a million independent threads waiting to run.

8.2 Idea #1: many slim cores

When a million threads are waiting, how fast any *single* thread finishes does not matter; what matters is how many threads finish per second. So the first idea is to remove everything that only helps a single thread go faster: the out-of-order logic, the branch predictor, the prefetcher and most of the cache. What remains is a simple core that executes instructions one after another, in order, and is perhaps a tenth of the size. The space saved on the chip is spent on *more cores*. Each core runs its own threads, all running the same program on different data, so ten simple cores finish roughly ten times as many threads, as long as there are enough threads to go round.

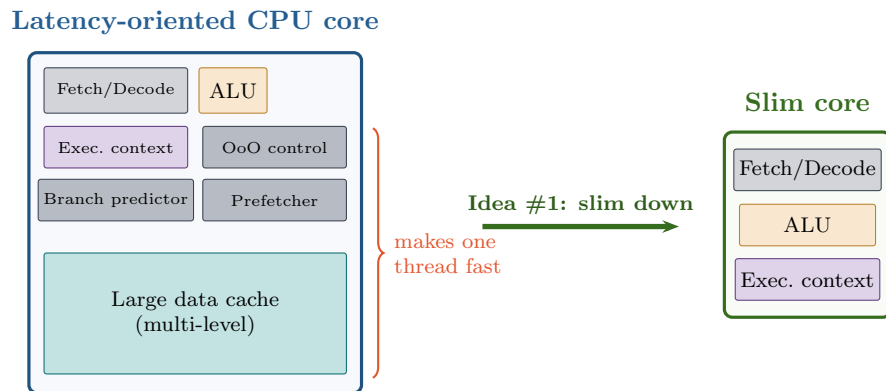


Figure 11: Idea #1. A CPU core devotes most of its area to features that accelerate a single instruction stream (grey and teal). Removing them leaves a slim core that runs one stream more slowly, but is small enough that many copies fit on the same die.

8.3 Idea #2: one instruction, many ALUs (SIMD)

Even a simple core spends a good share of its area and energy just fetching and decoding instructions. But neighbouring threads run the same program, so they execute the *same* instructions, only on different data. The second idea exploits this: keep the single fetch-and-decode unit of the slim core, but let it drive many ALUs at once, one for each of several threads (Figure 12). Each instruction is fetched and decoded once and then carried out by W ALUs side by side, each for its own thread, on that thread's own data. This is the SIMD (vector) idea from Section 2, and the cost of control is now shared among W threads. GPUs use much wider SIMD than CPUs, 16 to 64 lanes rather than 4 to 16, because data-parallel programs naturally supply that many threads executing the same instruction.

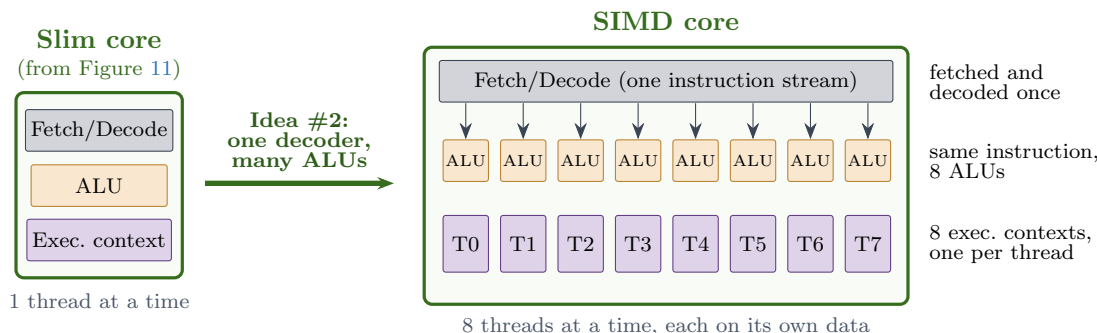


Figure 12: Idea #2. Start from the slim core of Figure 11 and keep its single Fetch/Decode unit, but give it eight ALUs and eight execution contexts, one for each of eight threads (T0 to T7). Each instruction is fetched and decoded once and then carried out by all eight ALUs, each for its own thread on that thread's own data, so the cost of fetching and decoding is shared by eight threads.

There is an important subtlety. A shader or a CUDA kernel is written for *one* thread, and it may contain *if* statements. If different threads in the same SIMD group go different ways at an *if*, they can no longer share one instruction at every step. GPUs handle this automatically, as Section 11 explains; for now it is enough to know that SIMD works best when neighbouring threads follow the same path through the code.

8.4 Idea #3: hide memory delays by switching between threads

Ideas #1 and #2 create a serious problem. We threw away the large caches and the prefetcher, the very machinery that protected a CPU core from slow memory. A load from main memory still takes hundreds of cycles. A slim core that simply waited for each load would spend most of its time doing nothing.

The third idea is to *tolerate* the delay instead of trying to avoid it. Each core keeps several independent groups of threads ready at the same time, with all of their registers held on the chip. When the group that is running has to wait for memory, the core immediately switches to another group that is ready, and because everything that group needs is already on the chip, the switch costs nothing. Figure 13 shows the effect. While one group waits for memory, the others keep the ALUs busy. Notice what this does and does not improve. No single group finishes any sooner: each one still has to wait for its memory loads, and it may even wait a little longer, because when its data arrives another group may be using the ALUs. What improves is the core as a whole. Its ALUs are almost never idle, so the core completes many more threads per second than if it waited for every load. In other words, switching between groups does not reduce latency; it raises *throughput*. It is the same trick a good cook uses: while one dish is in the oven, prepare the next one rather than stand and watch the oven.

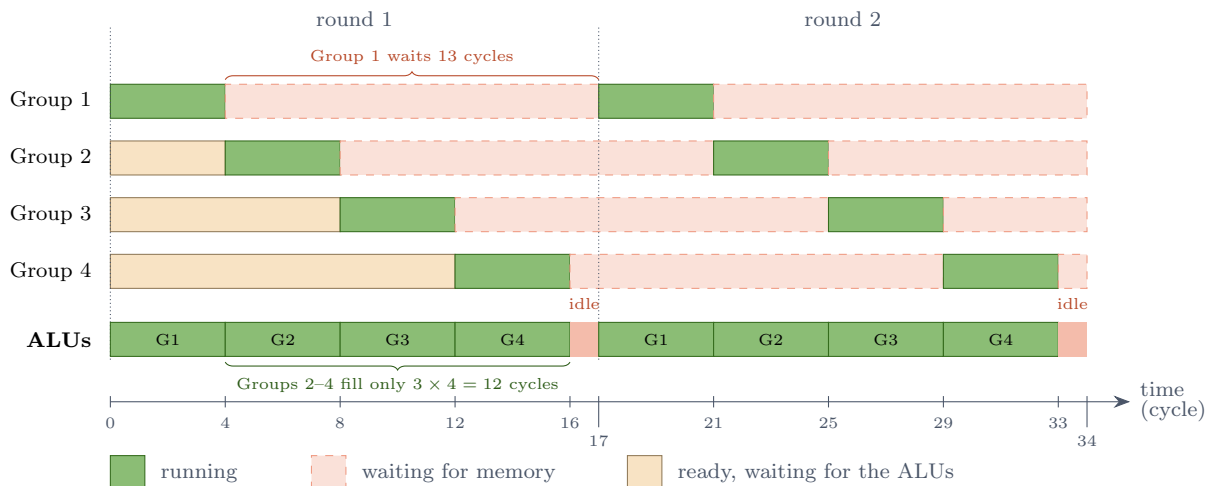


Figure 13: Idea #3. Four groups of threads share one core. To keep the picture small, this example assumes that each group runs for 4 cycles and then waits 13 cycles for a memory load (red); on a real GPU the wait is hundreds of cycles. Instead of idling, the core switches to another group that is ready. At the start, all four groups are ready at once, so groups 2 to 4 must wait their turn (yellow). In round 2, each group’s data arrives just as the ALUs become free, so no group waits in the ready state. The ALUs (bottom row) are busy almost all the time, except for one idle cycle per round. That cycle comes from simple arithmetic: while Group 1 waits 13 cycles for its data, the other three groups can keep the ALUs busy for only $3 \times 4 = 12$ cycles, so for one cycle no group is ready to run.

How many groups are needed to keep the ALUs busy all the time? Suppose each group can do C cycles of arithmetic before it has to wait for a memory access that takes L cycles. While one group waits for L cycles, the core must keep its ALUs busy with other groups, each of which supplies C cycles of arithmetic, so it needs about L/C other groups. So roughly

$$N_{\text{groups}} \gtrsim 1 + \frac{L}{C} \quad (1)$$

groups must be ready to keep the core busy. In Figure 13, $C = 4$ and $L = 13$, so Equation (1) asks for $1 + 13/4 \approx 4.25$ groups; four are almost, but not quite, enough, which is exactly the one idle cycle per round in the figure. On real GPUs, L is in the hundreds of cycles and C is often in the tens, so each core needs dozens of groups ready at once.

Idea #3 has a price, and it explains one of the most surprising features of a GPU: its very large number of registers.

Recall that registers are the small, fast storage slots next to the ALUs, where a thread keeps the values it is working on (Section 2.2). When a CPU switches from one thread to another, it must first copy the old thread’s registers out to memory and then copy the new thread’s registers in (Section 2.4). This takes thousands of cycles, far too long to do after every memory load.

A GPU avoids this copying altogether. Every thread that is kept ready keeps its own registers on the chip the whole time, so switching to another group only means letting the ALUs work from a different set of registers, which costs nothing. The price is size: a GPU core must hold the registers of *all* its ready threads at once. A modern NVIDIA GPU core, which NVIDIA calls a streaming multiprocessor or SM (Section 7.1), can for example keep 2,048 threads ready and has 65,536 registers, 256 KB in total, about as much storage as its cache [61]. A CPU core has only a few kilobytes of registers.

This also leads to one of the most important rules for making GPU programs fast. The register

storage of a core is fixed, so the more registers each thread needs, the fewer threads fit on the core, and the less memory delay can be hidden. Section 12.1 returns to this rule.

8.5 Putting the ideas together

Figure 14 builds a small, imaginary GPU from the three ideas.

- **Idea #1:** the chip has 16 slim cores.
- **Idea #2:** each core has one Fetch/Decode unit driving 8 ALUs, so it runs a *group* of 8 threads with one instruction stream, one thread per ALU.
- **Idea #3:** each core keeps 4 such groups ready, as in Figure 13, and switches between them whenever the running group waits for memory. Every thread in the 4 groups keeps its own registers on the chip, so each core holds $4 \times 8 = 32$ threads.

In every cycle, each core runs one instruction for one group, so the chip performs $16 \times 8 = 128$ operations per cycle. To hide memory delays, it keeps $16 \times 32 = 512$ threads on the chip at once.

Real GPUs follow exactly this recipe at a much larger scale: an NVIDIA B200 has 148 cores (which NVIDIA calls streaming multiprocessors), each able to keep 2,048 threads ready, for more than 300,000 threads on the chip at once [20, 61]. (NVIDIA’s product sheets also speak of thousands of “CUDA cores”; a CUDA core is a single ALU lane, not a core in the sense used in this paper.)

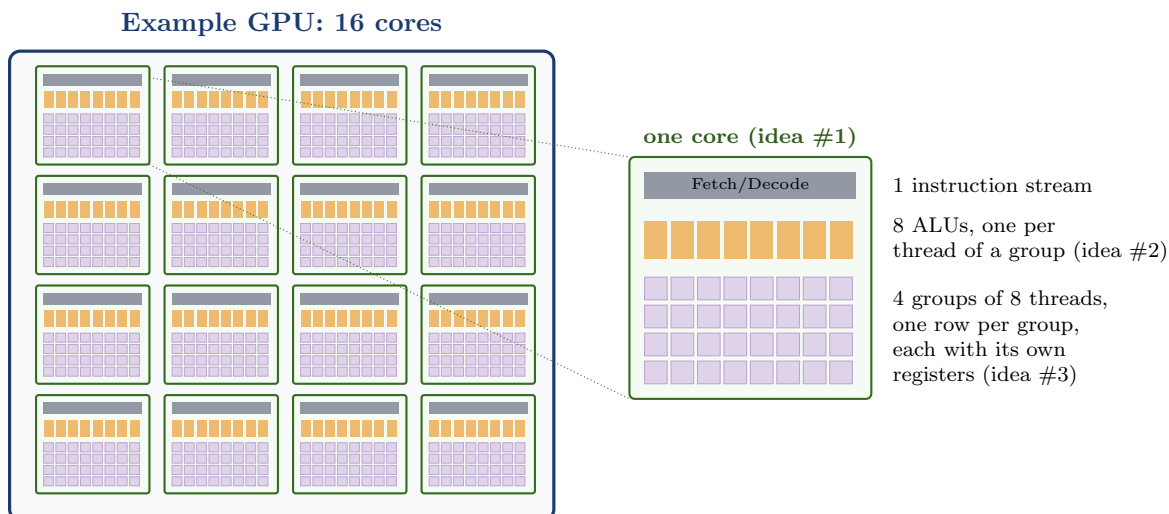


Figure 14: A toy GPU built from the three ideas: $16 \text{ cores} \times 8 \text{ ALUs} = 128$ operations per clock, and $16 \text{ cores} \times 4 \text{ groups} \times 8 \text{ threads} = 512$ threads kept on the chip to hide memory delays. Real GPUs differ only in scale and in the many refinements discussed in Section 10.

8.6 The result: a GPU next to a CPU

The three ideas produce a machine that is the opposite of a CPU in almost every respect. A CPU is **latency-oriented**: it is built to finish each thread as soon as possible. A GPU is **throughput-oriented**: it is built to finish as many threads per second as possible, and accepts that each one takes longer [23]. Table 3 puts the two side by side, using NVIDIA’s B200 as the example GPU.

Table 3: Design contrasts between latency-oriented CPUs and throughput-oriented GPUs. Figures are indicative of current server-class parts; the GPU column uses the NVIDIA B200 as a concrete example [20, 63].

	CPU (latency-oriented)	GPU (throughput-oriented)
Goal	Finish each thread quickly	Finish as many threads per second as possible
Cores	Tens to over a hundred large cores	100+ slim cores (148 on B200)
Threads kept ready per core	1 or 2	Up to 2,048
SIMD width	4–16 lanes, used explicitly	32 lanes, used automatically (Section 11)
Dealing with slow memory	Avoid waiting: caches, prefetching, out-of-order execution	Hide waiting: switch to other threads
Switching threads	By the operating system, microseconds	By hardware, every cycle, free
On-chip storage	Mostly cache (megabytes per core)	Large register file (256 KB per core); 126 MB shared cache
Memory bandwidth	A few hundred GB/s	About 8 TB/s (stacked HBM memory)
if statements	Cheap	Costly when neighbouring threads disagree
Best at	Irregular, step-by-step, branchy code	Regular, massively data-parallel code

Neither design is better in general; each is right for a different kind of program. A GPU pays off only when a program offers a very large number of threads: a GPU that can keep 300,000 threads ready is badly wasted on a problem with only ten thousand. The parts of a program that must run step by step, one after another, are better left to the CPU.

Real systems therefore use both. The CPU, called the **host**, runs the overall program and hands the data-parallel parts to the GPU, called the **device**. If a large share of the program cannot run in parallel, that share limits the overall speed-up, however fast the GPU is (a rule known as Amdahl’s law). And because a separate GPU card has its own memory, connected to the CPU by a link that is much slower than either memory, good programs copy their data to the GPU once and keep it there for many operations. Section 9 shows how such a program is written.

Takeaway: the three core ideas behind GPU design

(1) Use *many simple cores* instead of a few complex ones. (2) Let *one instruction drive many ALUs* at once (SIMD). (3) Keep *many threads ready on the chip* and switch between them for free, so that the ALUs keep working while some threads wait for memory. Almost everything else in this paper, from the way GPU programs are written to the rules for making them fast, follows from these three ideas.

9 How a GPU Program Is Written

This section shows what a GPU program looks like and how its work is divided up. We use CUDA, NVIDIA’s language, because it is the most widely used; the other GPU languages (Table 4) follow the same ideas with different names.

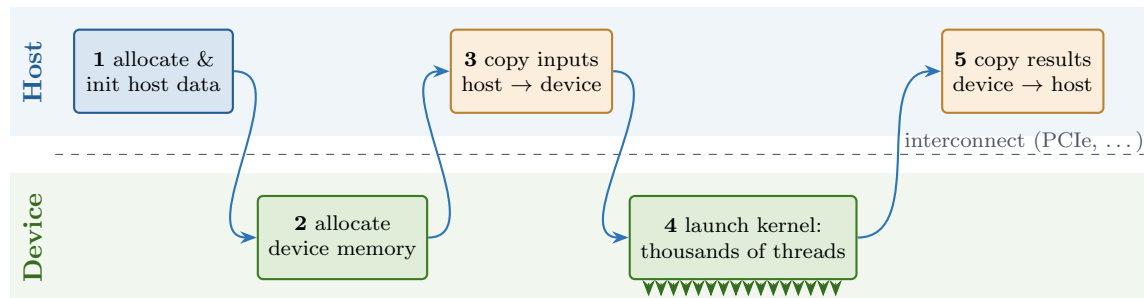


Figure 15: The five steps of a simple GPU program. Steps 3 and 5 copy data across the relatively slow link between the CPU and the GPU card, so good programs run many kernels (step 4) on the data before copying it back. A kernel launch does not make the CPU wait: the host can carry on with other work while the GPU computes.

9.1 Host, device and kernels

A GPU program always has two parts. The **host** part runs on the CPU: it is an ordinary program that reads input, prepares data and decides what to do. The **device** part runs on the GPU and does the heavy, data-parallel work. The device part is written as a **kernel**: a function that describes what *one* thread does, just as a shader (Section 3) describes what happens to one pixel. The host then *launches* the kernel, asking the GPU to run it on a very large number of threads at once, each working on a different piece of the data. This is the SPMD idea of Section 3.7, now available for any calculation:

- every thread runs the same kernel;
- every thread gets a unique number, its thread index;
- each thread uses its index to choose which part of the input to read and which part of the output to write; and
- each thread may take its own path through the kernel (for example at an `if`), depending on its data.

A typical GPU card has its own memory, separate from the CPU’s. A host program therefore follows the five steps of Figure 15: prepare the data, reserve memory on the GPU, copy the inputs there, launch the kernel, and copy the results back.

9.2 Organising threads: grids and blocks

A host program can launch a kernel with, say, a million threads. Rather than keeping them in one flat list, CUDA organises them in two levels, called the **thread hierarchy** (Figure 16); recent GPUs add an optional third level, described at the end of this subsection.

- Threads are grouped into **thread blocks** of up to 1,024 threads, arranged in one, two or three dimensions (a 2D image, for instance, maps naturally onto 2D blocks). All threads of a block run on the same GPU core at the same time, so they can cooperate: they can exchange data through a small, fast on-chip memory called **shared memory**, and they can wait for one another at a **barrier** (the function `__syncthreads()`), a point that no thread of the block passes until all have reached it.

- Blocks are in turn arranged into a **grid**, again in one to three dimensions. The blocks of a grid must be *independent*: they may run in any order, at the same time or one after another, and cannot wait for each other.

Each thread can ask which block it belongs to (`blockIdx`), how big the blocks are (`blockDim`), and its own position inside its block (`threadIdx`). From these it computes its unique position in the whole grid. In one dimension,

$$gid = blockIdx.x \times blockDim.x + threadIdx.x. \quad (2)$$

For example, with blocks of 256 threads, thread 5 of block 3 has global index $3 \times 256 + 5 = 773$, so it handles element 773 of the data. The same formula applies separately in the y and z directions.

Why two levels?

The split is a promise in both directions. Threads in the same block are guaranteed to run together on one core, so they can share fast memory and wait for each other cheaply. Blocks are promised nothing about each other, so the hardware is free to run them on any core, in any order. This freedom is what lets the same program run unchanged on a small GPU with a handful of cores, which works through the blocks a few at a time, and on a large GPU with over a hundred cores, which runs many more blocks at once. NVIDIA calls this **transparent scalability** [45].

A third, optional level: clusters.

Since NVIDIA’s Hopper GPUs of 2022, CUDA offers an optional extra level between the grid and the block, the **thread block cluster**. A cluster is a small group of blocks, at most 8 in portable code, that the hardware guarantees to run at the same time on neighbouring cores. That guarantee allows the blocks of a cluster to cooperate a little: threads of one block can read and write the shared memory of the other blocks in the same cluster, and all blocks of a cluster can wait for each other at a cluster-wide barrier [64]. Different clusters are still independent of each other, so transparent scalability is kept. Clusters are used mainly to speed up the large matrix calculations of AI (Section 13); the rest of this paper needs only grids and blocks.

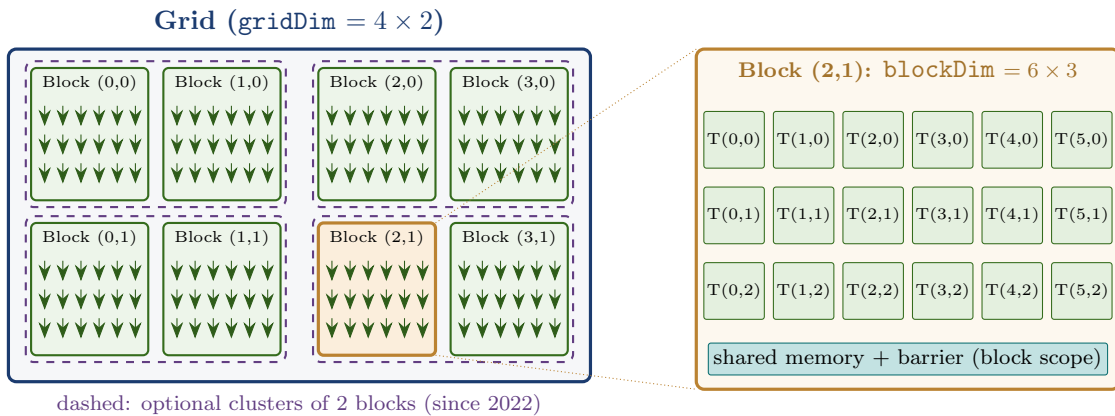


Figure 16: The thread hierarchy. A grid is made of independent thread blocks; each block is made of threads that can share memory and synchronise. A thread’s global position follows from its block index, the block dimensions and its index within the block. On recent GPUs, blocks can optionally be grouped into clusters (dashed, here of two blocks each) whose blocks run at the same time and can use each other’s shared memory.

9.3 A first kernel

Listing 3 is the GPU equivalent of “hello, world”: adding two arrays (vectors) element by element, $c[i] = a[i] + b[i]$.

Reading it from the top:

- **The kernel.** `__global__` marks `vec_add` as a kernel: a function that runs on the GPU and is launched from the host. Inside it, each thread computes its global index with Equation (2) and adds one pair of elements. The `if (gid < n)` check is needed because the number of threads launched is rounded up to a whole number of blocks, so the last block may contain a few threads with no element to work on.
- **The host program** follows the five steps of Figure 15 in order. By convention, names starting with `h_` point to host (CPU) memory and names starting with `d_` point to device (GPU) memory.
 - *Step 1* is ordinary C: `malloc` reserves CPU memory and a loop fills in the inputs.
 - *Step 2*: `cudaMalloc` reserves memory on the GPU.
 - *Step 3*: `cudaMemcpy` copies the inputs from CPU memory to GPU memory.
 - *Step 4*: the line `vec_add<<<grid, block>>>(...)` launches the kernel. The two numbers between `<<<` and `>>>` give the shape of the launch: `block` is the number of threads in each block (here 256), and `grid` is the number of blocks. With $n = 2^{20} = 1,048,576$ elements and 256 threads per block, the program needs $1,048,576/256 = 4,096$ blocks, one thread per element. The formula $(n + \text{block} - 1) / \text{block}$ rounds this division *up*, so that there are enough blocks even when n is not a multiple of 256; any extra threads in the last block are the ones that the `if (gid < n)` check turns away.
 - *Step 5*: `cudaMemcpy` copies the result back. It automatically waits until the kernel has finished, so the CPU never reads an incomplete result.

There is no loop over the elements in the kernel. As with the shader of Listing 2, that loop has been replaced by launching one thread per element.

9.4 Where data lives: memory spaces

A kernel can use several kinds of memory, each corresponding to a physical place on the GPU (Figure 17).

- **Registers** are private to each thread and are the fastest storage. The compiler decides how many each thread needs, and that number directly limits how many threads can be kept on a core at once (Section 12.1).
- **Shared memory** is a small, fast memory inside each GPU core (tens to a couple of hundred kilobytes), shared by the threads of one block. The program manages it explicitly, using it to exchange data between the threads of a block or as a fast, hand-managed store: if the threads of a block need the same data from global memory many times, the block can load it into shared memory once and then read it from there, which is much faster than going back to global memory each time.

Listing 3: Vector addition in CUDA: the kernel (top) and a complete host program (bottom), with the five steps of Figure 15 marked.

```

1  // ===== Device code: runs on the GPU, once per thread =====
2  __global__ void vec_add(const float *a, const float *b, float *c, int n)
3  {
4      int gid = blockIdx.x * blockDim.x + threadIdx.x;    // global index
5      if (gid < n)                                         // guard the tail
6          c[gid] = a[gid] + b[gid];
7  }
8
9  // ===== Host code: runs on the CPU =====
10 int main()
11 {
12     int    n    = 1 << 20;                               // about one million elements
13     size_t bytes = n * sizeof(float);
14
15     // Step 1: allocate and initialise the input data on the host
16     float *h_a = (float *)malloc(bytes);
17     float *h_b = (float *)malloc(bytes);
18     float *h_c = (float *)malloc(bytes);
19     for (int i = 0; i < n; i++) { h_a[i] = i; h_b[i] = 2 * i; }
20
21     // Step 2: allocate memory on the device (the GPU)
22     float *d_a, *d_b, *d_c;
23     cudaMalloc(&d_a, bytes);
24     cudaMalloc(&d_b, bytes);
25     cudaMalloc(&d_c, bytes);
26
27     // Step 3: copy the inputs from host to device
28     cudaMemcpy(d_a, h_a, bytes, cudaMemcpyHostToDevice);
29     cudaMemcpy(d_b, h_b, bytes, cudaMemcpyHostToDevice);
30
31     // Step 4: launch the kernel, one thread per element
32     int block = 256;                                       // threads per block
33     int grid  = (n + block - 1) / block;                  // enough blocks to cover n
34     vec_add<<<grid, block>>>>(d_a, d_b, d_c, n);
35
36     // Step 5: copy the result from device to host
37     cudaMemcpy(h_c, d_c, bytes, cudaMemcpyDeviceToHost);
38
39     // Clean up: free the device and host memory
40     cudaFree(d_a); cudaFree(d_b); cudaFree(d_c);
41     free(h_a);    free(h_b);    free(h_c);
42     return 0;
43 }

```

- **Global memory** is the large main memory of the GPU card (tens to hundreds of gigabytes), visible to every thread and to the host. It is slow to reach, so it is backed by caches: a small **L1** cache in each core and a large **L2** cache shared by all cores. NVIDIA’s data-centre GPUs stop there: there is no L3 cache (the H100’s L2 holds 50 MB). The blocks of a cluster (Section 9.2) share data through each other’s shared memory, not through an extra cache level. Some other GPUs do add a third level: AMD’s MI300X, for example, places a 256 MB *Infinity Cache* between its smaller L2 caches and main memory [19].
- **Constant** and **texture** memory are special read-only ways of accessing global memory, inherited from graphics, with their own small caches.

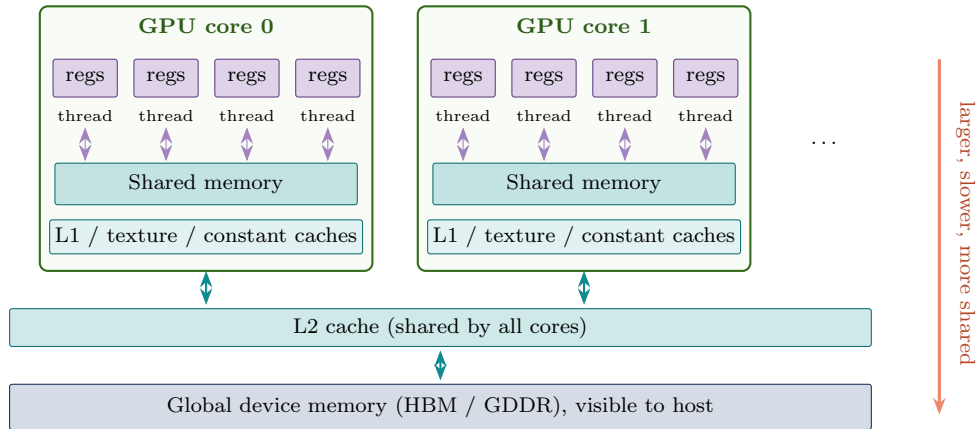


Figure 17: Where a GPU program’s data lives. Registers belong to one thread, shared memory to one block (it physically sits inside one core), and global memory to everyone, including the host. Between the cores and global memory sit two levels of cache, which the hardware manages automatically: a small L1 cache in each core (together with small caches for constant and texture data) and a large L2 cache shared by all cores. They keep copies of recently used global-memory data, so that repeated reads need not go all the way to global memory. Each step down is larger, slower (often by ten times or more) and shared more widely.

9.5 One model, many names

The same model, a grid of independent blocks of cooperating threads, is used by every low-level GPU programming language of Section 7.4: AMD’s HIP, the SYCL standard used by Intel, Apple’s Metal and the vendor-neutral OpenCL. Only the names differ. Table 4 translates between them; the “SIMT group” column refers to the group of threads (32 on NVIDIA GPUs) that the hardware actually executes together, explained in Section 11.

10 Inside a Modern NVIDIA GPU

We now open up a real GPU and see how the three ideas of Section 8 look in actual hardware. Our running example, here and for the rest of the paper, is NVIDIA’s Hopper H100 (2022). We describe the H100 because NVIDIA has documented its design in full [61]. Its successor, the Blackwell B200 (2024), keeps the same SM design, with more SMs and a few additions; where it differs, we say so [20, 27, 63]. We go from the outside in: first the whole chip, then how thread blocks are handed out, then one core, and finally how a core chooses what to execute on every cycle.

Table 4: Terminology across programming models and vendors.

	Whole launch	Group	SIMT group	Instance	Core
CUDA / NVIDIA	grid	thread block	warp (32)	thread	streaming multiprocessor (SM)
HIP / AMD	grid	block (work-group)	wavefront (64 or 32)	thread (work-item)	Compute Unit (CU)
SYCL / Intel	nd_range	work-group	sub-group	work-item	Xe-core (on Intel GPUs)
Metal / Apple	grid	threadgroup	SIMD-group (32)	thread	GPU core
OpenCL	NDRange	work-group	sub-group	work-item	compute unit

Before we start, we need one new term. In the programming model of Section 9, every thread looks independent: each one runs the kernel on its own data, as in the SPMD model of Section 3.7. The hardware, however, does not run each thread on its own, separately from the others. Instead, it runs them in fixed groups of 32 consecutive threads called **warps**. The 32 threads of a warp execute the same instruction together, each on its own data: this is the “one instruction, many ALUs” of idea #2. NVIDIA calls this way of executing threads SIMT, for *single instruction, multiple threads*.

SPMD and SIMT are two views of the same program.

- SPMD is the *programmer’s* view: one program, written for one thread and run by many threads, each thread separately on its own data item. Nothing in the program mentions warps.
- SIMT is the *hardware’s* view: when the kernel is launched, the hardware takes 32 threads at a time and puts them into a warp; the programmer does not do this. A warp has a single instruction stream: the hardware fetches and decodes each instruction once and then carries it out for all 32 threads of the warp at the same time, one thread per ALU, just as a SIMD instruction does. What makes this different from ordinary SIMD is who has to think about it. With ordinary SIMD, the programmer writes vector instructions that work on several data items at once. With SIMT, the programmer writes code for one thread only, and the hardware does the grouping. Each of the 32 threads of a warp keeps its own registers and its own data, so, seen from inside the program, every thread still behaves as if it were running on its own, exactly as the SPMD view promises.

The two views part company only when the threads of one warp want to do different things, for example when they go different ways at an `if` statement. A warp has only one instruction stream, so the hardware must then run the two paths one after the other. This is called **branch divergence**, and it is the main cost of the SIMT approach [68]. Section 11 explains SIMT and divergence in detail; for now, think of a warp as the unit that the hardware schedules.

10.1 The whole chip

Figure 18 shows the top-level layout of an H100: the GPU itself, its memory, and its links to the host and to other GPUs.

- The basic building block, the GPU “core” of Section 8, is the **streaming multiprocessor** (SM). An SM holds thread blocks, decides which warp runs next and executes instructions.
- SMs are grouped for practical reasons: two SMs form a **texture processing cluster** (TPC), and nine TPCs form a **graphics processing cluster** (GPC). The full chip has 8 GPCs and therefore 144 SMs; a few are switched off to tolerate manufacturing defects, so the H100 product uses 132 [61].
- The host does not send kernels to the SMs directly. It places kernel launches and data copies in queues called **CUDA streams**. Within one stream, launches and copies are carried out in the order they were queued; those in different streams may run at the same time. The launches reach the GPU over the PCIe link, and a unit called the **GigaThread engine** takes them from the streams and hands out their thread blocks to the SMs.
- All SMs share a large **L2 cache** (50 MB on H100), which sits in front of the GPU’s main memory. That memory is **HBM** (high-bandwidth memory): piles of memory chips stacked on top of each other and placed right next to the GPU die inside the same package. The H100 has five such stacks, holding 80 GB and delivering about 3.35 TB/s.
- Separate high-speed links, called **NVLink**, connect the GPU directly to other GPUs, so that large AI models can be spread over many GPUs.

How NVIDIA GPUs grow from one generation to the next.

Table 5 lists four generations of NVIDIA’s data-centre GPUs. Read down the columns and a pattern appears. The number of SMs grows with every generation, from 80 on the V100 to 148 on the B200. The L2 cache grows from 6 MB to 126 MB, and the memory bandwidth from about 1 TB/s to 8 TB/s. The SM itself, however, changes much more slowly. Its basic plan of four processing blocks, 32 FP32 units per processing block, 65,536 registers and up to 2,048 threads has stayed the same since the V100. Each generation adds a few new features inside the SM, listed in the last column of the table, but it does not redesign the SM from scratch.

There are two reasons for growing this way. First, an SM is a proven building block, and placing more copies of it on the chip is the simplest way to get more throughput: this is idea #1 of Section 8 again. Second, programs do not have to change. Because the blocks of a grid are independent (Section 9), the hardware may run them on any SM in any order. A kernel launched with thousands of blocks, such as the 4,096 blocks of Listing 3, therefore runs unchanged on a GPU with 80 SMs or with 148 SMs; the bigger GPU simply runs more blocks at the same time and finishes sooner.

Adding SMs has a limit, though. A single piece of silicon cannot be made larger than about 800 mm², because the machines that print chips cannot expose a larger area at once, and the H100 is already close to that size. Blackwell’s B200 therefore takes a different step. It is built from two separate pieces of silicon, called **dies**, placed side by side in one package. Each die has 80 SMs, of which 74 are switched on, for 148 SMs in total. The two dies are joined by a link so fast, about 10 TB/s, that software sees them as a single GPU [20, 63]. Its successor, Rubin, again uses two dies, together with a newer generation of memory, HBM4 [71].

10.2 Handing out thread blocks

When the host launches a kernel, the GigaThread engine goes through the thread blocks of the kernel’s grid and assigns each one to an SM that has room for it (Figure 19). “Room” means

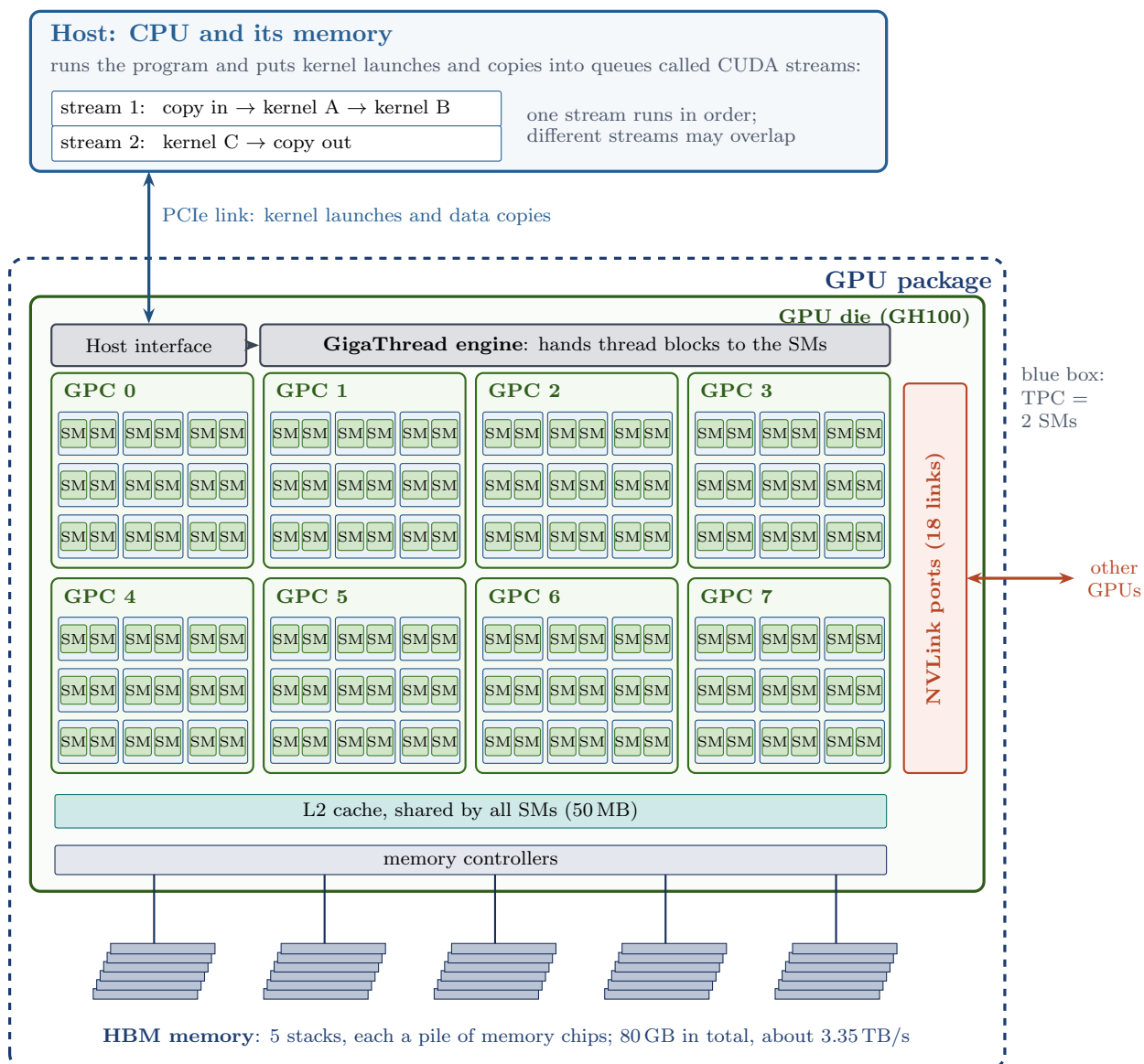


Figure 18: Organisation of an NVIDIA H100. Outside the GPU, the host CPU runs the program and places kernel launches and data copies in queues called CUDA streams; these reach the GPU over the PCIe link. Inside the GPU package sit the GPU die and its main memory. On the die, the GigaThread engine hands thread blocks to the streaming multiprocessors (SMs), which are grouped in pairs into texture processing clusters (TPCs, blue) and nine TPCs into a graphics processing cluster (GPC): 8 GPCs and 144 SMs on the full die, of which 132 are enabled. All SMs share the L2 cache, behind which memory controllers connect to the HBM stacks, each a pile of memory chips placed right next to the die. NVLink ports connect the GPU directly to other GPUs. Blackwell’s B200 places two such GPU dies in one package, with 148 SMs enabled in total, and presents them to software as a single GPU.

Table 5: NVIDIA data-centre GPUs used as running examples (published figures for the SXM variants).

GPU (year)	SMs	L2	Memory	B/W	New in the SM
V100 (2017)	80	6 MB	16/32 GB HBM2	0.9 TB/s	tensor cores
A100 (2020)	108	40 MB	40/80 GB HBM2e	1.6–2.0 TB/s	async copy, sparsity, TF32/BF16
H100 (2022)	132	50 MB	80 GB HBM3	3.35 TB/s	clusters, TMA, warpgroup MMA, FP8
B200 (2024)	148	126 MB	180 GB HBM3e	8 TB/s	tensor memory, <code>tcgen05</code> , FP4/FP6

enough free registers, enough free shared memory and enough free warp slots for the *entire* block, since all threads of a block must be on the same SM at the same time (otherwise they could not wait for each other at a barrier). When a block finishes, its resources are freed and the next waiting block takes its place. A large grid therefore flows through the chip in waves, with several blocks living on each SM at any moment. (The optional thread block clusters of Section 9.2 are handed out the same way, except that all blocks of a cluster are placed together on neighbouring SMs.)

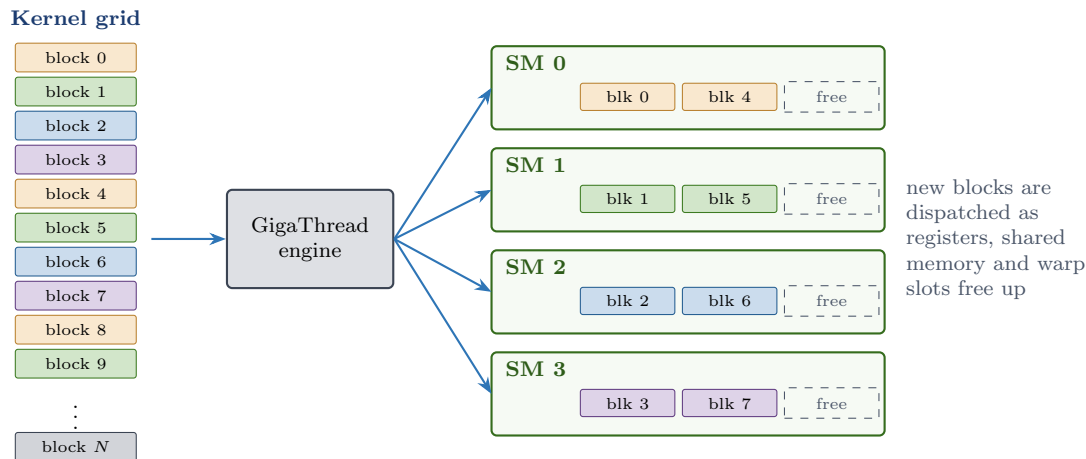


Figure 19: Dispatching thread blocks to SMs. The grid is a queue of independent blocks; the GigaThread engine places each one on any SM with enough free registers, shared memory and warp slots. Several blocks are typically resident on each SM at the same time.

10.3 Inside one SM

The whole chip, we saw, is made of many copies of one building block, the SM. The SM is built the same way. It is made of four identical copies of a smaller unit, which NVIDIA calls a **processing block**, plus a few resources that the four processing blocks share (Figure 20). We therefore first look inside one processing block, then at what the four share, and finally add everything up to get the SM. The figures are for the H100; Blackwell’s SM has the same plan, with one addition noted below.

One processing block.

When the GigaThread engine places a thread block on an SM (Section 10.2), the SM’s hardware splits the block into warps and assigns each warp to one of its four processing blocks, where the warp stays until it finishes (Section 10.4 gives the rule it uses). Each processing block can hold up

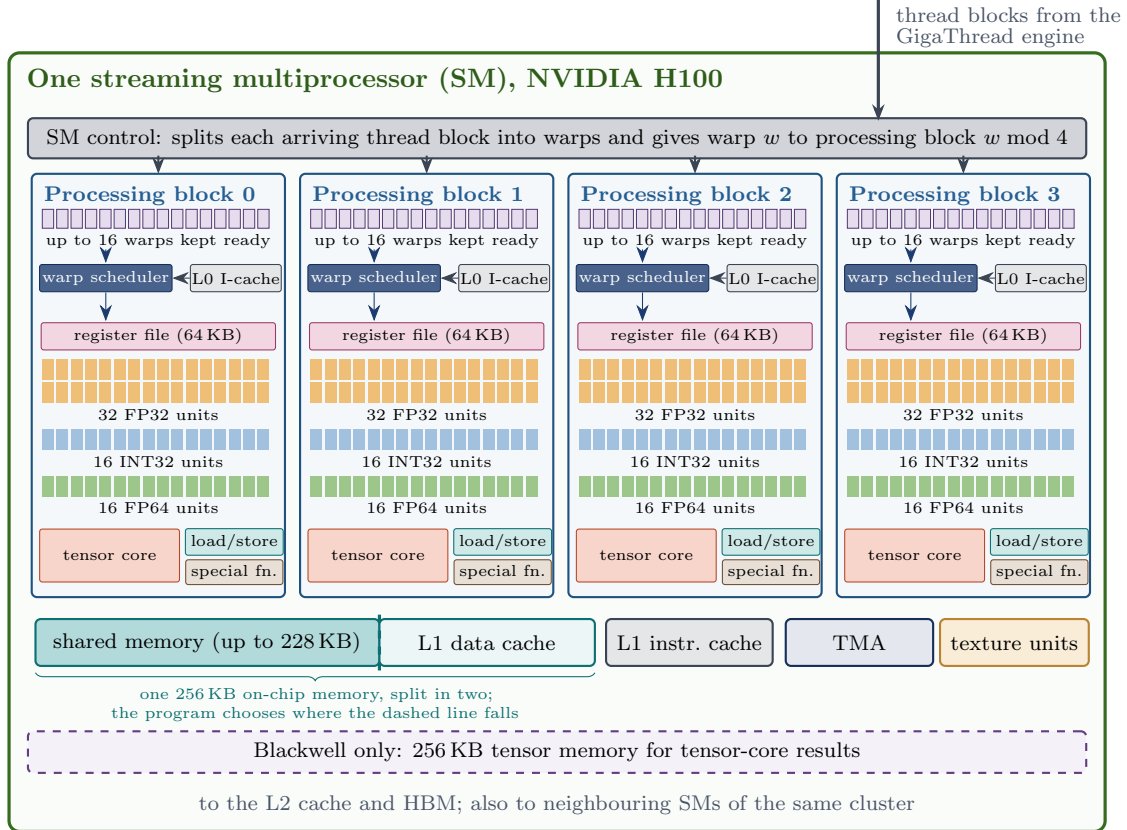


Figure 20: One streaming multiprocessor (SM) of an NVIDIA H100, from top to bottom. Thread blocks arrive from the GigaThread engine; the SM’s control logic splits each block into warps and gives warp w to processing block $w \bmod 4$. The four processing blocks (blue outlines) are identical. Each contains slots for up to 16 warps (purple), a warp scheduler (dark blue) that picks one of them each cycle and fetches its instructions from an L0 instruction cache (light grey), a 64 KB register file (pink), 32 FP32 units (orange), 16 INT32 units (light blue), 16 FP64 units (green), a tensor core (coral), load/store units (teal) and special-function units (brown). Below them are the parts the four processing blocks share: one 256 KB on-chip memory, divided by the program between shared memory and the L1 data cache (the dashed line marks the movable boundary); the L1 instruction cache; the TMA; and the texture units. Blackwell adds a 256 KB tensor memory (dashed). Compare with the toy core of Figure 14: the same three ideas, at real scale.

to 16 warps at a time, waiting their turn to be scheduled, that is, up to $16 \times 32 = 512$ threads. To run them, it contains the following parts, all shown in Figure 20.

- **A small instruction cache**, called the *L0 instruction cache*. A kernel’s instructions are stored in the GPU’s main memory, which is far too slow to visit for every instruction. The L0 cache keeps a copy of the instructions that this processing block’s warps are currently executing, right next to the warp scheduler, so that the next instruction is almost always at hand. Since all threads run the same kernel, the warps of a processing block mostly need the same few instructions, and a small cache is enough.
- **A warp scheduler**. On every clock cycle, it looks at its warps, picks one that is ready to run, fetches that warp’s next instruction from the L0 cache, and sends it to the execution units below. This is the single Fetch/Decode unit of idea #2: one instruction at a time, carried out for all 32 threads of a warp. Section 10.4 looks at how the choice is made.

- **A register file** of 64 KB, holding 16,384 registers of 32 bits each. It holds the registers of *all* the threads of *all* the warps assigned to this processing block, so that the scheduler can switch from one warp to another at no cost (idea #3). With the full 512 threads, that is $16,384/512 = 32$ registers per thread. A program that needs more registers per thread leaves room for fewer warps, which is why register use matters so much for speed (Section 12.1).
- **Execution units for ordinary arithmetic.** There are 32 units for 32-bit floating-point arithmetic (FP32), one for each thread of a warp, so a floating-point instruction can be carried out for a whole warp in a single cycle. There are also 16 units for integer arithmetic (INT32) and 16 for 64-bit floating-point arithmetic (FP64); an instruction of these kinds takes two cycles for a whole warp, half of the warp in each cycle. The numbers follow what programs need. FP64 units are large, and most GPU programs, graphics and AI above all, rarely use 64-bit numbers. Integer units are small, but integer instructions, mostly address and loop-counter calculations, are typically only about one in three of a program's arithmetic instructions [47]. Because the warp scheduler starts one instruction per cycle, a 16-wide integer unit busy for two cycles on one warp does not hold anything up: in the second cycle the scheduler can already send a floating-point instruction to the FP32 units.
- **One tensor core**, a unit that multiplies small matrices in one step, performing many multiply-adds at once rather than one per unit; Section 13 explains why AI needs this.
- **Load/store units**, which carry out the memory accesses of a warp, and **special-function units**, which compute functions such as sine, square root and reciprocal.

What the four processing blocks share.

Some resources sit outside the four processing blocks and serve all of them.

- **An L1 instruction cache**, larger than the L0 caches. When a processing block needs an instruction that is not in its L0 cache, it is fetched from this shared cache, and only if it is not there either does it come from the L2 cache or main memory.
- **256 KB of on-chip memory**, divided, as the program prefers, between the **L1 data cache** and the program's **shared memory** (Section 9); up to 228 KB can be used as shared memory. Because this memory is shared by all four processing blocks, the threads of one thread block can exchange data through it even when their warps run on different processing blocks.
- **The Tensor Memory Accelerator (TMA)**, a copy engine added in Hopper. A thread issues a single command, and the TMA then moves a whole tile of a matrix between global and shared memory on its own, while the threads keep computing; Section 13 shows why this matters for keeping the tensor cores busy.
- **Texture units**, which read images and other data in the special ways that graphics needs.
- On Blackwell only, a further 256 KB of **tensor memory**, reserved for the results of tensor-core calculations [27].

Adding it up: one SM.

We now have both halves of the SM: what is inside one processing block, and what the four processing blocks share. To get the whole SM, we put the two together. Every resource inside a processing block appears four times, so its number for the SM is four times its number for one processing block; the shared resources appear only once. Table 6 lists the result, line by line. Two

of these totals deserve to be worked out step by step, because they connect this section to earlier parts of the paper.

How fast can an SM compute? Start from one processing block. Its warp scheduler starts one instruction per cycle. If that instruction is a 32-bit floating-point operation, the block's 32 FP32 units carry it out for all 32 threads of the warp at once, so one processing block performs 32 FP32 operations per cycle. Now multiply up:

- one SM has four processing blocks: $4 \times 32 = 128$ FP32 operations per cycle;
- the H100 has 132 SMs: $132 \times 128 = 16,896$ FP32 operations per cycle;
- the most common floating-point instruction is a *fused multiply-add*, $a \times b + c$, which counts as two operations (Section 2): $16,896 \times 2 = 33,792$ FP32 operations per cycle;
- the clock runs at about 1.98 billion cycles per second: $33,792 \times 1.98 \times 10^9 \approx 67 \times 10^{12}$ FP32 operations per second.

This is exactly the 67 TFLOP/s of Section 2. In other words, the GPU's impressive peak speed is not the result of any single fast unit. It is one modest step, a single instruction carried out for the 32 threads of a warp, performed in every one of the GPU's $132 \times 4 = 528$ processing blocks in every cycle, about two billion times per second.

How many warps can be kept ready? Again start from one processing block. Its warp scheduler can hold up to 16 warps, and each warp is 32 threads, so one processing block keeps $16 \times 32 = 512$ threads ready. Multiplying up:

- one SM has four processing blocks: $4 \times 16 = 64$ warps, that is, $64 \times 32 = 2,048$ threads;
- the H100 has 132 SMs: $132 \times 64 = 8,448$ warps, that is, $8,448 \times 32 = 270,336$ threads on the chip at the same moment.

Why keep so many warps? This is idea #3 of Section 8 at work. In Figure 13, a core kept a few groups of threads ready and switched to another group whenever the running one had to wait for memory. On the H100, those groups are warps. When the warp that a scheduler is running has to wait for data from memory, which takes hundreds of cycles, the scheduler does not wait with it: in the next cycle it simply picks another of its 16 warps that is ready. With enough warps to choose from, the arithmetic units almost never sit idle.

This switching costs nothing only because every warp's registers stay on the chip the whole time, and the register file is sized for exactly that. One processing block holds 16 warps of 32 threads; if each thread uses 32 registers, they need $16 \times 32 \times 32 = 16,384$ registers, which is exactly the size of the processing block's register file. Four processing blocks then give the SM's 65,536 registers. A program whose threads need more than 32 registers each leaves room for fewer warps, and so hides memory delays less well (Section 12.1).

Table 6: From one processing block to one SM (NVIDIA H100). Totals for the whole GPU follow by multiplying the SM column by 132.

	One processing block	One SM (4 blocks + shared)
Warp schedulers (instructions started per cycle)	1	4
Warps kept ready (threads)	up to 16 (512)	up to 64 (2,048)
Registers	16,384 (64 KB)	65,536 (256 KB)
FP32 units	32	128
INT32 units / FP64 units	16 / 16	64 / 64
Tensor cores	1	4
L1 data cache + shared memory	– (shared)	256 KB (up to 228 KB shared)
Thread blocks kept at once	–	up to 32

The three design ideas, in real hardware.

In Section 8 we built a small, imaginary GPU from three design ideas (Figure 14): 16 slim cores, each driving 8 ALUs with one instruction stream and keeping 4 groups of threads ready. That GPU was a teaching device. The H100 shows that it was not a simplification: a real, current GPU is built from exactly the same three ideas, only at a much larger scale.

- **Idea #1, many slim cores.** The GPU’s “core” is the SM. It contains none of the machinery that makes a CPU core fast on a single thread: no out-of-order execution and no branch prediction. A warp’s instructions are started one after another, in program order. Because each SM is kept this simple, 132 of them fit on one H100 chip (148 on the B200), where the toy GPU had 16.
- **Idea #2, one instruction drives many ALUs.** In each processing block, one warp scheduler starts one instruction per cycle, and 32 FP32 units carry it out for the 32 threads of a warp. The work of fetching and decoding the instruction is done once and shared by 32 threads, where the toy GPU shared it by 8.
- **Idea #3, many threads kept ready to hide memory delays.** Each warp scheduler keeps up to 16 warps ready, and the register file holds all their registers, 256 KB per SM, as much storage as its L1 cache and shared memory together. When one warp waits for memory, the scheduler switches to another in the next cycle, at no cost. The toy GPU kept 4 groups per core; the H100 keeps 64 warps per SM.

This is why the H100 deserves a close look. Everything in the rest of the paper, from how warps handle `if` statements (Section 11) to the rules for writing fast GPU code (Section 12), is a consequence of these three ideas as they are built into this hardware.

10.4 Choosing what to run, every cycle

We can now follow a kernel all the way from the host program that launches it to the arithmetic units that carry out its instructions. The journey has five steps.

1. **From the host program to a CUDA stream.** The program on the CPU launches a kernel, for example with the line `vec_add<<<grid, block>>>(...)` of Listing 3. The launch does not run anything yet. The **CUDA runtime**, the software library through which the

host program talks to the GPU, places it in a queue called a **CUDA stream**, together with any requests the program has made to copy data from the CPU's memory to the GPU's memory or back, and the CPU carries on with its own work. A stream thus does not hold any data itself. It holds *commands*: requests to launch a kernel and requests to copy data, in either direction. Each command carries only the information needed to carry it out later. A copy command records where the data is now (its source address), where it should go (its destination address) and how many bytes to move, for example `cudaMemcpy(d_a, h_a, bytes, ...)` in Listing 3. A launch command records which kernel to run, its arguments, which are mostly addresses in GPU memory such as `d_a`, and the shape of the grid. In Listing 3, for example, the stream holds four commands, in this order: copy `a` to the GPU, copy `b` to the GPU, launch `vec_add`, and copy the result `c` back to the CPU. Within one stream, the commands are carried out strictly in the order they were queued, so the kernel never starts before its input data has arrived, and the result is never copied back before the kernel has finished. Kernel launches and copies in *different* streams, however, may run at the same time (Figure 18).

2. **From the CUDA stream to the GPU.** When the launch reaches the front of its stream, that is, when all the commands queued before it in the same stream have finished, it is sent over the PCIe link to the GPU. It enters through the GPU's host interface, the unit that receives everything the host sends, and is passed on to the GigaThread engine. The launch carries everything the GPU needs to know: which kernel to run, its arguments, and the shape of the grid, that is, how many thread blocks and how many threads in each.
3. **From the GigaThread engine to the SMs.** The GigaThread engine goes through the kernel's thread blocks and hands them, one by one, to SMs that have room for them: enough free registers, shared memory and warp slots for the whole block (Section 10.2, Figure 19). Blocks that do not fit yet wait until a block on some SM finishes and frees its resources.
4. **From the SM to the warp schedulers.** When a thread block arrives at an SM, the SM's control logic does two things. First, it splits the block into warps of 32 consecutive threads: threads 0–31 form warp 0, threads 32–63 form warp 1, and so on. A block of 256 threads, for example, becomes 8 warps. If the block size is not a multiple of 32, the last warp is only partly filled, and its unused places simply do no work; this is why block sizes are usually chosen as multiples of 32. Second, it hands each warp to the warp scheduler of one of its four processing blocks. In practice, warp w goes to processing block $w \bmod 4$: warps 0, 4, 8, ... go to processing block 0, warps 1, 5, 9, ... to processing block 1, and so on [28]. The 8 warps of our 256-thread block are thus spread evenly, two to each warp scheduler. What happens to a warp after this hand-over is so important that it gets its own paragraph below.
5. **From the warp scheduler to the arithmetic units.** Each warp scheduler now holds several warps, possibly from different thread blocks, each waiting to run its next instruction. On every clock cycle, the scheduler chooses one of them and sends that warp's next instruction to the arithmetic units of its processing block. How this choice is made, and why it keeps the units busy even while many warps are waiting for memory, is the subject of the rest of this subsection.

A warp's life inside its processing block.

Once a warp has been handed to a processing block, it stays there until it has finished, that is, until all its threads have executed the last instruction of the kernel. It never moves to another processing block, and never to another SM.

The reason is the register file. When the warp arrives, its processing block reserves space for the registers of all 32 of its threads in its own register file, and the temporary values that its threads compute, such as loop counters and intermediate results, live there from the warp’s first instruction to its last. Moving the warp would mean copying all those registers somewhere else, which is exactly the costly switch that a CPU makes when it changes threads (Section 2.4) and that a GPU is designed to avoid. Because every waiting warp keeps its state where it is, the scheduler can switch from one warp to another between two cycles at no cost at all: idea #3 depends on it.

Three consequences follow, and the rest of the paper relies on them.

- **A processing block can hold only so many warps.** Each warp scheduler can hold up to 16 warps at a time, and the register file must have room for all their registers. A whole SM therefore holds at most 64 warps, and fewer if each thread needs many registers (Section 12.1).
- **Resources are freed only when a warp or a thread block finishes.** A warp’s place in its warp scheduler and its registers stay reserved until the warp has executed the kernel to its last instruction. A thread block’s shared memory stays reserved until all of its warps have done so. Only then can the GigaThread engine send the next thread block of the kernel to that SM (step 3 above).
- **Small thread blocks waste processing blocks.** Since the warps of a thread block are spread over the four processing blocks, a thread block needs at least $4 \times 32 = 128$ threads, that is, four warps, to give every warp scheduler something to do. A smaller block leaves some processing blocks without any of its warps. A block of 64 threads, for example, has only two warps, which go to processing blocks 0 and 1; processing blocks 2 and 3, with their warp schedulers and arithmetic units, get nothing from it and can only be kept busy by other thread blocks on the same SM.

Choosing a warp, every cycle.

At any moment, each warp held by a warp scheduler is in one of two situations.

- It is *ready*: the inputs of its next instruction are available and the unit that instruction needs is free.
- It is *waiting*: for example for data from memory, for the result of an earlier instruction that has not finished yet, or for the other warps of its block at a barrier.

On every clock cycle, the scheduler looks at its warps, picks one that is ready, and sends that warp’s next instruction to the arithmetic units of its processing block. Waiting warps are simply skipped; they lose nothing, because their state stays in place. Figure 21 shows one scheduler at work over 14 cycles, with six of its warps. By the $w \bmod 4$ rule, warps 0, 4, 8, 12, 16 and 20 all belong to processing block 0. Each warp runs briefly, issues a memory load and then waits many cycles for its data; meanwhile the scheduler keeps choosing other warps that are ready. A cycle is wasted only when *no* warp is ready. In cycle 13 of the figure, all six warps are waiting for memory at the same time, so processing block 0 issues nothing in that cycle.

Put the four processing blocks together and an SM can start up to four warp instructions per cycle, one per processing block, and each processing block stays busy as long as at least one of its warps is ready. This is idea #3 working in hardware: many warps, each waiting part of the time, together keep the units busy all the time.

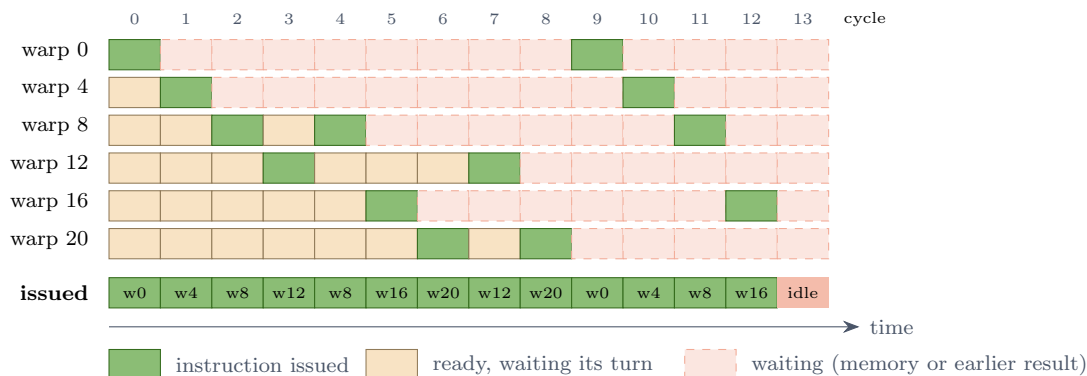


Figure 21: One warp scheduler at work. Six of the warps held by processing block 0 are shown (warps 0, 4, 8, ..., by the $w \bmod 4$ rule). On each cycle the scheduler issues one instruction from a ready warp (green); the other ready warps wait their turn (yellow). After a warp issues a memory load, it waits many cycles for the data (red), and the scheduler issues instructions from other warps in the meantime. In cycle 13, all six warps are waiting for memory, so the scheduler has no instruction to issue and the arithmetic units of processing block 0 do nothing for that cycle. If the processing block had held more warps, one of them could have been ready to fill that cycle. The number of warps an SM holds at once, as a fraction of the maximum it can hold, is called occupancy (Section 12.1).

The compiler's part.

A warp is ready only when the inputs of its next instruction are available. So something must keep track of when the result of each earlier instruction will be ready. On NVIDIA GPUs, much of this tracking is done by the compiler, not by the hardware. Every instruction in NVIDIA's machine code (called **SASS**) carries a few extra bits of scheduling information, filled in by the compiler [25, 28]. How the compiler fills them depends on whether the instruction's timing is known in advance.

- **Instructions whose timing is fixed.** For most arithmetic instructions, such as a floating-point add or multiply, the number of cycles from issue until the result is ready is always the same, and the compiler knows it: about 4 cycles for a 32-bit floating-point multiply-add on recent GPUs [28]. For every two successive instructions A and B of a warp, the compiler writes a **stall count** into A: the number of cycles the warp scheduler must wait after issuing A before it may issue B. The compiler chooses this count so that B does not issue before its inputs are ready. It also orders the instructions to keep the stall counts as small as possible, for example by placing independent instructions between an instruction and the one that uses its result. The hardware does not check these dependences itself; it only obeys the stall counts. The compiler must therefore set them correctly; otherwise an instruction would read an old value.
- **Instructions whose timing is not fixed.** The time taken by a memory load depends on where the data is found (L1, L2 or HBM) and on what the other warps are doing, so the compiler cannot work it out in advance. The same holds for a few other instructions, such as texture reads. For these, the compiler and the hardware share the job, using a few **dependence counters** kept for each warp. The compiler marks each load with the counter it should use, and marks the instruction that uses the loaded data with the counter it must wait for. While the kernel runs, the hardware does the counting: issuing the load increments its counter, and the data arriving from memory decrements it. The warp scheduler does not issue the waiting instruction until that counter is back to zero.

Both kinds of waiting apply to one warp only. While a warp waits out its stall count, or waits for a dependence counter, it is simply not ready, and the scheduler issues instructions from the other warps of its processing block, as in Figure 21. Because the compiler handles every instruction with a fixed timing, the hardware does not need circuits that compare the inputs of each instruction with every result still being computed. This keeps each SM small and simple, in the spirit of idea #1.

10.5 Values shared by a whole warp

Many values in a kernel are identical for all 32 threads of a warp: the size of the data, the address where an array starts, a loop counter, the block number. Computing such a value 32 times, once per thread, wastes energy and registers. Since 2018 (the Turing generation), NVIDIA SMs include a small **uniform datapath**: a few registers and instructions that work on such a value once for the whole warp [29]. The compiler decides which values qualify, using an analysis that proves a value is the same for every thread of a warp (a topic covered in depth in a companion paper [68]). AMD GPUs have had a similar “scalar unit” since 2012; Section 14.4 compares the two.

Takeaway: a modern GPU in one paragraph

A modern NVIDIA GPU is about 130 to 150 identical SMs sharing a large L2 cache and very fast stacked memory. Each SM is made of four identical processing blocks, each with its own warp scheduler, register file and 32 FP32 units. A kernel launch travels from a CUDA stream on the host to the GigaThread engine, which hands thread blocks to SMs as space allows. Each SM splits its blocks into warps of 32 threads and gives every warp to one processing block, where it stays, with its registers, until it finishes. On every cycle, each processing block starts one instruction for one ready warp, skipping the warps that are waiting for memory.

11 How a Warp Executes: the SIMT Model

There is an apparent contradiction in what we have seen so far. The programmer writes a kernel for *one* thread, as if every thread ran on its own (Section 9). The hardware, however, executes one instruction for 32 threads at a time (Section 10). This section explains how the two fit together, and what happens when the 32 threads of a warp want to do different things.

11.1 Thirty-two threads, one instruction

NVIDIA calls its GPU execution model **Single Instruction, Multiple Threads** (SIMT), a name introduced with the G80 chip in 2006 [38]. The 32 threads of a warp run in **lock-step**:

- they share one program counter, so at each step all of them execute the same instruction;
- each thread nevertheless has its *own* registers, and therefore its own data; and
- a 32-bit **execution mask**, one bit per thread, records which threads take part in the current instruction: if a thread’s bit is set, the thread executes the instruction; if it is not set, the thread does not.

In hardware terms, SIMT runs each warp as a stream of 32-wide SIMD instructions (Section 2.6), in which each lane belongs to a different thread and the execution mask decides which lanes take part.

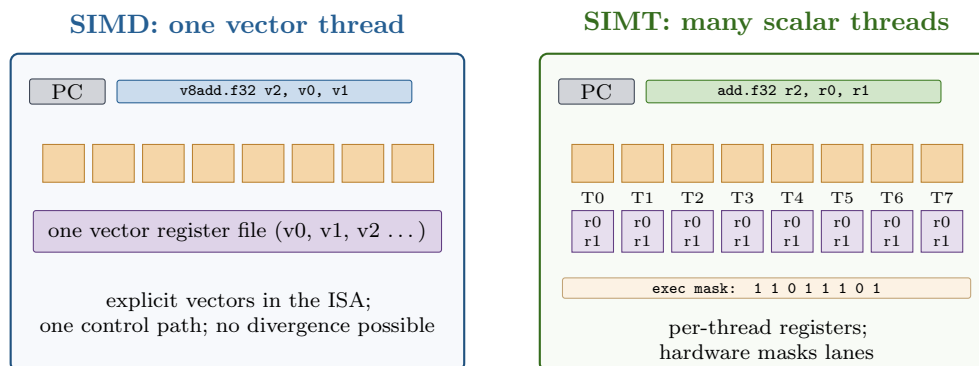


Figure 22: SIMD and SIMT side by side. Both share one program counter across eight ALUs. In SIMD the eight lanes belong to one thread and the width is visible in the instruction. In SIMT each lane is a separate scalar thread with its own registers, and an execution mask decides which threads take part in each instruction.

11.2 SIMD versus SIMT

Figure 22 and Table 7 compare SIMT with the ordinary SIMD vector instructions of a CPU. With **SIMD**, there is *one* thread whose instructions happen to work on vectors of 8 or 16 numbers. The programmer (or the compiler) must rewrite the code explicitly in terms of vectors of that exact width. An `if` is also harder: because there is only one thread, a branch goes the same way for all lanes. When the condition differs from lane to lane, the programmer (or the compiler) must turn the `if` into straight-line code: compute the results of both paths for all lanes, then use a mask of the condition to keep, in each lane, the result of the path that lane should have taken. With **SIMT**, the programmer writes plain scalar code for one thread, and the hardware groups 32 such threads into a warp by itself. The width of 32 is invisible in the program, and every thread is free to take its own path at an `if`. Underneath, the execution of a warp at such an `if` is close to the SIMD case. When the threads of a warp disagree, the warp runs both paths, one after the other, and on each path the execution mask switches off the threads that did not take it (Section 11.3). The difference lies in two points.

- **Who does the masking.** The programmer writes an ordinary `if`; the compiler and the hardware together turn it into masked execution. On NVIDIA GPUs the hardware does most of this work; on AMD GPUs the compiler does more of it (Section 14.3).
- **When the cost is paid.** The straight-line SIMD code computes both paths every time, unless the programmer adds extra tests. A warp decides at run time, each time it reaches the `if`: if all 32 threads take the same path, the warp runs only that path, and the other path costs nothing. The warp runs both paths only when its threads actually disagree.

11.3 When threads disagree: branch divergence

Consider the kernel fragment in Figure 23. Threads with an even index take the `if` path (path A) and threads with an odd index take the `else` path (path B). Every warp contains both kinds. Since a warp can execute only one instruction at a time, it cannot run both paths at once. Instead it runs them *one after the other*: first path A with the odd threads switched off in the mask, then path B with the even threads switched off. After the `if-else`, the two halves join up again and continue together with all threads switched on; this is called **reconvergence**. While each path

Table 7: SIMD and SIMT compared.

	SIMD	SIMT
Who runs the code	One thread whose instructions work on whole vectors	32 ordinary threads that execute together in a warp
What the programmer writes	Explicit vector code for a known width (e.g. 8 lanes)	Plain code for one thread; the width of 32 is invisible
Registers	One set of vector registers	Registers per thread
if statements	No per-lane branching; the program computes both paths for all lanes and masks the results	Each thread may branch its own way; compiler and hardware mask the threads, and both paths run only when threads disagree
Memory accesses	Contiguous vector loads, written explicitly	Each thread computes its own address; hardware merges neighbouring ones (Section 12.2)

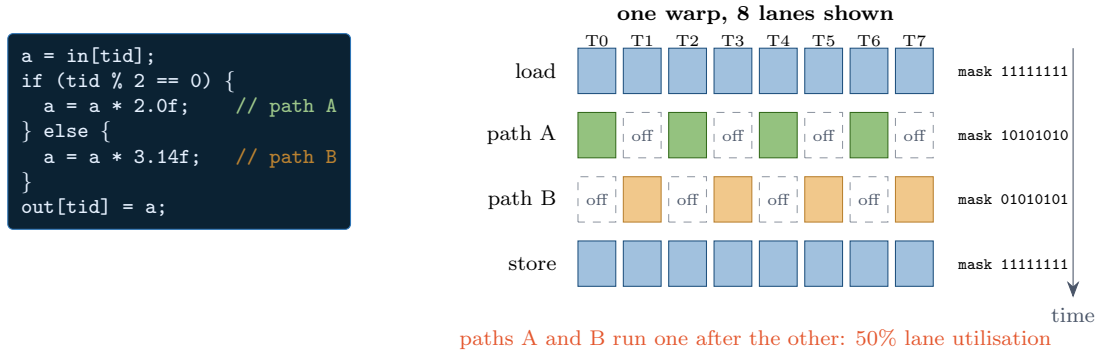


Figure 23: Branch divergence within a warp. Lanes that evaluate the condition differently cannot execute their paths simultaneously; the hardware runs each path in turn under an execution mask and reconverges afterwards.

runs, half of the ALUs do nothing useful, so this code runs at half speed. The effect is called **branch divergence**.

Two points are worth stressing.

1. **Divergence only matters *inside* a warp.** If all 32 threads of warp 0 take path A and all 32 threads of warp 1 take path B, each warp follows a single path and nothing is lost. Rewriting a condition so that it is the same for a whole warp (for example, testing `tid / 32` instead of `tid % 2`) can remove the cost entirely, as Section 12.4 demonstrates.
2. **The cost grows with each split.** With several nested or data-dependent ifs and loops, a warp may split into many small groups that run one after another. In the worst case all 32 threads go different ways, and the warp runs at 1/32 of its normal speed. In general, a warp split k ways runs at about $1/k$ of its full speed, and measurements on NVIDIA GPUs from 2016 to 2025 show that this still holds on every generation [21].

How the hardware handles divergence.

When a warp reaches an `if`, each of its threads evaluates the condition. If all 32 threads agree, the warp simply follows that path, and nothing special happens. If they disagree, the warp can

run only one path at a time, so the hardware keeps a note of three things: which threads are waiting to run the other path, where that other path starts, and where the two paths meet again. The meeting point is the first instruction that both paths reach, such as the statement right after the `if-else`; the compiler marks it in the machine code. The hardware then runs one path with only that path's threads switched on in the execution mask. When this path reaches the meeting point, the hardware switches to the other path and its threads. When that path arrives too, all the threads are switched on again and the warp continues together.

An `if` inside another `if` adds a second note while the first is still pending, and the inner `if` must be finished before the outer one can be. NVIDIA GPUs up to 2016 therefore kept these notes in a small **stack** for each warp, in which the newest note is always dealt with first [38].

Since the Volta generation (2017), NVIDIA GPUs remember, for each thread, which instruction it has reached [59]. With this, the hardware no longer has to finish one path before it starts the other: it can run a little of one path, then a little of the other, and still bring all the threads together at the meeting point. A warp still runs only one path at a time, so divergence costs as much as before. What changes is that two threads of the same warp may now be at different instructions for a while. This helps a thread that waits for a value from another thread of its warp: before 2017, if the hardware ran the waiting thread's path first, the other thread never got a turn, and the program hung. But a program that relies on the 32 threads of a warp always running each instruction together, for example to read a value that another thread of the warp has just written, can now go wrong. The programmer must mark each point where the threads of the warp have to be together, by calling `__syncwarp()` in the kernel; the compiler does not add this call on its own. This does not mean that programmers now handle divergence themselves. Ordinary code with `ifs` and loops needs no change: the hardware still splits and rejoins warps on its own. Only code in which threads of the same warp exchange values needs `__syncwarp()`, just as threads of different warps already needed `__syncthreads()` (Section 9). CUDA had in fact never promised that the threads of a warp run in lock-step; older code simply relied on how the hardware happened to behave.

How the compiler helps.

Using an analysis called **divergence analysis** (or uniformity analysis), the compiler works out which values and which `if` conditions are guaranteed to be the same for all threads of a warp. Such values can be computed once per warp in the uniform datapath (Section 10.5), and such `ifs` can never split a warp, so the hardware never needs to keep notes for them. These techniques are surveyed in detail in the companion paper [68]. AMD GPUs handle divergence in a quite different, more compiler-driven way, which we compare in Section 14.3.

Takeaway: SIMT is a contract

SIMT lets the programmer think about one thread at a time while the hardware executes 32 at once. The deal is this: the *correctness* of a program never depends on warps, but its *speed* does. Code in which the threads of a warp follow the same path, and read neighbouring memory locations (Section 12.2), runs at full speed; code that does not still gives the right answer, only more slowly.

12 What Makes GPU Code Fast

The three design ideas of Section 8 are what make a GPU a throughput machine: many slim cores (idea #1), each driving many ALUs with one instruction (idea #2), and switching between many

threads to hide memory delays (idea #3). Together they let a GPU finish far more work per second than a CPU. But each idea also assumes something about the kernel it runs: that there are enough threads to switch between, that the threads of a warp follow the same path and access memory in a pattern that the hardware can serve efficiently, and that data fetched from memory is used many times rather than fetched again and again. A kernel that gives the right answer but ignores these assumptions can run far slower than the hardware allows. This section explains what the design ideas ask of a kernel for it to run efficiently, in three parts: keeping enough warps on each SM, accessing memory the right way, and making the computation itself efficient. It then applies all of them to one complete example, a kernel that adds up a million numbers.

12.1 Keeping enough warps on each SM: occupancy

Idea #3 hides memory delays only if each SM has enough warps on hand to switch between. How many warps fit is limited by four resources that every SM has in fixed supply and that are shared by everything running on it. The numbers below are those of an SM of the H100, the NVIDIA GPU we examined in Section 10 (Table 6):

- **Registers:** 65,536 per SM, divided among the warps. Each thread keeps its registers for its whole lifetime.
- **Shared memory:** up to 228 KB per SM, divided among the blocks.
- **Warp slots:** at most 64 warps (2,048 threads) per SM. This follows from Section 10.3: each of the four processing blocks can hold at most 16 warps, and $4 \times 16 = 64$.
- **Block slots:** at most 32 thread blocks per SM, however small they are.

The number of warps present on an SM, as a fraction of the maximum number of warps the SM can hold, is called **occupancy**. On the H100, for example, an SM holding 32 warps runs at an occupancy of $32/64 = 50\%$. The rule is simple: *the fewer resources each thread or block needs, the more of them fit on an SM at once*, and the more memory delay the SM can hide.

A worked example: registers.

The compiler decides how many registers r each thread of a kernel needs. Two hardware rules then decide how many registers the SM actually sets aside. NVIDIA documents both for every GPU generation, and they have been the same on all recent generations, including the H100 [64].

- **Registers are set aside in fixed-size chunks.** The SM does not give a warp exactly $32 \times r$ registers. It rounds this number up to a multiple of 256. Because a warp has 32 threads, this is the same as rounding each thread's r up to a multiple of 8 ($256/32 = 8$), which we write $8\lceil r/8 \rceil$ (the brackets $\lceil \cdot \rceil$ mean “round up”). A kernel whose threads need 30 registers, for example, is given 32 registers per thread. For the same reason, every thread is given at least 8 registers, even if it needs fewer.
- **A thread can use at most 255 registers.** This limit applies to r , the number of registers the compiler gives each thread [53]. The chunk rule above still applies afterwards: for a thread with $r = 255$, the SM rounds up and sets aside 256 registers per thread (one of them unused), that is, $32 \times 256 = 8,192$ registers for the warp. Some kernels, however, have more values in use at the same moment than 255 registers can hold. For such a kernel, the compiler stores some of the values in memory and loads them back when they are needed again. This

is called **register spilling**, and it makes the kernel slower, because memory is much slower than registers.

One warp therefore takes $32 \times 8 \lceil r/8 \rceil$ registers: at least 256 (when $r \leq 8$) and at most 8,192 (when r is between 249 and 255). An H100 SM has 65,536 registers and can hold at most 64 warps (Table 6). So the number of warps that can fit on one SM, given the registers each warp takes, is

$$W_{\text{SM}} = \min\left(64, \left\lfloor \frac{65,536}{32 \cdot 8 \lceil r/8 \rceil} \right\rfloor\right) = \min\left(64, \left\lfloor \frac{256}{\lceil r/8 \rceil} \right\rfloor\right), \quad (3)$$

ignoring some further rounding to whole blocks. (The brackets $\lfloor \cdot \rfloor$ mean “round down”.) A kernel that uses up to 32 registers per thread fits the full 64 warps. One that uses 64 fits only 32 warps; 128 registers, 16 warps; and the maximum of 255, just 8 warps, two per processing block (Figure 24).

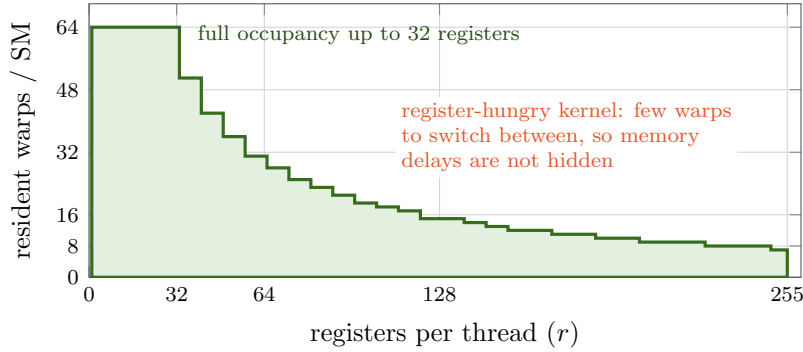


Figure 24: Resident warps per SM of the H100 as a function of register use, from Equation (3). Doubling the registers per thread halves the warps available to hide memory latency. Shared memory and block limits impose further caps not shown.

A worked example: shared memory.

A kernel states how much shared memory each of its thread blocks needs; call this s . All the threads of a block use the same shared memory, so shared memory is counted per block, not per thread. On the H100, an SM can use up to 228 KB as shared memory (it is part of the same on-chip memory as the L1 cache, Section 10.3), and CUDA itself reserves 1 KB of it for each thread block [53]. Each block therefore takes $s + 1$ KB, and the number of blocks that fit on one SM, as far as shared memory is concerned, is

$$B_{\text{SM}} = \left\lfloor \frac{228 \text{ KB}}{s + 1 \text{ KB}} \right\rfloor. \quad (4)$$

A block always brings all of its warps with it, so the number of warps that fit is B_{SM} times the number of warps in each block. For example, if each block uses 48 KB, it takes 49 KB, and $\lfloor 228/49 \rfloor = 4$ blocks fit. With blocks of 256 threads (8 warps each), that is 32 warps per SM, half the maximum of 64, however few registers the threads use.

A worked example: block slots.

An H100 SM can hold at most 32 thread blocks at the same time, whatever the size of each block. How many warps these 32 blocks bring depends on how many warps each block contains:

- With blocks of 32 threads, each block is one warp, so 32 blocks bring 32 warps. The SM has room for 64 warps, but the block limit lets it fill only half of them.

- With blocks of 64 threads, each block is two warps, so 32 blocks bring 64 warps, and the SM is full.
- With larger blocks, fewer than 32 blocks are enough to fill the SM with all 64 warps, so the block limit is never the one that stops it.

So on the H100, the block limit matters only when blocks have fewer than 64 threads. Section 10.4 gave a further reason to use blocks of at least 128 threads.

Putting the limits together.

An SM can take another thread block only if there are enough registers, enough shared memory, a free block slot and enough free warp slots for the whole block. The number of warps that actually fit is therefore set by whichever of these resources runs out first. NVIDIA's tools calculate and report all of these limits [64].

Occupancy is a means, not a goal.

Warps are kept on an SM for one reason (Section 10.4): on every cycle, each warp scheduler issues an instruction from one of its ready warps, and a cycle is wasted only when none of its warps is ready. Having many warps is one way to make sure that some warp is ready. There is a second way: a warp stays ready for longer if its next instructions do not depend on results that are still being computed or loaded. For example, suppose a thread needs four values from memory, and none of the four addresses depends on another of the values. The warp can then issue all four loads one after another, without waiting for any of them, so the four loads travel to memory at the same time and the warp waits for memory once instead of four times. If instead each load needed the result of the one before it, the warp would have to wait four times in a row.

This has two consequences.

- **Fewer warps are enough to keep the scheduler busy.** Because each warp waits less often, each warp is ready for a larger share of the time. The scheduler therefore finds a ready warp on most cycles even when it holds fewer warps.
- **Each thread needs more registers.** The four values that are on their way from memory must each have a register to arrive in, so the thread uses more registers than a thread that loads one value at a time. By the rule of Equation (3), more registers per thread means fewer warps fit on the SM, so the occupancy is lower.

The two effects fit together: fewer warps are enough, and fewer warps fit. Such a kernel can run as fast as a kernel with full occupancy, and often faster, because data kept in registers does not have to be fetched from memory again. Volkov showed this with measured examples [75]. Higher occupancy therefore helps only when the warps already on the SM cannot keep the schedulers busy on their own.

12.2 Accessing memory the right way

A kernel works with two kinds of memory that the programmer controls directly: **main memory** (the HBM outside the chip, also called global memory, Section 10) and **shared memory** (inside each SM). Each is organised in a fixed way, and each serves a warp fastest when the 32 threads of the warp access it in a pattern that matches this organisation. This subsection takes the two memories in turn: how each is organised, how a warp's access to it is served, what makes that access fast or slow, and what this asks of a kernel.

Main memory: how it is organised.

Main memory is a long row of bytes, and each byte has its own number, its **address**, counting up from 0. An array is stored in this row as one block of consecutive bytes: a **float** takes 4 bytes, so element `a[0]` occupies the first 4 bytes of the array, `a[1]` the next 4, and so on (Figure 25).

Reading main memory is efficient only when data moves in blocks rather than as single numbers. The hardware therefore never moves a single byte or a single number between main memory and an SM. It always moves a whole block of 32 consecutive bytes, called a **sector**, and every sector starts at an address that is a multiple of 32 [50]. One sector holds eight 4-byte **float** values. If a thread needs only `a[3]`, the hardware still fetches the whole sector that contains it, `a[0]` to `a[7]`.

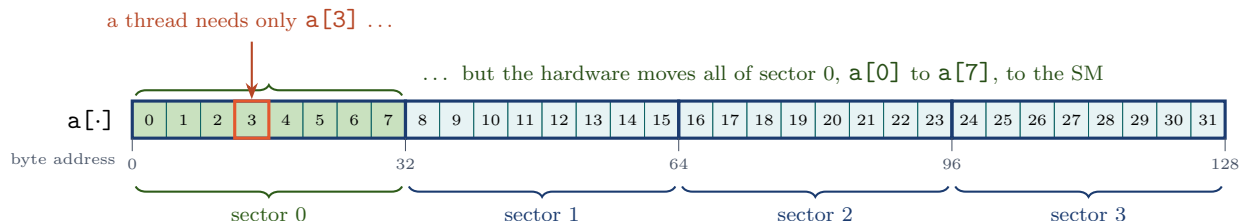


Figure 25: How main memory is organised. An array of **float** values is stored as consecutive 4-byte elements; the numbers in the cells are the element indices. Memory moves data only in whole 32-byte sectors (blue outlines), each starting at a byte address that is a multiple of 32, so each sector holds eight **floats**. To deliver `a[3]` alone, the hardware fetches all of sector 0.

Main memory: how a warp's access is served.

When a warp executes a load instruction, each of its 32 threads supplies its own address. The hardware does not handle the 32 addresses one by one. It collects them, works out which sectors they fall in, and fetches each of those sectors once [50]. So the memory bandwidth a load uses is set by the number of sectors fetched, 32 bytes each, not by the number of bytes the threads actually need.

Main memory: what makes an access fast or slow.

Consider a warp that reads one value per thread from an array `a` of 4-byte **float** values. Let `tid` be a thread's number within its warp, from 0 to 31. Assume the array starts at an address that is a multiple of 32, so that `a[0]` to `a[7]` fill the first sector, `a[8]` to `a[15]` the second sector, and so on. The 32 threads need one value each, so the warp needs $32 \times 4 = 128$ bytes in total. In practice, how many bytes the hardware has to fetch to deliver these 128 bytes depends on which elements the threads read.

- **Neighbouring elements:** `a[tid]`. Thread 0 reads `a[0]`, thread 1 reads `a[1]`, and so on up to thread 31, which reads `a[31]`. These 32 values fill exactly the first four sectors. The hardware fetches 4 sectors, 128 bytes, and every byte fetched is used. Such an access is called **coalesced**.
- **Every second element:** `a[2*tid]`. Thread 0 reads `a[0]`, thread 1 reads `a[2]`, and so on up to thread 31, which reads `a[62]`. These values are spread over the first eight sectors. The hardware fetches 8 sectors, 256 bytes, of which the threads use only 128: half of the bandwidth is wasted [50].
- **Every eighth element or further apart:** `a[8*tid]`. Thread 0 reads `a[0]`, thread 1 reads `a[8]`, and so on up to thread 31, which reads `a[248]`. Each value now lies in a sector of its own.

The hardware fetches 32 sectors, 1,024 bytes, of which the threads use only 128: seven eighths of the bandwidth is wasted. Larger distances between the elements, or elements chosen at random, give the same result.

- **Neighbouring elements, shifted:** $a[tid+1]$. Thread 0 reads $a[1]$, and so on up to thread 31, which reads $a[32]$. The values are still neighbours, but they no longer start at the beginning of a sector, so they reach into a fifth sector. The hardware fetches 5 sectors, 160 bytes, for the 128 bytes used: a small cost.

Table 8 sums up the four cases, and Figure 26 draws a coalesced and a strided access for eight threads.

Table 8: Bytes fetched from main memory when a warp of 32 threads reads one 4-byte value per thread (128 bytes needed in all).

Access	Elements read	Sectors	Bytes fetched	Bytes used
$a[tid]$	$a[0], a[1], \dots, a[31]$	4	128	100%
$a[2*tid]$	$a[0], a[2], \dots, a[62]$	8	256	50%
$a[8*tid]$	$a[0], a[8], \dots, a[248]$	32	1,024	12.5%
$a[tid+1]$	$a[1], a[2], \dots, a[32]$	5	160	80%

Main memory: what this asks of a kernel.

Memory bandwidth is the resource that most GPU programs run out of first (Section 12.3), so every wasted byte costs speed directly. The rule that follows is: *neighbouring threads of a warp should access neighbouring addresses*. In practice, this means letting each thread work on the element whose index is its own thread number, as in $a[tid]$, and storing data so that values the threads read at the same moment lie next to each other. For example, a program that keeps a list of particles, each with a position and a speed, does better to keep all positions in one array and all speeds in another than to store each particle’s position and speed side by side.

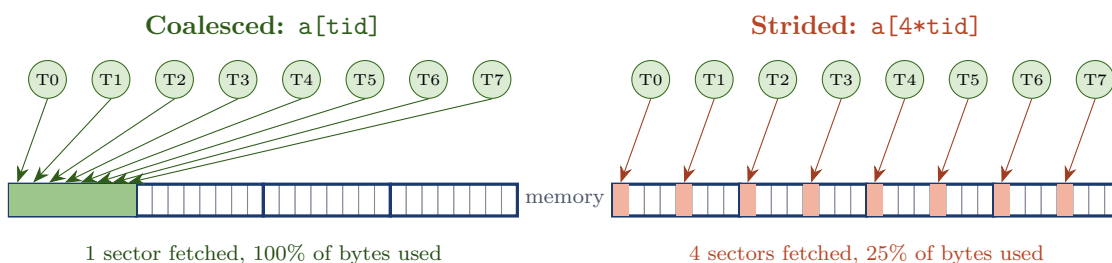


Figure 26: Main-memory access by eight threads (T0 to T7). Each small cell is one 4-byte `float`; blue outlines mark 32-byte sectors, the unit in which memory delivers data. Left: with $a[tid]$, the eight values fill one sector, and every byte fetched is used. Right: with $a[4*tid]$, the eight values fall in four sectors, and only a quarter of the bytes fetched are used.

Shared memory: how it is organised.

Shared memory sits inside each SM (Section 10.3). Like main memory, it is a row of bytes, each with its own address. Shared memory, however, hands out data in units of 4 bytes, called **words**: word 0 is bytes 0 to 3, word 1 is bytes 4 to 7, and so on. A `float` or an `int` fills exactly one word.

The 32 threads of a warp often read shared memory at the same moment, typically with each thread needing one word. If shared memory were a single unit that could hand out only one word

per clock cycle, the 32 words would come out one after another and would take 32 cycles. Shared memory is therefore split into 32 separate units, called **banks**. Each bank stores its own share of the words and can deliver one word per clock cycle [50], and all 32 banks work at the same time. Together, the 32 banks can deliver 32 words in one cycle, one for each thread of a warp.

The words of shared memory are spread over the banks in a fixed way. It helps to picture shared memory as a table with 32 columns (Figure 27): each column is one bank, and the words fill the table row by row. Word 0 goes to bank 0, word 1 to bank 1, and so on up to word 31 in bank 31; the next row starts again at bank 0, with word 32, then word 33 in bank 1, and so on. So bank 0 holds words 0, 32, 64, ..., bank 1 holds words 1, 33, 65, ..., and in general word w lives in bank $w \bmod 32$, the remainder of w divided by 32.

bank 0 holds words 0, 32, 64, ...

↓

bank	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
words 0–31	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
words 32–63	32	33	34	35	36	37	38	39	40	41	42	43	44	45	46	47	48	49	50	51	52	53	54	55	56	57	58	59	60	61	62	63
words 64–95	64	65	66	67	68	69	70	71	72	73	74	75	76	77	78	79	80	81	82	83	84	85	86	87	88	89	90	91	92	93	94	95

... and so on, row after row

Figure 27: How shared memory is organised. Shared memory is built from 32 banks, drawn as the 32 columns; the numbers in the cells are word numbers. Consecutive 4-byte words go to consecutive banks, row after row, so word w lives in bank $w \bmod 32$. All 32 banks work at the same time, each delivering one word per cycle.

Shared memory: how a warp’s access is served.

When a warp reads shared memory, each of its 32 threads names one word. All 32 banks work in the same cycle, each delivering one word. So if the 32 threads need words that lie in 32 different banks, the whole warp is served in a single cycle.

Shared memory: what makes an access fast or slow.

Consider again a warp in which each thread reads one 4-byte word, this time from an array s in shared memory, with $s[0]$ in bank 0. As before, tid is a thread’s number within its warp, from 0 to 31.

- **Different banks:** $s[\text{tid}]$. Thread 0 reads $s[0]$ from bank 0, thread 1 reads $s[1]$ from bank 1, and so on up to thread 31, which reads $s[31]$ from bank 31. Every bank delivers one word, and the whole warp is served in one cycle.
- **Different words in the same bank:** $s[2*\text{tid}]$. Thread 0 reads $s[0]$ and thread 16 reads $s[32]$; both words lie in bank 0. In the same way, threads 1 and 17 both need bank 2, and so on: every bank that is used must deliver two different words. A bank can deliver only one word per cycle, so the hardware splits the access into two rounds, one word per bank per round [50]. This is called a **bank conflict**, here a two-way conflict, and the access runs at half speed. With $s[32*\text{tid}]$, all 32 threads need different words from bank 0: a 32-way conflict, 32 times slower.
- **The same word:** $s[0]$. If all threads read the very same word, the bank delivers it once and the hardware passes it to all of them. This is called a **broadcast**, and it causes no conflict.

Figure 28 draws the first two cases, with 16 banks and 16 threads per warp instead of 32 to keep the picture small.

Shared memory: what this asks of a kernel.

Data and indexing should be arranged so that the threads of a warp read words in different banks, or the very same word. Consecutive words for consecutive threads are always safe. When a stride cannot be avoided, a common remedy is **padding**. Take a table of 32×32 values stored row by row: all values of one column lie in the same bank, so reading a column causes a 32-way conflict. Adding one unused word at the end of every row shifts each row by one bank, and the 32 values of a column then fall in 32 different banks. Section 12.4 meets a bank conflict in a real kernel and removes it.

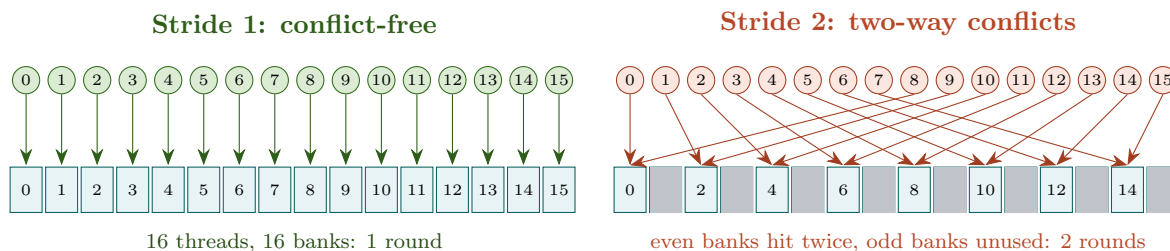


Figure 28: Shared-memory access, drawn with 16 banks and 16 threads per warp to keep the picture small (real GPUs have 32 of each). Left: with `s[tid]`, every bank delivers one word, and the access takes one round. Right: with `s[2*tid]`, threads 0 and 8 both need bank 0, threads 1 and 9 both need bank 2, and so on, so the access takes two rounds and half of the banks sit idle.

12.3 Making the computation efficient

Sections 12.1 and 12.2 made sure that each SM has enough warps to switch between and that each access to memory wastes as little as possible. What remains is the computation the kernel itself performs. Three questions decide how fast a kernel can run: does it launch enough threads to keep the whole GPU busy, how much arithmetic does it do for each byte it moves, and how expensive is each operation. This subsection takes them in turn, and then collects all the guidelines of this section in one table.

Enough threads to fill the whole GPU.

An H100 has 132 SMs. Each SM has 64 warp slots, 16 in each of its four processing blocks, so it can hold at most $64 \times 32 = 2,048$ threads at the same time, and only if its other limits, the registers, the shared memory and the block slots, allow it (Section 12.1). To keep every SM fully occupied, a kernel therefore needs at least $132 \times 2,048 = 270,336$ threads at the same time, grouped into thread blocks, because blocks are what the GigaThread engine hands out to the SMs (Section 10.2). A block can have at most 1,024 threads, so filling one SM takes at least two blocks, and filling all 132 SMs takes at least 264 blocks. A kernel launched with only 66 blocks, for example, falls short in two ways, however well each block is written:

1. **Too few SMs are used.** A block runs on a single SM, so at most 66 of the 132 SMs have anything to do.
2. **Too few threads are present.** The 66 blocks together hold at most $66 \times 1,024 = 67,584$ threads, only a quarter of the 270,336 that the GPU can hold.

Launching just enough blocks to fill the GPU once is not enough either, because of what happens at the end of a kernel. A kernel usually has more blocks than the GPU can hold at the same time,

because the data it works on is large; the GigaThread engine starts as many as fit and hands out a new block to an SM whenever one of its blocks finishes (Section 10.2). Blocks do not all take the same time, however. Near the end of the kernel, no blocks are left to hand out, and an SM that finishes its last block has nothing to do while other SMs are still running theirs. If the kernel has only as many blocks as the GPU can hold at once, all of its blocks run in this final round, and the idle time can be a large part of the total. If the kernel has many times more blocks, each SM runs many rounds of blocks one after another, and only the last round is uneven, so the idle time at the end becomes a small part of the total.

The guideline is therefore to choose algorithms that split into many independent pieces, and to launch many more blocks than the GPU can hold at the same time.

Arithmetic against memory traffic: the roofline.

Every kernel does two things: it moves data between main memory and the SMs, and it does arithmetic on that data. The GPU has a limit for each. An H100 can perform about 67 TFLOP/s of ordinary 32-bit arithmetic (Section 10.3) and can move about 3.35 TB/s between main memory and the SMs (Section 10) [62]. Which of the two limits a kernel reaches first depends on how much arithmetic it performs for each byte it moves. This ratio is called the **arithmetic intensity** I of the kernel, measured in floating-point operations (FLOP) per byte.

Vector addition (Listing 3) shows how this works. For each element, it reads two 4-byte values and writes one, so it moves 12 bytes, and it performs one addition: $I = 1/12$, about 0.083 FLOP per byte. At 3.35 TB/s, main memory can supply the data for $3.35 \times 10^{12}/12 \approx 0.28 \times 10^{12}$ additions per second, so the kernel can never exceed about 0.28 TFLOP/s. That is about 1/240 of the 67 TFLOP/s the arithmetic units could deliver: they spend almost all of their time waiting for data.

The same reasoning applies to any kernel on the H100. For a kernel with arithmetic intensity I , main memory can supply data for at most $I \times 3.35$ TFLOP/s of arithmetic, and the arithmetic units can perform at most 67 TFLOP/s. The kernel can run no faster than the smaller of the two. The two limits are equal when $I = 67/3.35 = 20$ FLOP per byte. A kernel with an intensity lower than 20 FLOP per byte is limited by memory; a kernel with an intensity higher than 20 is limited by arithmetic. Most simple kernels, like vector addition, sit far below 20 and are limited by memory. A large matrix multiplication sits far above it, because it reuses every number it loads in many multiply-adds (Section 13).

The same holds for any GPU, with that GPU's own two numbers. Write P_{peak} for its peak arithmetic rate and B_{peak} for its memory bandwidth. The speed of a kernel with arithmetic intensity I , measured in floating-point operations per second, can then be at most

$$P_{\text{attainable}} = \min(P_{\text{peak}}, I \times B_{\text{peak}}). \quad (5)$$

This is the **roofline model** [77], named after the shape of its graph (Figure 29): a rising line for the memory limit, and a flat “roof” for the arithmetic limit. The two meet at $I = P_{\text{peak}}/B_{\text{peak}}$, which is 20 FLOP per byte for the H100.

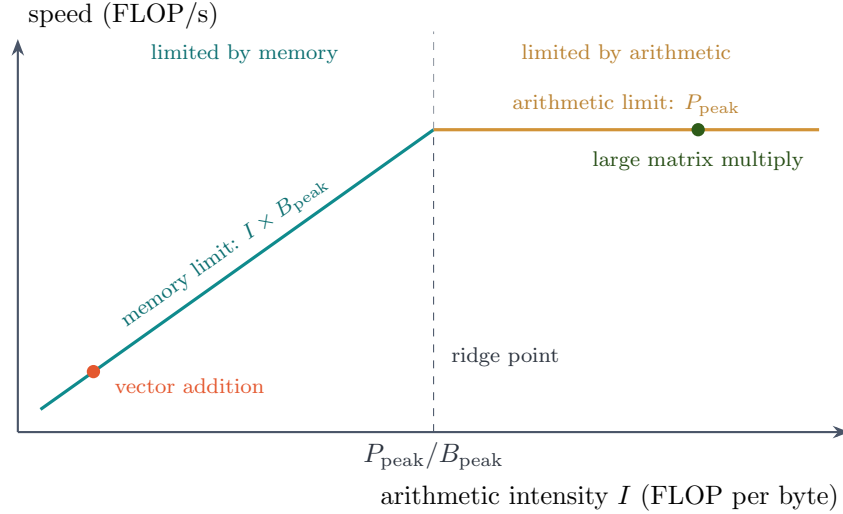


Figure 29: The roofline model, for any GPU. A kernel’s speed is limited either by memory, along the rising line $I \times B_{\text{peak}}$, or by arithmetic, along the flat roof P_{peak} , whichever is lower. The two limits meet at the ridge point $I = P_{\text{peak}}/B_{\text{peak}}$. For the H100, with $B_{\text{peak}} = 3.35 \text{ TB/s}$ and $P_{\text{peak}} = 67 \text{ TFLOP/s}$ for ordinary 32-bit arithmetic, the ridge point is at 20 FLOP per byte; vector addition ($I \approx 0.083$) sits far to its left, and a large matrix multiplication far to its right.

Move data as little as possible.

For a kernel limited by memory, the way to run faster is to raise its arithmetic intensity: to do more arithmetic for every byte fetched from main memory. Two habits help. The first is *reuse*: data that will be needed again is kept in registers or shared memory, instead of being read from main memory a second time. The second is *recomputation*: a value that can be computed again from data already on the chip is often cheaper to recompute than to store in main memory and read back. Together with the access patterns of Section 12.2, which make sure that the bytes fetched are bytes used, these habits let a kernel get the most out of the memory bandwidth it has.

Make each operation cheaper: lower precision.

Once the data is on the chip, the operations themselves can be made cheaper. GPUs offer fast, slightly less accurate versions of mathematical functions such as e^x and $\sin x$ (`_expf` and `_sinf` in CUDA). Narrower number formats, such as 16-bit instead of 32-bit values, help in two ways: each value takes fewer bytes, so the same memory bandwidth moves more values, and the tensor cores perform many more operations per second on narrow formats (Section 13). Where the final result is not affected by the small extra error, this speed comes for free [69].

All the guidelines together.

Table 9 collects the guidelines of this section, together with the guideline of Section 11.3 to keep the threads of a warp on the same path. For each one, it gives the reason and the design idea of Section 8 that the guideline follows from.

Table 9: The guidelines of this section, the reason for each, and the design idea it follows from.

Guideline	Why it matters	Design idea	Section
Launch many more blocks than the GPU can hold at once	Every SM gets blocks to run, and few SMs sit idle while the last blocks finish	#1: many cores to keep busy	Section 12.3
Keep enough warps on each SM (limit registers per thread and shared memory per block)	The more warps an SM holds, the more often its schedulers find a ready warp while others wait for memory	#3: hide memory delays by switching between threads	Section 12.1
Let neighbouring threads access neighbouring addresses	Main memory delivers whole 32-byte sectors; bytes fetched but not used waste bandwidth	#2: the 32 threads of a warp access memory together	Section 12.2
Let the threads of a warp use different shared-memory banks, or the same word	Each bank delivers only one word per cycle, so threads that need different words from the same bank wait in turn	#2: the 32 threads of a warp access memory together	Section 12.2
Keep the threads of a warp on the same path	A warp runs different paths one after the other, with part of its threads switched off	#2: one instruction for all 32 threads of a warp	Section 11.3
Reuse data on the chip; recompute instead of reloading	Arithmetic is cheaper than main-memory traffic, and most kernels are limited by memory	#1: slim cores with little cache, so reuse must be planned by the kernel	Section 12.3
Use lower precision where accuracy allows	More values move per byte, and tensor cores run much faster on narrow formats	(tensor cores, Section 13)	Section 12.3

12.4 A worked example: summing a million numbers into one

We now apply these guidelines to one complete problem: adding up all the numbers in a large array to get their total. This is a different problem from vector addition (Listing 3). Vector addition adds two arrays element by element and produces a whole array of results, each one independent of the others. Here, every number in the array contributes to a single result. Combining all elements of an array into one value (their sum, maximum or minimum) is called a **reduction**. It looks trivial, yet it is the classic teaching example for GPU optimisation [24], because it is *not* naturally parallel, for the reason just given, and because getting it fast touches almost every guideline of this section. We develop the kernel in four versions, each fixing one problem of the last.

The idea: add in a tree.

On a CPU, summing n numbers is a loop of $n - 1$ additions, each one waiting for the previous result. But addition does not care about grouping: $(a + b) + (c + d)$ equals $((a + b) + c) + d$. So we can arrange the additions as a tree (Figure 30). In the first step, pairs of numbers are added, all pairs at the same time; in the second step, pairs of those results; and so on. Eight numbers need only three steps, and a million need only twenty ($\log_2 n$ steps for n numbers), as long as there are enough ALUs to do each step's additions in parallel.

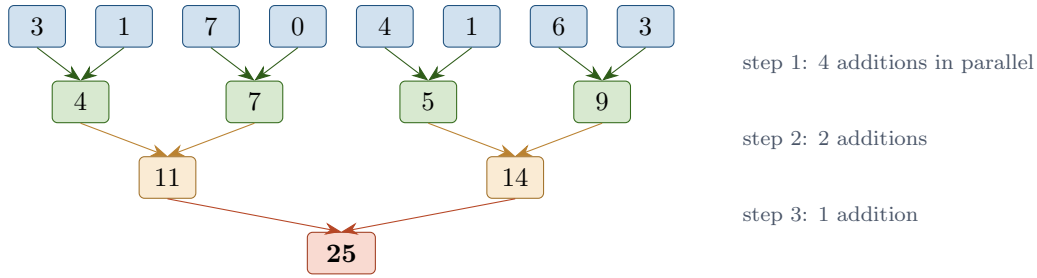


Figure 30: Tree reduction of eight values. The steps run one after another, $\log_2 8 = 3$ steps in all, and the additions *within* each step run in parallel: 4 additions, then 2, then 1. The total is still $8 - 1 = 7$ additions, the same as a simple loop, but they take 3 steps instead of 7.

Fitting the tree onto a GPU.

On a GPU, the natural plan is to give each number to its own thread and let the threads build the tree together. But each step of the tree reads the results of the step before, so before a step can start, every thread must know that all additions of the previous step are finished. Threads can wait for each other like this only at a barrier, and a barrier works only among the threads of one thread block (`_syncthreads()`, Section 9). Threads of different blocks may run on different SMs, or not even at the same time (Section 10.2), so they cannot wait for each other inside a kernel. A kernel therefore cannot build one single tree over all the numbers, as in Figure 30. Instead, it builds many small trees, one inside each block, and the results of these small trees are then added together.

To make this concrete, we use one example for the rest of this section: an array of 2^{20} (about one million) integers, summed by thread blocks of 128 threads each. (Because the numbers are integers, the additions are not floating-point operations, but the reasoning of Sections 12.1 to 12.3 about memory and arithmetic applies in the same way.) We choose 128 threads because, as Section 10.4 showed, 128 threads (four warps) is the smallest block size that gives each of the four processing blocks of an SM one warp of the block.

One launch: a small tree in each block.

A single launch of the kernel works on an array of n numbers. The array is cut into chunks of 128 numbers, one chunk for each block, so that every thread of a block has one number; the launch therefore uses $n/128$ blocks, rounded up.

1. Each thread copies its number from main memory into shared memory, the fast memory that the threads of a block share on their SM (Section 10.3). If n is not a multiple of 128, some threads of the last block have no number; they put zero into shared memory instead, which does not change the sum.
2. The threads add up the block's 128 numbers step by step, as in Figure 30, with a barrier after every step: in the first step, 64 threads each add a pair of numbers; in the next, 32 threads each add a pair of those results; and so on, for $\log_2 128 = 7$ steps.
3. One thread writes the final result, the sum of the block's 128 numbers, to main memory. This sum is called the block's **partial sum**.

After one launch, the n numbers have been replaced by $n/128$ partial sums, whose total is the same as the total of the n numbers.

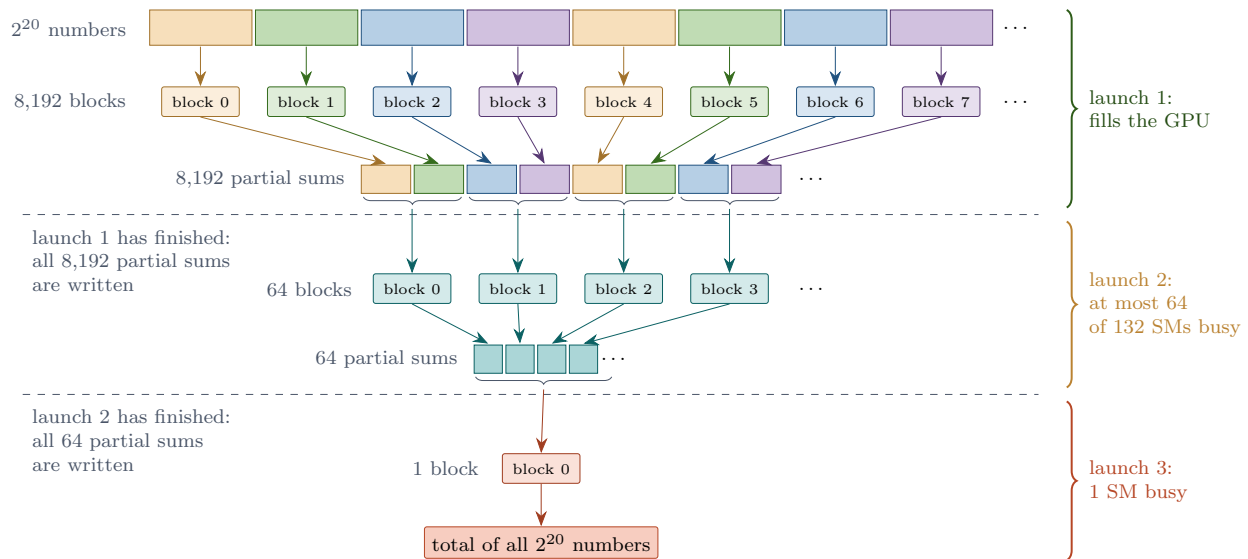


Figure 31: Summing 2^{20} numbers with a series of launches of the same kernel. In each launch, every block adds up a chunk of 128 numbers with a small tree and writes their sum, its partial sum, to main memory; the braces mark the chunk that each block of launches 2 and 3 reads. Each launch works on the partial sums of the launch before, and starts only after that launch has finished. The array shrinks by a factor of 128 per launch, so only the first launch has enough blocks to fill the GPU.

A series of launches.

Adding up the partial sums is the same problem again, only 128 times smaller. So the host program launches the same kernel again, this time on the partial sums, and repeats this, each launch working on the partial sums written by the launch before, until a launch uses a single block. That block's partial sum is the total of the whole array [24]. The launches go into the same CUDA stream, so each one starts only after the one before has finished (Section 10.4): when a launch begins, every partial sum it reads has already been written. The end of a launch thus provides the wait between blocks that a barrier cannot provide.

For our array of 2^{20} numbers, the series has three launches (Figure 31):

1. **Launch 1** works on the 2^{20} numbers with $2^{20}/128 = 8,192$ blocks and writes 8,192 partial sums.
2. **Launch 2** works on these 8,192 numbers with $8,192/128 = 64$ blocks and writes 64 partial sums.
3. **Launch 3** works on these 64 numbers with a single block, in which 64 threads have a number and 64 threads use zero. Its partial sum is the total of all 2^{20} numbers.

Listing 4 shows the host program in full: it copies the numbers to the GPU as in Listing 3, launches the kernel until one number is left, and copies that number back. In it, `reduce` stands for any of the kernel versions below. The host program itself performs no additions: it only starts the launches, and every addition is done by the GPU.

The program does not have to do anything special to put all launches into the same CUDA stream. A launch that names no stream, as every launch in Listing 4 does, goes into the GPU's **default stream**, and the operations of one stream run in the order in which the program issued them [64]. The final `cudaMemcpy` also goes into the default stream, so it starts only after the last launch has finished, and the host program waits until the copy is complete before it prints the total.

Listing 4: Host program: copy the numbers to the GPU, launch the reduction kernel repeatedly until one number is left, and copy that number back.

```

1  #include <stdio>
2  #include <cuda_runtime.h>
3
4  // reduce stands for one of the kernels reduce_v1 ... reduce_v4 (Listings 5-8)
5  __global__ void reduce(const int *in, int *partial, int n);
6
7  int main()
8  {
9      const int N = 1 << 20;                // 2^20 numbers to add
10     int *h_data = new int[N];             // the numbers, in host memory
11     for (int i = 0; i < N; i++) h_data[i] = 1; // example: all ones
12
13     int *d_data, *d_buf1, *d_buf2;         // arrays in GPU memory
14     cudaMalloc(&d_data, N * sizeof(int));
15     cudaMalloc(&d_buf1, (N / 128) * sizeof(int)); // room for 8,192 partial sums
16     cudaMalloc(&d_buf2, (N / 128) * sizeof(int)); // a second, spare buffer
17     cudaMemcpy(d_data, h_data, N * sizeof(int), cudaMemcpyHostToDevice);
18
19     const int *in = d_data;                // input of the next launch
20     int *out = d_buf1;                     // output of the next launch
21     int *spare = d_buf2;
22     int n = N;                             // how many numbers are left to add
23
24     while (n > 1) {
25         int blocks = (n + 127) / 128;      // n / 128, rounded up
26         reduce<<<blocks, 128>>>(in, out, n); // one partial sum per block
27         n = blocks;                        // partial sums: the new array
28         in = out;                          // ... and the next input
29         int *t = out; out = spare; spare = t; // swap the two buffers
30     }
31
32     int total;                             // copy the one result back
33     cudaMemcpy(&total, in, sizeof(int), cudaMemcpyDeviceToHost);
34     printf("total = %d\n", total);          // prints 1048576
35
36     cudaFree(d_data); cudaFree(d_buf1); cudaFree(d_buf2);
37     delete[] h_data;
38     return 0;
39 }

```

The number of blocks for a launch is $n/128$, rounded up, so that every number has a thread. In C, dividing one integer by another rounds *down*, so the program computes $(n + 127) / 128$ instead, the same trick as in Listing 3: adding 127 before dividing turns rounding down into rounding up. In our example, the three launches compute:

- launch 1: $n = 2^{20}$, and $(2^{20} + 127)/128 = 8,192$ blocks;
- launch 2: $n = 8,192$, and $(8,192 + 127)/128 = 64$ blocks;
- launch 3: $n = 64$, and $(64 + 127)/128 = 1$ block.

In launch 3, plain division would give $64/128 = 0$ blocks, and the last 64 numbers would never be added.

The program uses two buffers for partial sums, `d.buf1` and `d.buf2`, and swaps them after every launch. The reason is that a launch must not write its partial sums into the same array that it is reading. Blocks run in no fixed order, so if block 1 wrote its partial sum into element 1 of its own input, it could overwrite a number that block 0 has not yet read. So each launch reads one buffer and writes the other: in our example, launch 1 reads the original numbers and writes `d.buf1`, launch 2 reads `d.buf1` and writes `d.buf2`, and launch 3 reads `d.buf2` and writes `d.buf1`. The three lines at the end of the loop arrange this: the output just written becomes the next input (`in = out`), and the next output goes into the other buffer (the swap of `out` and `spare`).

What the series of launches costs.

The shrinking array has a price, and it comes from the nature of the problem rather than from the GPU. Launch 1, with 8,192 blocks, has far more blocks than an H100 can hold at once: a block of 128 threads is 4 warps, so an SM holds at most $64/4 = 16$ such blocks, and the whole GPU $132 \times 16 = 2,112$ (Section 12.3). Launch 2, however, has only 64 blocks, so at most 64 of the 132 SMs have anything to do, and each of them holds only 4 of its 64 warps. Launch 3 has a single block, which runs on one SM while the other 131 SMs are idle. The later launches therefore use the GPU very poorly, and each launch also has a fixed cost of its own for being started. Nothing can be done about this: as the numbers are added up, fewer and fewer of them remain, and there is less and less to do in parallel. The cost stays small because almost all of the additions happen in launch 1. Adding up 2^{20} numbers takes $2^{20} - 1 = 1,048,575$ additions; launch 1 performs $8,192 \times 127 = 1,040,384$ of them, over 99%, launch 2 performs $64 \times 127 = 8,128$, and launch 3 the last 63.

Launch 1 also follows the guideline on main-memory access: each thread reads the element whose index is its own global thread number, so the loads are coalesced (Section 12.2). All the versions below follow this plan of launches. They differ only in how the threads of a block add up their 128 numbers inside shared memory.

Version 1: the obvious kernel.

In all versions, `ltid` is a thread's number within its block, from 0 to 127, and `gtid` its number within the whole launch. Listing 5 is the natural first attempt; it carries out the three steps of one launch described above.

1. Each thread copies element `gtid` of the input from main memory into a shared-memory array `s_data`, at position `ltid`, or zero if `gtid` is not smaller than n . The block then waits at a barrier, so that no thread starts adding before all 128 numbers are in shared memory.
2. The block adds up its 128 numbers in shared memory with the tree of Figure 30, in $\log_2 128 = 7$ steps. In each step, numbers are added in pairs; the code calls the distance between the two numbers of a pair `s`.
 - **Step 1** ($s = 1$). The 128 numbers form 64 pairs of neighbours: elements 0 and 1, elements 2 and 3, and so on up to elements 126 and 127. For each pair, the thread whose `ltid` is the even number adds the right-hand element into its own element: thread 0 adds `s_data[1]` to `s_data[0]`, thread 2 adds `s_data[3]` to `s_data[2]`, and so on. Afterwards, elements 0, 2, 4, ..., 126 hold 64 sums of two numbers each.
 - **Step 2** ($s = 2$). These 64 sums lie two places apart, so they are paired again with a distance of 2: elements 0 and 2, elements 4 and 6, and so on. Now the threads whose `ltid`

is a multiple of 4 add: thread 0 adds `s_data[2]` to `s_data[0]`, thread 4 adds `s_data[6]` to `s_data[4]`, and so on. Afterwards, elements 0, 4, 8, ..., 124 hold 32 sums of four numbers each.

- **Steps 3 to 7.** Each step doubles the distance `s` and halves the number of threads that add: in the step with distance `s`, the threads whose `ltid` is a multiple of $2 \times s$ add the element `s` places to their right. In the last step ($s = 64$), only thread 0 adds: `s_data[64]` to `s_data[0]`.

The code picks the threads that add with the test `ltid % (2 * s) == 0`. A barrier after each step makes sure that a step is finished before the next one reads its results.

3. After the 7 steps, `s_data[0]` holds the sum of the block's 128 numbers, and thread 0 writes it to main memory as the block's partial sum.

Listing 5: Version 1: interleaved addressing (correct but slow).

```

1  __global__ void reduce_v1(const int *in, int *partial, int n)
2  {
3      __shared__ int s_data[128];           // one element per thread
4      int ltid = threadIdx.x;               // number within the block
5      int gtid = blockIdx.x * blockDim.x + threadIdx.x; // number within the launch
6
7      s_data[ltid] = (gtid < n) ? in[gtid] : 0; // main memory -> shared
8      __syncthreads();                     // all 128 numbers loaded
9
10     for (int s = 1; s < blockDim.x; s *= 2) { // s = 1, 2, 4, ..., 64
11         // threads that add: ltid = 0, 2, 4, ..., 126 when s = 1,
12         //                      ltid = 0, 4, 8, ..., 124 when s = 2, ...
13         if (ltid % (2 * s) == 0)           // ltid a multiple of 2*s
14             s_data[ltid] += s_data[ltid + s];
15         __syncthreads();                   // finish this step
16     }
17     if (ltid == 0) partial[blockIdx.x] = s_data[0]; // the block's partial sum
18 }

```

The kernel gives the right answer, but it is slow, for two reasons.

- **Every warp diverges.** In step 1, only the threads with an even `ltid` have an addition to do, so *every* warp contains both threads that add and threads that do not. Each warp therefore runs the addition with half of its threads switched off (Section 11.3). In later steps, the threads that add become rarer but stay spread across all four warps, so every warp keeps running with most of its threads switched off (Figure 32, top).
- **The remainder operation is slow.** The test `ltid % (2 * s)` needs the remainder of a division. GPUs have no single fast instruction for integer division, so this test takes many instructions.

Version 2: removing divergence.

Version 1 is slow because the threads that add are spread out: in step 1 they are threads 0, 2, 4, ..., 126, so every warp contains some of them. Listing 6 keeps exactly the same additions, with the same pairs of elements added in every step, but gives them to different threads, so that the threads that add are always the lowest-numbered ones.

To do this, each thread first computes the element it adds into, $\text{index} = 2 \times s \times \text{ltid}$, and then adds the element s places to its right: $\text{s_data}[\text{index}] += \text{s_data}[\text{index} + s]$. It does this only if index lies within the block's 128 elements.

- **Step 1** ($s = 1$). Here $\text{index} = 2 \times \text{ltid}$. Thread 0 adds $\text{s_data}[1]$ to $\text{s_data}[0]$, thread 1 adds $\text{s_data}[3]$ to $\text{s_data}[2]$, thread 2 adds $\text{s_data}[5]$ to $\text{s_data}[4]$, and so on up to thread 63, which adds $\text{s_data}[127]$ to $\text{s_data}[126]$. These are the same 64 pairs as in Version 1, but they are now added by threads 0 to 63. Threads 64 to 127 get an index of 128 or more and add nothing.
- **Step 2** ($s = 2$). Here $\text{index} = 4 \times \text{ltid}$. Thread 0 adds $\text{s_data}[2]$ to $\text{s_data}[0]$, thread 1 adds $\text{s_data}[6]$ to $\text{s_data}[4]$, and so on up to thread 31, which adds $\text{s_data}[126]$ to $\text{s_data}[124]$. Again these are the same 32 pairs as in Version 1, now added by threads 0 to 31.
- **Steps 3 to 7**. In the step with distance s , threads 0 to $64/s - 1$ add, and all other threads add nothing. In the last step ($s = 64$), only thread 0 adds: $\text{s_data}[64]$ to $\text{s_data}[0]$, as in Version 1.

Listing 6: Version 2: the working threads are the lowest-numbered ones.

```

1  __global__ void reduce_v2(const int *in, int *partial, int n)
2  {
3      __shared__ int s_data[128];           // one element per thread
4      int ltid = threadIdx.x;               // number within the block
5      int gtid = blockIdx.x * blockDim.x + threadIdx.x; // number within the launch
6
7      s_data[ltid] = (gtid < n) ? in[gtid] : 0; // main memory -> shared
8      __syncthreads();                     // all 128 numbers loaded
9
10     for (int s = 1; s < blockDim.x; s *= 2) { // s = 1, 2, 4, ..., 64
11         // threads that add: ltid = 0, 1, 2, ..., 63 when s = 1,
12         //                      ltid = 0, 1, 2, ..., 31 when s = 2, ...
13         int index = 2 * s * ltid;           // element this thread adds to
14         if (index < blockDim.x)             // index inside the block
15             s_data[index] += s_data[index + s]; // add the element s places right
16         __syncthreads();                     // finish this step
17     }
18     if (ltid == 0) partial[blockIdx.x] = s_data[0]; // the block's partial sum
19 }

```

Version 2 removes one of the two problems of Version 1, but brings in a new one.

- **No warp diverges any more.** With 128 threads (four warps), step 1 gives an addition to threads 0–63, which are all the threads of warps 0 and 1, while warps 2 and 3 have no addition to do and skip the step entirely. In step 2, only warp 0 has additions to do. Each warp is now either completely busy or completely idle, so no warp diverges until fewer than 32 threads have additions left.
- **The threads of a warp now hit the same shared-memory banks.** Recall from Section 12.2 that word w of shared memory lies in bank $w \bmod 32$, and that two threads of a warp that need different words from the same bank must wait in turn. In step 1, each thread of warp 0 adds into word $2 \times \text{ltid}$:

- threads 0 to 15 use words 0, 2, 4, ..., 30, which lie in banks 0, 2, 4, ..., 30;

- threads 16 to 31 use words 32, 34, 36, ..., 62, which lie in the *same* banks 0, 2, 4, ..., 30.

So threads 0 and 16 both need bank 0, threads 1 and 17 both need bank 2, and so on. Each of these banks must deliver two different words, so the access takes two rounds instead of one: a two-way bank conflict. The same happens for the words `index + s` that the threads read. In step 2, the threads of warp 0 use words 0, 4, 8, ..., and four of them share each bank in use, a four-way conflict, so every access takes four rounds. The later steps also have conflicts, until only one thread is left.

Version 3: removing bank conflicts.

Version 2 has bank conflicts because the threads of a warp use words that are spaced apart: two words apart in step 1, four words apart in step 2. Version 3 removes this spacing. It pairs the numbers differently, so that the threads of a warp always use consecutive words.

Instead of adding neighbours, each step adds the second half of the remaining numbers onto the first half. Thread `ltid` adds `s_data[ltid + s]` into `s_data[ltid]`, where the distance `s` now starts at 64 and halves in every step.

- **Step 1** ($s = 64$). Threads 0 to 63 add: thread 0 adds `s_data[64]` to `s_data[0]`, thread 1 adds `s_data[65]` to `s_data[1]`, and so on up to thread 63, which adds `s_data[127]` to `s_data[63]`. Afterwards, elements 0 to 63 hold 64 sums of two numbers each.
- **Step 2** ($s = 32$). Threads 0 to 31 add: thread 0 adds `s_data[32]` to `s_data[0]`, and so on up to thread 31, which adds `s_data[63]` to `s_data[31]`. Afterwards, elements 0 to 31 hold 32 sums of four numbers each.
- **Steps 3 to 7**. The distance `s` halves to 16, 8, 4, 2 and 1, and threads 0 to $s - 1$ add. In the last step ($s = 1$), only thread 0 adds: `s_data[1]` to `s_data[0]`.

The pairs are different from those of Versions 1 and 2, but the order in which numbers are added does not change their sum (Figure 30), so the result is the same. In Listing 7, the loop starts with $s = 128/2 = 64$, and `s >>= 1` halves `s` after every step (shifting a number one bit to the right divides it by 2).

Listing 7: Version 3: sequential addressing.

```
1  __global__ void reduce_v3(const int *in, int *partial, int n)
2  {
3      __shared__ int s_data[128];           // one element per thread
4      int ltid = threadIdx.x;               // number within the block
5      int gtid = blockIdx.x * blockDim.x + threadIdx.x; // number within the launch
6
7      s_data[ltid] = (gtid < n) ? in[gtid] : 0; // main memory -> shared
8      __syncthreads();                     // all 128 numbers loaded
9
10     for (int s = blockDim.x / 2; s > 0; s >>= 1) { // s = 64, 32, 16, ..., 1
11         // threads that add: ltid = 0, 1, ..., 63 when s = 64,
12         //                   ltid = 0, 1, ..., 31 when s = 32, ...
13         if (ltid < s)                       // ltid below s
14             s_data[ltid] += s_data[ltid + s]; // add the element s places right
15         __syncthreads();                     // finish this step
16     }
17     if (ltid == 0) partial[blockIdx.x] = s_data[0]; // the block's partial sum
18 }
```

Version 3 keeps the improvement of Version 2 and removes its new problem.

- **No warp diverges while $s \geq 32$.** The threads that add are threads 0 to $s - 1$, the lowest-numbered ones, as in Version 2. In step 1, warps 0 and 1 are completely busy and warps 2 and 3 completely idle; in step 2, only warp 0 is busy. Only in the last five steps, when fewer than 32 threads add, does warp 0 diverge.
- **No bank conflicts.** In step 1, the 32 threads of warp 0 add into words 0 to 31, which lie in banks 0 to 31, one word in each bank. The words they read, 64 to 95, also lie in banks 0 to 31, one word in each bank. Warp 1 does the same with words 32 to 63 and 96 to 127. In every step, the threads of a warp use consecutive words, every bank delivers at most one word, and each access takes a single round.

Figure 32 compares how the three versions assign threads to elements.

Version 1: interleaved, divergent

active threads 0, 2, 4, ...: every warp half idle

$s = 1$	T0		T2		T4		T6		T8		T10		T12		T14	
$s = 2$	T0				T4				T8				T12			

Version 2: contiguous threads, strided data

no divergence, but stride-2s accesses: bank conflicts

$s = 1$	T0		T1		T2		T3		T4		T5		T6		T7	
$s = 2$	T0				T1				T2				T3			

Version 3: sequential addressing

consecutive threads, consecutive words

$s = 8$	T0	T1	T2	T3	T4	T5	T6	T7								
$s = 4$	T0	T1	T2	T3												

Figure 32: Which thread works on which element in the first two steps of the tree, drawn for a block of 16 elements. Dark cells are the elements written in that step, labelled with the thread that writes them; light cells are the second elements read. Version 1 spreads the working threads out, so every warp diverges. Version 2 gathers the working threads together, but they read every second word, which causes bank conflicts. Version 3 gathers both the threads and the words they read.

Version 4: finishing inside a single warp.

Version 3 still wastes time in its last five steps. When s is 16, 8, 4, 2 and 1, only threads 0 to $s - 1$ add, and all of them belong to warp 0. Yet each of these steps still ends with `__syncthreads()`, which makes all four warps of the block wait, although warps 1, 2 and 3 have nothing left to do. And each step still passes its values through shared memory.

Version 4 (Listing 8) removes both costs from these last steps by using a **warp shuffle**. A warp shuffle lets a thread read a value directly from a register of another thread in the same warp, without going through shared memory. The shuffle used here is `__shfl_down_sync(0xffffffff, v, s)`: every thread of the warp receives the value of the variable v held by the thread s places further along, so thread l_{tid} receives the v of thread $l_{tid} + s$. The first argument, `0xffffffff`, is a 32-bit number with all 32 bits set, one bit for each thread of the warp: it says that all 32 threads take part. The `_sync` in the name means that the shuffle itself brings these 32 threads together before they exchange values, so no separate barrier is needed [64].

Version 4 adds up the block's 128 numbers in four stages.

1. **One step in shared memory.** As in Version 3, threads 0 to 63 add `s_data[ltid + 64]` into `s_data[ltid]`, followed by a barrier. Elements 0 to 63 now hold 64 sums. (The loop in Listing 8 runs while $s > 32$; with 128 threads, that is the single step with $s = 64$.)
2. **From shared memory into registers.** Warp 0 takes over. Each of its 32 threads adds two of the 64 sums and keeps the result in its own register variable v : thread l_{tid} computes $v = s_data[l_{tid}] + s_data[l_{tid} + 32]$. Now 32 sums are left, one in each thread of warp 0.
3. **Five shuffle steps.** With $s = 16$, thread 0 adds the v of thread 16 to its own, thread 1 adds that of thread 17, and so on up to thread 15, which adds that of thread 31; threads 0 to 15 now hold 16 sums. With $s = 8$, threads 0 to 7 then hold 8 sums, and so on, until after $s = 1$ thread 0 holds the sum of all 128 numbers. (A thread with no partner s places further along, such as thread 20 when $s = 16$, receives its own value back; its result is never used.)
4. **Writing the result.** Thread 0 writes its v to main memory as the block's partial sum.

Compared with Version 3, the last five steps have no block-wide barriers, since only warp 0 takes part and the shuffle keeps its threads together. They do not touch shared memory, since the values move from register to register. And they do not diverge, since all 32 threads of warp 0 execute every shuffle and addition, even those whose results are not used.

Listing 8: Version 4: the last warp finishes with warp shuffles (assumes 32-thread warps).

```

1  __global__ void reduce_v4(const int *in, int *partial, int n)
2  {
3      __shared__ int s_data[128];           // one element per thread
4      int ltid = threadIdx.x;               // number within the block
5      int gtid = blockIdx.x * blockDim.x + threadIdx.x; // number within the launch
6
7      s_data[ltid] = (gtid < n) ? in[gtid] : 0; // main memory -> shared
8      __syncthreads();                     // all 128 numbers loaded
9
10     for (int s = blockDim.x / 2; s > 32; s >>= 1) { // with 128 threads: s = 64 only
11         if (ltid < s)                         // threads 0, 1, ..., 63
12             s_data[ltid] += s_data[ltid + s];
13         __syncthreads();                     // finish this step
14     }
15     if (ltid < 32) {                         // warp 0 finishes alone
16         int v = s_data[ltid] + s_data[ltid + 32]; // 64 sums -> 32, in registers
17         for (int s = 16; s > 0; s >>= 1) // s = 16, 8, 4, 2, 1
18             v += __shfl_down_sync(0xffffffff, v, s); // add v of thread ltid + s
19         if (ltid == 0) partial[blockIdx.x] = v; // the block's partial sum
20     }
21 }

```

What the four versions show.

None of the four versions changes the algorithm or the number of additions: every version adds up the block's 128 numbers in 7 steps of a tree. Each version only changes *which thread* performs each addition and *which words* of shared memory it uses, so that the code fits the hardware better:

- Version 2 keeps the threads of a warp on the same path (Section 11.3);
- Version 3 lets the threads of a warp use different shared-memory banks (Section 12.2);
- Version 4 lets the last warp work on its own, from register to register, without barriers that make the whole block wait.

These are the same guidelines that Table 9 collected for any kernel. The kernel could be improved further; for example, each thread could add several numbers while it loads them into shared memory, so that each block sums more numbers and fewer blocks and launches are needed [24]. These four versions are enough to show the idea, and we stop here.

13 GPUs in the Age of AI

Since about 2017, the most important use of data-centre GPUs has shifted from scientific computing, also called high-performance computing (HPC), to artificial intelligence (AI). The three design ideas of Section 8 still describe these GPUs, but each new generation has added hardware aimed at the work that AI needs. This section follows these changes in NVIDIA's data-centre GPUs, in four

steps: units that multiply matrices, ways to feed those units with data, smaller numbers, and chips built from more than one piece of silicon.

13.1 Why AI means matrix multiplication

A neural network is, at its heart, a long chain of very large matrix multiplications: each layer multiplies its input (a matrix of numbers) by a matrix of learned weights. Training a large network repeats these multiplications an enormous number of times. Matrix multiplication suits a GPU well: the same multiply-and-add operation is repeated on large amounts of data, and every element of the result can be computed independently. Unlike vector addition (Section 12.3), it also reuses each number it loads many times, so its arithmetic intensity is high and the GPU’s full arithmetic power can be put to use.

A short calculation shows how high. Multiplying two $N \times N$ matrices of 32-bit numbers takes N^3 multiply-adds, that is $2N^3$ floating-point operations. The data it must move are the two input matrices and the result: $3N^2$ numbers of 4 bytes each, $12N^2$ bytes. If each number moved between main memory and the chip only once, the arithmetic intensity would be

$$I = \frac{2N^3}{12N^2} = \frac{N}{6} \text{ FLOP per byte.}$$

For $N = 4,096$ this is about 683 FLOP per byte, far above the H100’s ridge point of 20 FLOP per byte and about 8,000 times the 0.083 FLOP per byte of vector addition (Section 12.3). A real kernel cannot keep whole matrices on the chip; it works on blocks (tiles) of them, so its intensity is lower than this ideal value.

13.2 Tensor cores: matrix units inside the SM

NVIDIA’s Volta V100 (2017) added **tensor cores**: units that, instead of one multiply-add per ALU per cycle, multiply two small matrices and add the result to a third in one operation. The V100 has 640 tensor cores, eight per SM (the H100 and B200 have four per SM, Section 14.2), and NVIDIA quotes a peak of 125 TFLOP/s for them, with 16-bit inputs; the same GPU’s ordinary 32-bit units reach 15.7 TFLOP/s [59], about one eighth of that.

Unlike an ordinary instruction, which each thread carries out on its own values, a tensor-core operation multiplies small blocks of matrices (tiles) whose values belong to many threads together. How many threads start such an operation, and how they start it, has changed several times since Volta. Section 14.5 describes how it works on the H100 and the B200.

As the tensor cores became faster, the main difficulty became to *feed* them with data quickly enough.

13.3 Feeding the tensor cores

NVIDIA’s answers to the feeding problem form a progression: each step takes a data-movement job away from the threads and gives it to dedicated hardware.

- **Asynchronous copies (Ampere, 2020).** An instruction copies data from main memory directly into shared memory, without passing through the threads’ registers [60]. The thread that starts the copy can continue computing while the copy happens [64], so the next piece of data can be loaded while the current one is used.

- **The Tensor Memory Accelerator (Hopper, 2022).** With the copy engine of Section 10.3, the addresses of a whole block of a matrix are generated by the hardware instead of by the threads [48].
- **Thread block clusters (Hopper, 2022).** Because the blocks of a cluster (Section 9.2) can read each other's shared memory, data loaded by one block can be used by the others [53, 64].
- **Tensor memory (Blackwell, 2024).** Each SM has 256 KB of tensor memory, in which the tensor cores keep their results instead of the threads' registers, and two blocks on neighbouring SMs can work together on one larger matrix operation [27, 55].

13.4 Smaller numbers

Neural networks tolerate low numerical precision well. Smaller numbers take less memory and less memory traffic, and a tensor core can process more of them per cycle: on Hopper, for example, 8-bit numbers double the throughput compared with 16-bit numbers [48]. Each generation has therefore added smaller formats to its tensor cores:

- *Volta (2017)*: 16-bit floating-point inputs [59];
- *Ampere (2020)*: TF32, which keeps the 8-bit exponent of a 32-bit number but only 10 bits of mantissa (19 bits in all, with the sign), and BF16, another 16-bit format [60];
- *Hopper (2022)*: two 8-bit formats, FP8 [48];
- *Blackwell (2024)*: 6- and 4-bit formats, FP6 and FP4. To make such small numbers usable, a block of values shares one scale factor: 16 values in NVIDIA's NVFP4 format, 32 values in the community-defined MXFP4 format [54, 63, 69].

13.5 Beyond one piece of silicon

Finally, chips have reached the largest size that can be manufactured in one piece. Blackwell GPUs therefore join two dies of this largest size with a 10 TB/s link into a single, unified GPU [63]. Its successor, Rubin, which went into full production in 2026 [57], keeps two dies, moves to 288 GB of the new HBM4 memory with a quoted 22 TB/s of bandwidth, and pushes further towards small number formats [71].

Taken together, these changes show how the GPU has adapted to AI: the general-purpose cores of the earlier sections remain, and around them each generation adds units for matrix arithmetic, hardware that moves data to those units, smaller number formats, and more silicon per GPU.

14 NVIDIA and AMD: Two Implementations of the Same Design Ideas

So far every example has used NVIDIA hardware. AMD, the other major maker of large GPUs, builds on exactly the same three ideas, but at almost every level it has made a different implementation choice. Programmers and compiler engineers who move between the two usually discover these differences one surprise at a time; this section collects them in one place. It is the most technical section of the paper, and a reader meeting GPUs for the first time can skim it and return later. For AMD we use its data-centre GPUs, the Instinct MI300X (2023) and MI355X (2025); for NVIDIA, the H100 and B200 of Section 10.

Different names for the same things.

Before comparing the two designs, we first clear up their vocabulary. NVIDIA and AMD often use different names for the same concept or the same part of the hardware, and a reader who meets AMD’s documentation after NVIDIA’s can easily take a familiar idea for a new one. AMD’s programming language, **HIP**, is a close copy of CUDA (Section 7.4), so the programs look almost the same; only the names differ. Table 10 lists the names used in the rest of this section, and Table 4 gave the corresponding names in other programming languages.

Table 10: The same concepts under NVIDIA’s and AMD’s names.

Concept	NVIDIA (CUDA)	AMD (HIP)
Software platform (compilers, libraries, tools)	CUDA platform	ROCm platform
Programming language	CUDA C++	HIP C++
The GPU’s building block, the “core” of Section 8	streaming multiprocessor (SM)	compute unit (CU)
One of the four parts of that core, each with its own arithmetic units	processing block	SIMD unit
Group of threads placed on one core as a whole; its threads can share memory and wait for each other at a barrier	thread block	work-group
Group of threads that execute one instruction together	warp (32 threads)	wavefront (64 threads on data-centre GPUs)
One thread	thread	thread, or work-item
Fast memory shared by the threads of a block	shared memory	local data share (LDS)
Hardware that computes a value once for a whole warp or wavefront, instead of once per thread, when the value is the same for all its threads	uniform datapath	scalar unit

14.1 Two family trees

The two companies reached today’s designs by very different routes. NVIDIA has refined a single design for twenty years, while AMD changed its basic design once and later split it into two branches. Figure 33 draws both histories on one time line.

NVIDIA: one line since 2006.

NVIDIA’s G80 of 2006, of the Tesla generation, introduced the SM and, with it, CUDA (Section 7.2). Every NVIDIA GPU since then has been built from SMs and programmed with CUDA. Each new generation, given a new name, has kept this basis and added to it:

- Fermi (2010) added caches, error-correcting memory and fast 64-bit arithmetic for scientific computing (Section 7.3);
- Kepler (2012), Maxwell (2014) and Pascal (2016) made the SM more efficient and the chips larger;

- Volta (2017) added tensor cores for AI (Section 13);
- Turing (2018) added the uniform datapath (Section 10.5);
- Ampere (2020), Hopper (2022) and Blackwell (2024) enlarged the tensor cores and added features such as thread block clusters (Section 9.2);
- Rubin went into full production in 2026, with shipments starting in the same year [57].

Since Pascal (2016), NVIDIA has made separate chips for data centres and for graphics cards. Both kinds are built from SMs. On a graphics card, the SMs run the shaders of games (Section 7.1); in a data centre, they run CUDA programs for science and AI. A graphics card can run CUDA programs too, but it has fewer of the units that data-centre work needs, such as fast 64-bit arithmetic. Because the basis never changed, a CUDA program written for an old NVIDIA GPU still runs on a new one.

AMD: one change of design, then two branches.

AMD's history has more turns.

- **TeraScale (from 2007).** These GPUs already followed the three design ideas of Section 8: many cores, each running 64-thread wavefronts on 16-lane SIMD units over four cycles, and several wavefronts per core to hide memory delays [2]. But they added a fourth idea on top. Each lane of a SIMD unit could carry out several independent operations of the *same* thread at once, four or five, packed side by side into one very wide instruction. This approach is called **VLIW** (very long instruction word). It relies on the compiler to find enough independent operations inside each thread to fill every instruction. Graphics shaders often have them, for example the four components of a colour or a position, but general-purpose programs often do not, and many slots of each instruction then stayed empty [34].
- **Graphics Core Next, GCN (2012).** AMD dropped VLIW and kept only the three design ideas, as NVIDIA had done since the G80. Each lane now carries out one operation of one thread per instruction, and a compute unit has four 16-lane SIMD units running 64-thread wavefronts, plus a scalar unit [1]. To keep the units busy, GCN relies entirely on having many threads to choose from (idea #3), rather than on the compiler finding independent operations inside one thread. AMD's data-centre GPUs still use this GCN compute unit today, extended with matrix units, and it is the compute unit that Section 14.2 compares with NVIDIA's SM.
- **Two branches (2019–2020).** AMD then split its line in two. **RDNA**, for graphics cards, changed the compute unit to run 32-thread wavefronts natively; it can still run 64-thread wavefronts, as two halves [3]. **CDNA**, for data centres, kept GCN's 64-thread wavefronts and added matrix units for AI [4]. The CDNA line runs from the MI100 (2020) through the MI200 (2021), MI300 (2023) and MI350 (2025) to the MI400, launched in 2026 [14, 35].

This section compares two recent GPUs from each company. From AMD, it uses the MI300X, a member of the MI300 series (CDNA 3), and the MI355X, a member of the MI350 series (CDNA 4). From NVIDIA, it uses the H100 of the Hopper generation and the B200 of the Blackwell generation.

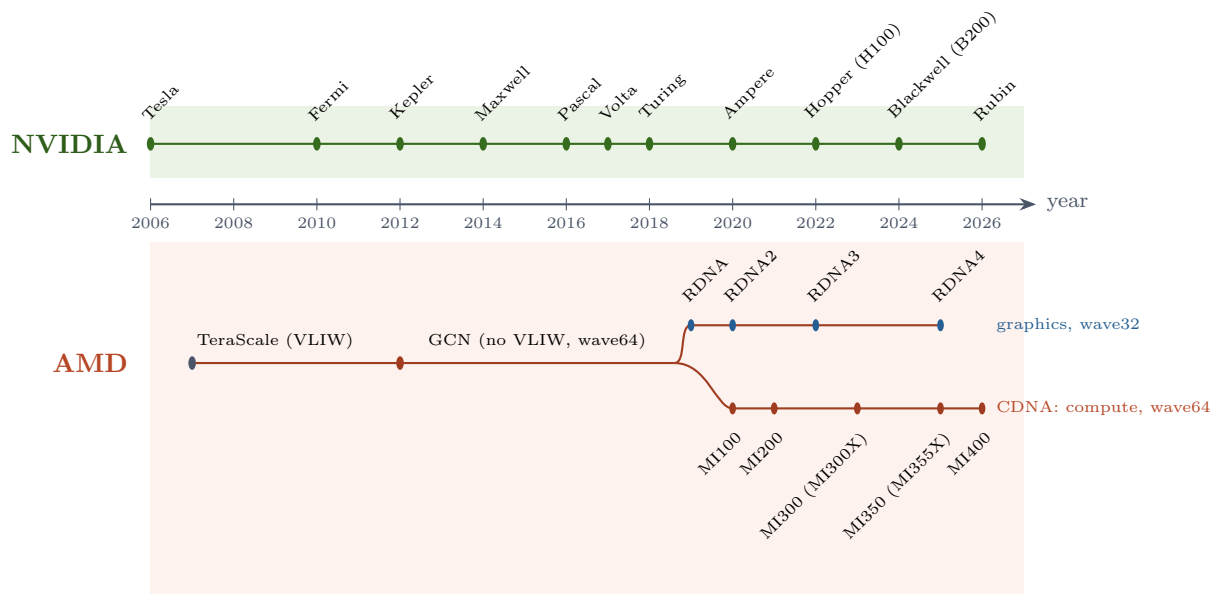


Figure 33: Architectural lineages. NVIDIA has refined one SM-based line since 2006, building separate graphics and data-centre chips on it since Pascal. AMD dropped the VLIW instructions of TeraScale with GCN in 2012 and in 2019–2020 split into RDNA for graphics and CDNA (the Instinct MI series) for data-centre computing. The GPUs compared in this section are named in brackets.

14.2 The core: SM versus compute unit

We first describe the core of each company’s GPU on its own: NVIDIA’s SM, which Section 10 already took apart, and then AMD’s compute unit. Only after that do we compare the two. Figure 34 draws both cores at the same level of detail, and Table 11 lists their resources side by side.

NVIDIA: the streaming multiprocessor.

As Section 10.3 showed for the H100, an SM consists of four processing blocks.

- **Threads.** The SM runs warps of 32 threads, and it can hold up to 64 warps, that is 2,048 threads, at the same time: 16 warps in each processing block.
- **Arithmetic.** Each processing block has 32 FP32 units, one for each thread of a warp. Its warp scheduler can therefore start one warp instruction every cycle, and the instruction runs for all 32 threads at once. These units carry out only FP32 operations; FP64 and integer operations have their own, separate units.
- **Registers.** The SM has 65,536 registers of 32 bits, 256 KB in all, a quarter in each processing block.
- **Shared memory.** A 256 KB block of on-chip memory serves both as the L1 data cache and as shared memory; up to 228 KB of it can be used as shared memory.
- **Other units.** Each processing block also has a tensor core for matrix arithmetic (Section 13) and a uniform datapath for values that are the same for all threads of a warp (Section 10.5) [18].

AMD: the compute unit.

An AMD CDNA compute unit (CU) also consists of four parts, called **SIMD units**, the counterparts of NVIDIA's processing blocks [12].

- **Threads.** The CU runs wavefronts of 64 threads, and it can hold up to 32 wavefronts, that is again 2,048 threads, at the same time: 8 wavefronts in each SIMD unit [10, 39].
- **Arithmetic.** Each SIMD unit has only 16 ALUs, a quarter of the 64 threads of a wavefront. One instruction of a wavefront is therefore carried out in four cycles: threads 0–15 in the first cycle, threads 16–31 in the second, threads 32–47 in the third and threads 48–63 in the fourth. A SIMD unit can thus start a new wavefront instruction only every fourth cycle.

The CU keeps its four SIMD units busy by giving them turns, one SIMD unit per cycle [12, 13, 34]. In each cycle, the CU's scheduler takes three steps:

1. it picks the next SIMD unit in round-robin order: SIMD0, SIMD1, SIMD2, SIMD3, then SIMD0 again;
2. among the wavefronts held by that SIMD unit (up to 8), it chooses one whose next instruction is ready, that is, one whose input values are already available;
3. it issues that instruction to the SIMD unit.

With four SIMD units taking turns, each one is picked every fourth cycle, and four cycles is exactly the time a SIMD unit needs for one wavefront instruction. Each SIMD unit therefore receives its next instruction just as it finishes the previous one. The next instruction may come from the same wavefront as before or from a different one. If none of the unit's wavefronts is ready, the SIMD unit stays idle until its next turn, four cycles later.

Unlike NVIDIA's FP32 units, the ALUs are not limited to FP32: the same vector instructions also carry out FP64 and integer arithmetic [7], and FP64 runs at the full rate of 128 operations per cycle per CU [6]. CDNA 2 added *packed* FP32 instructions, in which one instruction carries out two FP32 operations for each thread, on two pairs of values held in neighbouring registers. AMD states that these instructions double the vector FP32 rate of a CU [5], to 256 operations per cycle on CDNA 3 [6].

- **Registers.** AMD calls the registers of a SIMD unit **vector registers**, or VGPRs (vector general-purpose registers). The name comes from what one register holds.

A wavefront instruction names its registers once, for all 64 threads. For example, the instruction `v_add_f32 v2, v0, v1` adds register `v0` to register `v1` and writes the sum to register `v2`, in each of the 64 threads of the wavefront. Each thread has its own values, so one register such as `v0` holds 64 values of 32 bits, one for each thread: the value of thread 0 in lane 0, the value of thread 1 in lane 1, and so on up to lane 63. A register that holds one value for each thread of a wavefront in this way is a vector register. (The scalar registers of Section 14.4 are different: one scalar register holds a single value for the whole wavefront.)

Each SIMD unit has its own register file of 512 vector registers [10]. One vector register takes $64 \times 4 = 256$ bytes, so the register file of one SIMD unit holds 512×256 bytes = 128 KB, and the four SIMD units of a CU hold 512 KB.

The 512 vector registers of a SIMD unit are shared by all the wavefronts it holds, and they are handed out one wavefront at a time. The steps are as follows.

1. *The compiler counts.* When the compiler compiles a kernel, it counts how many vector registers the kernel’s code uses, and records this number in the kernel’s metadata. Call it v . Every wavefront that runs the kernel needs v vector registers; because one vector register already holds a value for each of the 64 threads, these v registers serve the whole wavefront.
2. *The hardware rounds up.* The hardware hands out vector registers in groups of 8 [7]. It therefore rounds v up to the next multiple of 8. A kernel that uses 100 vector registers, for example, receives 104 per wavefront.
3. *Each wavefront gets its own registers.* When a wavefront of the kernel starts on a SIMD unit, it receives that rounded number of the unit’s 512 vector registers as its own.

How many wavefronts a SIMD unit can hold therefore depends on v (after rounding): it is 512 divided by v , rounded down to a whole number [10]. A second limit applies as well: a SIMD unit has room to track at most 8 wavefronts (the **Threads** bullet above), however few registers each one needs. Some examples:

- $v = 40$: the registers would allow $512/40 = 12.8$, that is 12 wavefronts, but the limit of 8 applies, so the SIMD unit holds 8;
- $v = 64$: $512/64 = 8$ wavefronts, exactly the limit;
- $v = 104$ (the kernel above): $512/104 = 4.9$, rounded down to 4 wavefronts; the remaining $512 - 4 \times 104 = 96$ registers are too few for a fifth wavefront and stay unused;
- $v = 128$: $512/128 = 4$ wavefronts;
- $v = 256$: $512/256 = 2$ wavefronts;
- $v = 512$: 1 wavefront.

The more vector registers a kernel uses, the fewer wavefronts a SIMD unit holds, and the fewer wavefronts it can switch between to hide memory delays. This is the same trade-off as on NVIDIA GPUs (Section 12.1), counted here in vector registers per wavefront.

The largest possible value, $v = 512$, gives one wavefront the whole register file of its SIMD unit, and that wavefront is then the only one on the unit. On CDNA 3 and CDNA 4, the 512 vector registers of a wavefront are of two kinds [7, 10]:

- up to 256 ordinary vector registers, used by all vector instructions;
- up to 256 *accumulation* registers, used by the matrix instructions of Section 13, which add many products into a running total, the accumulator. A matrix instruction can keep this total in accumulation registers or in ordinary vector registers.

Both kinds count towards v .

- **Shared memory.** The CU’s shared memory, the local data share (LDS), holds 64 KB on CDNA 3 and 160 KB on CDNA 4 [6, 9]. The L1 data cache is a separate 32 KB memory.
- **Other units.** Each SIMD unit has its own matrix core for matrix arithmetic, four per CU [6, 76], and the CU has a scalar unit with its own scalar registers for values that are the same for all threads of a wavefront (Section 14.4).

(AMD’s graphics GPUs, RDNA, use a different compute unit with 32-thread wavefronts; this section concerns only the data-centre CDNA design.)

Comparing the two cores.

Put side by side, the two cores differ in four ways.

- **Width of a thread group.** An AMD wavefront has twice as many threads as an NVIDIA warp. A wider group shares the cost of fetching and decoding each instruction among more threads, which is idea #2 of Section 8 pushed further. But it suffers more from divergence (Section 11.3): if any one of 64 threads takes a different path at an `if`, the whole wavefront must run both paths.
- **Register budget at full occupancy.** An SM and a CU can each hold at most the same number of threads, 2,048: 64 warps of 32 threads in an SM, and 32 wavefronts of 64 threads in a CU. A core reaches full occupancy (Section 12.1) when it holds all 2,048 of these threads. The number of registers each thread can then use is its *register budget*; a kernel whose threads need more registers than this budget cannot reach full occupancy. In both cores the registers are divided among the four parts, so we compute the budget for one part: a processing block on NVIDIA and a SIMD unit on AMD. To compare the two, we count in the same unit on both sides: one 32-bit value of one thread.
 - *NVIDIA processing block.* Its register file holds 16,384 registers (64 KB), and each register is one 32-bit value of one thread. At full occupancy the processing block holds 16 warps, that is $16 \times 32 = 512$ threads. Each thread's budget is therefore $16,384/512 = 32$ values.
 - *AMD SIMD unit.* Its register file holds 512 vector registers (128 KB), and each vector register holds 64 values, one for each thread of a wavefront: $512 \times 64 = 32,768$ values in all. At full occupancy the SIMD unit holds 8 wavefronts, that is $8 \times 64 = 512$ threads. Each thread's budget is therefore $32,768/512 = 64$ values. In AMD's own unit, each wavefront gets $512/8 = 64$ vector registers.

The two parts hold the same number of threads, 512, but the SIMD unit has twice as much register storage, 128 KB against 64 KB. At full occupancy an AMD thread can therefore use twice as many values as an NVIDIA thread: 64 against 32. The same holds for whole cores: 256 KB for 2,048 threads in an SM, 512 KB for 2,048 threads in a CU.

- **Shared memory.** NVIDIA offers more shared memory per core: up to 228 KB per SM, against 64 KB (CDNA 3) or 160 KB (CDNA 4) per CU.
- **FP32 arithmetic per core.** This point compares how much FP32 arithmetic one core can carry out in one clock cycle. As in Section 2, a fused multiply-add, $a \times b + c$, counts as two floating-point operations.
 - *NVIDIA SM.* The SM has $4 \times 32 = 128$ FP32 units, and each completes one fused multiply-add per cycle. The SM therefore carries out $128 \times 2 = 256$ FP32 operations per cycle [61].
 - *AMD CU.* The CU has $4 \times 16 = 64$ ALUs, half as many. With ordinary instructions, each ALU completes one fused multiply-add per cycle, so the CU carries out $64 \times 2 = 128$ FP32 operations per cycle. With packed FP32 instructions, which carry out two FP32 operations for each thread, this rate doubles to $64 \times 2 \times 2 = 256$ operations per cycle [5, 6].

So although the SM has twice as many arithmetic units, both cores reach the same peak of 256 FP32 operations per cycle. The CU reaches this peak only when the compiler can use

packed instructions, that is, when each thread has two independent FP32 operations of the same kind, for example two multiply-adds, that can be carried out together.

Because one NVIDIA core and one AMD core differ this much inside, the number of cores says little about how two chips compare: an H100 has 132 SMs and an MI300X 304 CUs; a B200 has 148 SMs and an MI355X 256 CUs.

Table 11: Core-level resources: NVIDIA Hopper/Blackwell SM versus AMD CDNA 3/CDNA 4 CU [5–7, 9, 10, 12, 27, 61, 76].

	NVIDIA SM	AMD CDNA CU
Threads that execute together	warp, 32 threads	wavefront, 64 threads
Arithmetic units	4 processing blocks $\times$ 32 FP32 units	4 SIMD units $\times$ 16 ALUs
Instruction rate	1 warp instruction per processing block per cycle	1 wavefront instruction per SIMD unit every 4 cycles; the SIMD units take turns, one per cycle
Peak FP32 operations per cycle	256 (128 units $\times$ 2 for FMA)	256 (64 ALUs $\times$ 2 for FMA $\times$ 2 for packed)
Register file	256 KB: 65,536 registers, each one 32-bit value of one thread	512 KB: 4 $\times$ 512 vector registers, each 64 values of 32 bits, one per thread of a wavefront
Most 32-bit values one thread can use	255	512
Most registers one warp / wavefront can use	8,192 registers (32 $\times$ 256: each thread’s 255 rounded up to 256)	512 vector registers (up to 256 ordinary + up to 256 accumulation)
Max threads held at once	2,048 (64 warps)	2,048 (32 wavefronts, 8 per SIMD unit)
Values shared by the whole group	uniform datapath (since 2018)	scalar unit and scalar registers (since 2012)
Memory shared by a thread block / work-group	shared memory, up to 228 KB	LDS, 64 KB (CDNA 3), 160 KB (CDNA 4)
L1 data cache	unified with shared memory (256 KB total)	separate 32 KB vector L1
Matrix units	4 tensor cores; 256 KB TMEM (Blackwell)	4 matrix cores (one per SIMD unit)
Chip totals	132 SMs (H100), 148 SMs (B200)	304 CUs (MI300X), 256 CUs (MI355X)

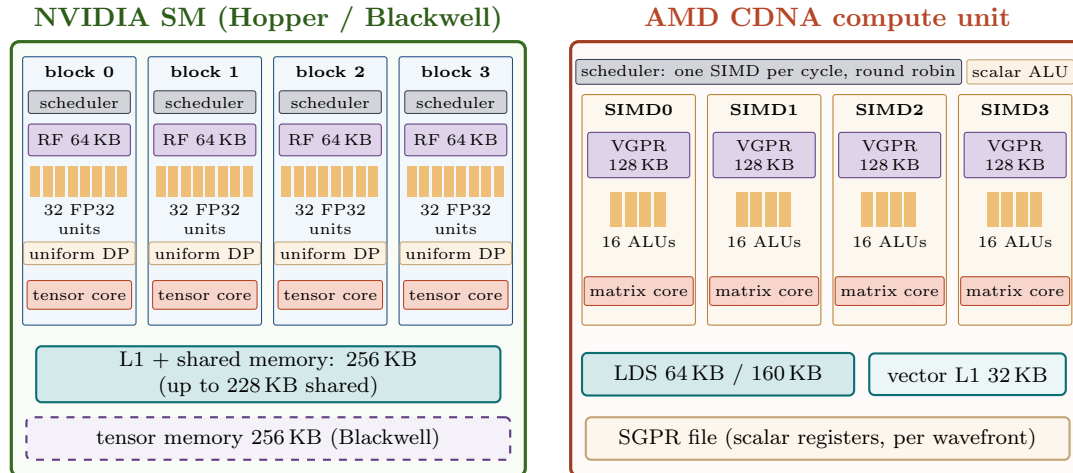


Figure 34: An NVIDIA SM (left) and an AMD CDNA compute unit (right), drawn at the same level of detail. Both cores consist of four parts. The SM consists of four processing blocks. Each processing block has a warp scheduler, a 64 KB register file, 32 FP32 units, a uniform datapath and a tensor core, and starts one warp instruction, for 32 threads, every cycle. The CU consists of four SIMD units. Each SIMD unit has a 128 KB vector register file, 16 ALUs and a matrix core, and needs four cycles for one wavefront instruction, for 64 threads; the CU’s scheduler gives the four SIMD units a turn one after the other, one per cycle, and the scalar unit is shared by the whole CU. Below the four parts, the SM has 256 KB of on-chip memory that serves as both L1 cache and shared memory (up to 228 KB of it as shared memory), and on Blackwell a separate 256 KB tensor memory. The CU has a separate LDS (64 KB on CDNA 3, 160 KB on CDNA 4), a separate 32 KB vector L1 cache, and a file of scalar registers used by the scalar unit (Section 14.4).

14.3 Handling divergence

Section 11.3 explained what happens when the threads of a group disagree at an `if`: the group cannot run both paths at the same moment, so while the threads of one path run, the threads of the other path are switched off. NVIDIA and AMD GPUs both work this way, but they divide the job differently between hardware and compiler. The description below is based on the companies’ public documentation and, for AMD, also on its open-source compiler. To make it concrete, we compile one small kernel for both GPUs and read the machine code that comes out.

The example kernel.

Listing 9 shows the kernel. Each thread reads one number `x`. Odd-numbered threads add a number from array `b`, and even-numbered threads divide by a number from array `c`. Odd and even threads alternate, so in every warp and every wavefront half of the threads take the `if` path and the other half take the `else` path: every group diverges.

Listing 9: The example kernel. The same source compiles as CUDA and as HIP.

```
1  __global__ void divide(int *a, const int *b, const int *c) {
2      int tid = threadIdx.x;
3      int x = a[tid];
4      if (tid % 2 != 0)        // odd threads
5          x = x + b[tid];
6      else                    // even threads
7          x = x / c[tid];
8      a[tid] = x;
9  }
```

We compiled the kernel with Clang 18.1 for the AMD MI300X (gfx942). For NVIDIA, we generated PTX with Clang 18.1 and translated it with NVIDIA’s `ptxas` 12.9 into machine code for the H100 (sm_90). In both cases `threadIdx.x` was replaced by the compiler’s equivalent built-in function. The listings below are abridged: we removed address arithmetic and waiting instructions and replaced the body of the division by a comment (...); every line that remains is exactly as the tools produced it. The comment line above each instruction, which we added, says what the instruction does and what its result is.

NVIDIA: predicates, a per-thread branch, and a convergence barrier.

Listing 10 shows the H100 machine code. The number at the start of each line is the instruction’s address.

Listing 10: The example kernel compiled for the NVIDIA H100 (abridged).

```

1 // Read this thread's index, tid, into R9.
2 /*0010*/      S2R R9, SR_TID.X ;
3 // R0 = tid AND 1: 1 in odd threads, 0 in even threads.
4 /*0040*/      LOP3.LUT R0, R9.reuse, 0x1, RZ, 0xc0, !PT ;
5 // P0 = (R0 != 1): true in even threads, false in odd threads.
6 /*0070*/      ISETP.NE.U32.AND P0, PT, R0, 0x1, PT ;
7 // All threads: load x = a[tid] into R0 (R0 is reused).
8 /*0080*/      LDG.E R0, desc[UR4][R2.64] ;
9 // Odd threads only (P0 false): load b[tid] into R5.
10 /*00c0*/     @!P0 LDG.E R5, desc[UR4][R4.64] ;
11 // Set up convergence barrier B0; after BSYNC, threads continue at 0x2d0.
12 /*00d0*/     BSSY B0, 0x2d0 ;
13 // Odd threads only: R7 = R0 + R5, that is, x + b[tid].
14 /*00e0*/     @!P0 IMAD.IADD R7, R0, 0x1, R5 ;
15 // Odd threads only: jump to BSYNC at 0x2c0. Even threads go on below.
16 /*00f0*/     @!P0 BRA 0x2c0 ;
17 // Even threads: load c[tid] into R5.
18 /*0130*/     LDG.E R5, desc[UR4][R4.64] ;
19 // Even threads: R7 = x / c[tid] (24 instructions, omitted).
20 // ...
21 // Wait here until all threads of barrier B0 have arrived.
22 /*02c0*/     BSYNC B0 ;
23 // All 32 threads: store R7 into a[tid].
24 /*02d0*/     STG.E desc[UR4][R2.64], R7 ;
25 // End of the kernel.
26 /*02e0*/     EXIT ;

```

The code works in four steps.

1. *Each thread computes its own condition.* The instructions at 0010–0070 compute `tid & 1` and compare it with 1. The result goes into `P0`, a one-bit *predicate register* that every thread has for itself: `P0` is true in the even threads and false in the odd threads.
2. *The short if path is predicated.* An instruction written `@!P0` is carried out only by the threads in which `P0` is false [65]. The load of `b[tid]` (00c0) and the addition (00e0) are therefore executed by the odd threads only. The warp issues these instructions once; the even threads simply do nothing during them. No branch is needed for this path.
3. *The long else path is skipped by a branch.* Before the branch, `BSSY` (“barrier set convergence synchronization point” [49]) sets up a convergence barrier `B0` for the threads of the warp; its

operand, 0x2d0, is the address just after the matching BSYNC. The branch @!P0 BRA 0x2c0 is then taken only by the odd threads, which jump straight to the BSYNC at 02c0. The even threads do not take the branch and continue with the division at 0100–02b0. At this moment the threads of one warp are at two different places in the program, which is possible because each thread has its own program counter [59].

4. *The threads meet again.* BSYNC (“synchronize threads on a convergence barrier” [49]) makes the threads of B0 wait for one another. The odd threads, which jumped straight there, wait at the barrier until the even threads have finished the division and arrived too. From the store at 02d0 on, all 32 threads of the warp run together again.

In this code the compiler decides *which* threads run which instructions (with P0) and *where* the threads meet again (with BSSY and BSYNC). Keeping track of where each thread is, and switching threads off and on while the warp runs the division, is done by the hardware. We are not aware of public NVIDIA documentation that describes in detail how the hardware schedules the two groups of threads. NVIDIA does state that a program must not assume that the threads of a warp run in lock-step, and that it can force them to run together again by calling `--syncwarp()` [46, 59].

AMD: the compiler switches threads off and on with EXEC.

On AMD GPUs, a wavefront has a single program counter, and all branches are scalar instructions that move the whole wavefront [7]. A branch therefore cannot send the odd threads to one place and the even threads to another. Instead, the wavefront goes through *both* paths, and a 64-bit register called EXEC, the execution mask, decides which threads take part: a vector instruction is carried out only by the threads whose bit in EXEC is 1 [7]. Listing 11 shows the MI300X machine code; the comments, which follow the definitions of the instructions in AMD’s instruction-set manual [7] and the comments of the LLVM developers on this sequence [41], say what each instruction achieves. Note that a jump such as `s_cbranch_execz` moves the whole wavefront. It is taken only when no thread at all is on, so that the wavefront can skip a path that none of its threads needs; in our example both paths have threads, so neither jump is taken and the wavefront runs both paths, one after the other.

Listing 11: The example kernel compiled for the AMD MI300X (abridged).

```

1 ; ---- Before the if ----
2 ; Mark each thread as odd (1) or even (0): tid AND 1.
3   v_and_b32_e32    v0, 1, v0
4 ; Every thread loads its own x = a[tid].
5   global_load_dword v4, v2, s[4:5]
6 ; Every thread evaluates the condition "tid is odd";
7 ; VCC records the answer, one bit per thread.
8   v_cmp_eq_u32_e32 vcc, 1, v0
9
10 ; ---- Enter the if path ----
11 ; Save the current mask, and switch on only the odd threads.
12   s_and_saveexec_b64 s[0:1], vcc
13 ; Remember the even threads, for the else path.
14   s_xor_b64        s[0:1], exec, s[0:1]
15 ; If no thread takes the if path, skip it.
16   s_cbranch_execz   .LBB0_2
17
18 ; ---- The if path: odd threads only ----
19 ; Load b[tid].

```

```

20  global_load_dword  v0, v2, s[6:7]
21  ; x + b[tid].
22  v_add_u32_e32      v5, v0, v4
23
24  .LBB0_2:
25  ; ---- Enter the else path ----
26  ; Switch all threads back on, and remember the odd threads.
27  s_or_saveexec_b64  s[0:1], s[0:1]
28  ; Switch off the odd threads: only the even threads stay on.
29  s_xor_b64          exec, exec, s[0:1]
30  ; If no thread takes the else path, skip it.
31  s_cbranch_execz    .LBB0_4
32
33  ; ---- The else path: even threads only ----
34  ; Load c[tid].
35  global_load_dword  v2, v2, s[2:3]
36  ; x / c[tid] (28 instructions, omitted).
37  ...
38
39  .LBB0_4:
40  ; ---- Leave the if-else ----
41  ; Switch the odd threads back on: all threads are on again.
42  s_or_b64            exec, exec, s[0:1]
43  ; Every thread stores its result into a[tid].
44  global_store_dword v[0:1], v5, off
45  ; End of the kernel.
46  s_endpgm

```

So on AMD GPUs, every step of divergence is an ordinary instruction that the compiler chose: the comparison, the saving, changing and restoring of EXEC, and the optional jumps over a path whose mask is empty. In this example both masks are non-empty, so the wavefront runs both paths, one after the other, with half of its threads switched off each time. The hardware only obeys EXEC. This method needs the program’s control flow in a nested form, in which every path that splits off joins again at a single point; LLVM rearranges the divergent parts of a program into this form before it emits the mask instructions [40, 68]. For complex control flow that cannot easily be put into this form, AMD’s instruction set also offers special instructions, S_CBRANCH_G_FORK, S_CBRANCH_I_FORK and S_CBRANCH_JOIN, which keep a small stack of saved masks and program addresses in scalar registers; the manual describes them as slower than the simple method above and to be used only when necessary [7].

Practical consequences.

- **Lock-step assumptions.** AMD’s documentation guarantees that the threads of a wavefront run in lock-step [11]. NVIDIA has given no such guarantee since 2017: code that assumes the threads of a warp run in lock-step is invalid there and must call `__syncwarp()` [46] (Section 11.3). HIP provides `__syncwarp()` as well [11], so portable code should synchronise explicitly on both.
- **What the compiler controls.** On AMD, every line of Listing 11 that handles divergence is an instruction the compiler chose and can optimise. On NVIDIA, the compiler chooses the predicates, the branch and the convergence barrier (Listing 10), and the hardware runs the two groups of threads.

14.4 Computing a value once for the whole group

Section 10.5 introduced values that are the same for every thread of a warp, and NVIDIA’s uniform datapath, which computes such a value once for the whole warp. Consider this loop, in which each thread adds up numbers from one column of a table:

```
for (int i = 0; i < n; i++)
    sum += a[i * width + tid];
```

Only `tid`, and therefore `sum`, differ from thread to thread. The loop counter `i`, the limit `n`, the row offset `i * width` and the decision whether to run the loop once more are the same in all threads. Both companies give each core a unit that holds such a value once for the whole warp or wavefront and computes it once.

- **AMD: the scalar unit.** Every compute unit has had one since GCN in 2012 [1]. AMD’s manual describes it as an ALU “which operates on one value per wavefront (common to all work items)”, with its own scalar registers [7]. The scalar unit also carries out all control flow, including the changes to EXEC described in Section 14.3.
- **NVIDIA: the uniform datapath** (Section 10.5), added in 2018, is integer-only and works alongside the main datapath [29].

In the loop above, values such as `i`, `n` and `i * width` can live in the scalar or uniform registers, while `sum` and the final address, which depend on `tid`, need one copy per thread.

On both GPUs, the compiler decides which values go into these registers, as Section 10.5 describes for NVIDIA.

14.5 Matrix engines

Section 13 explained why AI needs units that multiply small matrices much faster than the ordinary arithmetic units can, and introduced NVIDIA’s tensor cores. AMD’s counterparts are called matrix cores. Both compute $D = A \times B + C$ on small blocks of numbers. They differ in how a program starts such an operation, which threads take part, and where the matrices are kept while the unit works on them.

NVIDIA: tensor cores in the H100 and B200.

Each SM of the H100 and of the B200 has four tensor cores, one in each processing block (Figure 34, Table 11) [27, 61]. The two generations differ in how a program starts a matrix multiplication and where the matrices are kept while the tensor cores work on them.

- *H100 (Hopper).* A *warpgroup* of four warps, 128 threads, works together on one matrix product, with the instruction `wgmma.mma_async`. The operation is asynchronous: the threads can carry on with other instructions while it runs. Matrix *A* comes from the threads’ registers or from shared memory, matrix *B* from shared memory, and the result *D* is kept in the threads’ registers [56].
- *B200 (Blackwell).* A single thread starts the operation, with the instruction `tcgen05.mma`, on behalf of its whole thread block (or of a pair of blocks). Matrix *A* comes from shared memory or from the 256 KB *tensor memory* that each SM has, matrix *B* from shared memory, and the result *D* is written to the tensor memory [27, 55].

From the H100 to the B200, starting a matrix operation has thus moved from a group of 128 threads to a single thread, and the result has moved from the threads’ registers to a separate memory next to the tensor cores.

AMD: matrix cores in the MI300X and MI355X.

Each compute unit of the MI300X and of the MI355X has four matrix cores, one in each SIMD unit (Figure 34, Table 11) [6, 76]. A program uses them through MFMA (matrix fused multiply-add) instructions. These are ordinary wavefront instructions: all 64 threads of a wavefront together carry out one MFMA operation [8]. The name of an instruction states its shape. For example, on the MI300X, `v_MFMA.F32.32X32X1.2B.F32` computes two products $D = C + A \times B$ at once, in which D and C are 32×32 matrices of 32-bit floating-point numbers, A is 32×1 and B is 1×32 [7]. The matrices are kept in the wavefront’s vector registers. For each matrix, the instruction itself states whether it is held in the ordinary vector registers or in the accumulation registers of Section 14.2: one bit in the instruction makes this choice for A , one for B , and one for C and D together [7]. Because the result takes several cycles to appear, the instruction stream must sometimes contain a number of independent instructions between an MFMA instruction and the first instruction that uses its result [7]. AMD describes the MFMA instructions of the MI355X (CDNA 4) in the same way, as wavefront-level instructions on vector registers [8].

So on the MI300X and MI355X a matrix operation is still started by a whole wavefront, and its matrices stay in the wavefront’s registers.

Small number formats.

The newer GPUs also multiply matrices of very small floating-point numbers, which AI models increasingly use [69]. The H100 supports 8-bit numbers (FP8) [61], and the B200 adds 6- and 4-bit numbers (FP6 and FP4) [51]. The MI300X supports FP8, and the MI355X adds the 6- and 4-bit formats MXFP6 and MXFP4 [9].

Takeaway: same ideas, different bets

Both companies build GPUs from the same three ideas of Section 8: many slim cores, one instruction driving many ALUs, and many threads kept ready to hide memory delays. They implement them differently. NVIDIA uses groups of 32 threads, handles divergence mostly in hardware, offers more shared memory per core, and has moved its tensor cores away from the individual warp. AMD uses groups of 64 threads, handles divergence with masks that the compiler controls, has had a scalar unit since 2012, gives each core a larger register file, and keeps its matrix instructions as ordinary wavefront instructions. Code that respects the three ideas runs well on both; code that hard-wires one company’s group size or lock-step behaviour does not.

15 Conclusion

This paper has followed the GPU from a chip that only drew images to a general-purpose parallel processor. Drawing a 3D scene means running the same small calculation on millions of vertices and pixels, each independently of the others (Section 3). Graphics hardware first built each stage of this work as a separate fixed-function circuit, which could be adjusted but not programmed, until in 1999 all the stages fitted on one chip, the first GPU (Section 4). When two of its stages became programmable, the GPU became a parallel computer that understood only graphics (Section 5); researchers used it for other calculations by disguising them as drawing (Section 6), and unified cores and CUDA finally removed the disguise (Section 7). Figure 35 places these milestones, and those of the later sections, on one time line.

Behind every GPU of this kind stand three design ideas (Section 8): many slim cores, one instruction driving many ALUs, and many threads kept ready so that memory delays can be

hidden by switching between them. These ideas explain why a GPU is a throughput machine rather than a latency machine like a CPU, why a GPU program is written for one thread and launched on a grid of blocks of threads (Section 9), why the hardware runs threads in warps of 32 (Section 10), and why threads that disagree at an if cost time (Section 11). They also explain the rules for fast GPU code (Section 12): keep enough warps on each SM, let the threads of a warp access neighbouring addresses, avoid shared-memory bank conflicts, keep the threads of a warp on the same path, and give the GPU enough work to fill it. The reduction example showed these rules at work, the same algorithm being fitted to the hardware step by step.

The demands of AI have led GPU designers to add new hardware around the general-purpose cores (Section 13): tensor cores for matrix arithmetic, hardware that moves data to them, smaller number formats, and GPUs built from more than one die. Finally, the comparison of NVIDIA and AMD GPUs (Section 14) showed the same three ideas implemented in different ways: groups of 32 threads against 64, divergence handled mostly by the hardware against masks that the compiler controls, a uniform datapath against a scalar unit, and tensor cores that have moved away from the individual warp against matrix instructions that remain ordinary wavefront instructions. Reading the machine code that both compilers produce for the same small kernel made these differences concrete.

The details of GPUs change with every generation, but the three design ideas, and the reasoning that connects them to performance, have stayed the same since the first unified GPUs. Asking how a piece of code uses many cores, how it keeps the threads of a group on the same path and the same memory region, and whether it gives the hardware enough threads to hide memory delays remains the most useful way to understand both a GPU and the programs that run on it.

We have tried to give a precise and connected introduction to how GPUs evolved and how they work, and we have checked the facts against public sources wherever we could. The breadth of material the paper covers, from graphics hardware of the 1980s to the machine code of current GPUs, makes it possible that some errors have crept in despite this care. Any such errors are ours alone, and we will be grateful to readers who point them out.

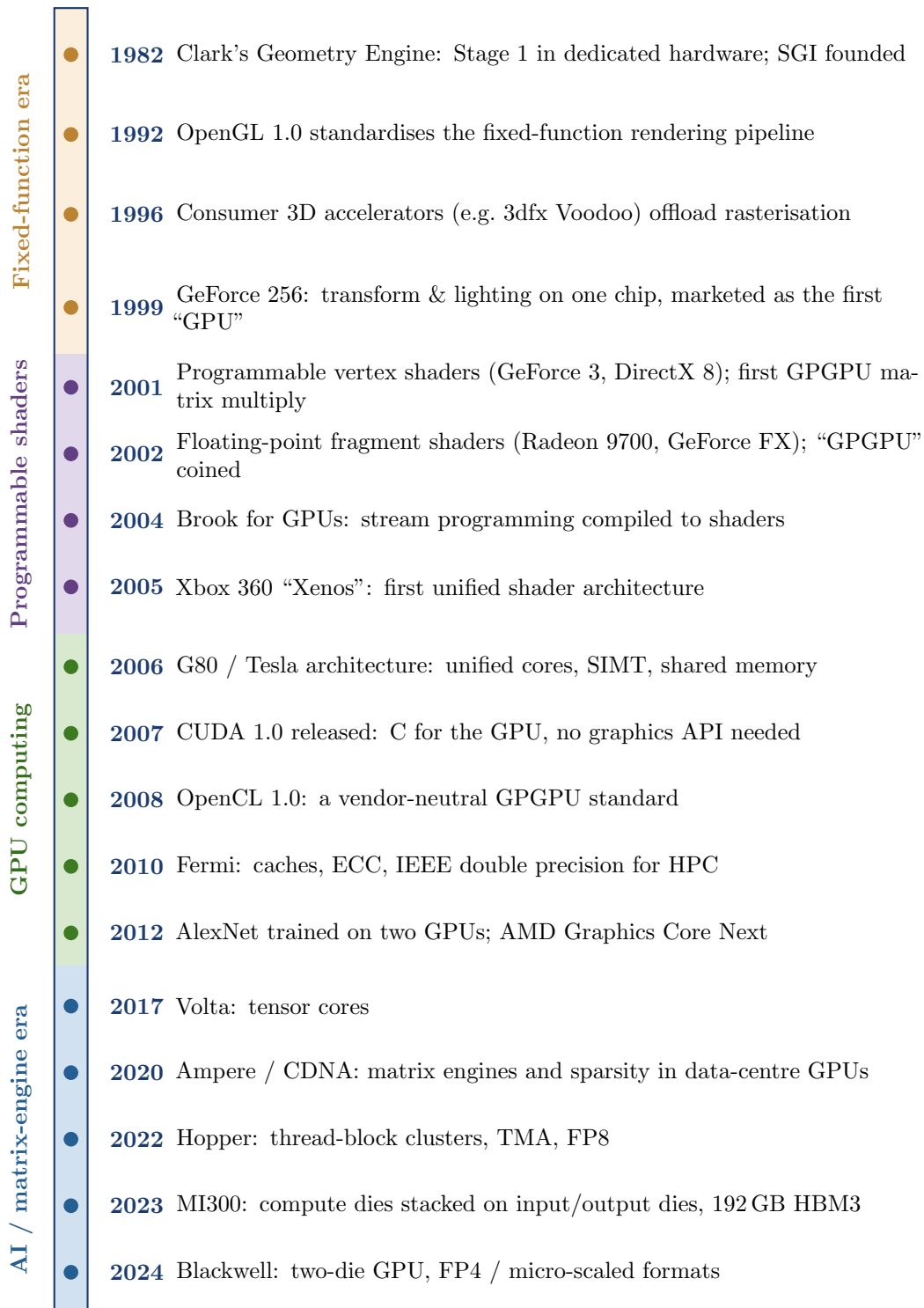


Figure 35: A time line of GPU history. Shading on the spine marks four eras: fixed-function graphics hardware, programmable shaders (the period of “GPGPU through graphics APIs”), general-purpose GPU computing with dedicated languages, and the current era of matrix engines, low-precision arithmetic and multi-die GPUs.

References

- [1] Advanced Micro Devices. AMD graphics cores next (GCN) architecture. White paper, 2012.
- [2] Advanced Micro Devices. AMD accelerated parallel processing OpenCL programming guide, rev. 2.3, 2013.
- [3] Advanced Micro Devices. Introducing RDNA architecture. White paper, 2019.
- [4] Advanced Micro Devices. Introducing AMD CDNA architecture. White paper, 2020.
- [5] Advanced Micro Devices. AMD CDNA 2 architecture. White paper, <https://www.amd.com/content/dam/amd/en/documents/instinct-business-docs/white-papers/amd-cdna2-white-paper.pdf>, 2021.
- [6] Advanced Micro Devices. AMD CDNA 3 architecture. White paper, 2023.
- [7] Advanced Micro Devices. AMD Instinct MI300 CDNA3 instruction set architecture reference guide. <https://www.amd.com/content/dam/amd/en/documents/instinct-tech-docs/instruction-set-architectures/amd-instinct-mi300-cdna3-instruction-set-architecture.pdf>, 2024.
- [8] Advanced Micro Devices. Matrix core programming on AMD CDNA3 and CDNA4 architecture. ROCm Blogs, <https://rocm.blogs.amd.com/software-tools-optimization/matrix-cores-cdna/README.html>, 2025.
- [9] Advanced Micro Devices. AMD Instinct MI350 series microarchitecture. ROCm documentation, <https://rocm.docs.amd.com/en/latest/reference/gpu-arch/mi350.html>, 2025.
- [10] Advanced Micro Devices. Occupancy math on the AMD MI355X GPU (CDNA4): A from-first-principles guide. ROCm blogs, 2025.
- [11] Advanced Micro Devices. HIP C++ language extensions. https://rocm.docs.amd.com/projects/HIP/en/latest/how-to/hip-cpp-language_extensions.html, 2025.
- [12] Advanced Micro Devices. HIP documentation: Hardware implementation. <https://rocm.docs.amd.com/projects/HIP/>, 2025.
- [13] Advanced Micro Devices. Pipeline descriptions. ROCm Compute Profiler documentation, <https://rocm.docs.amd.com/projects/rocmprofiler-compute/en/docs-6.4.1/conceptual/pipeline-descriptions.html>, 2025.
- [14] Advanced Micro Devices. AMD delivers full-stack compute for the agentic AI era. Press release, <https://ir.amd.com/news-events/press-releases/detail/1294/aai-2026-amd-delivers-full-stack-compute-for-the-agentic-ai-era>, 2026.
- [15] Apple Inc. Metal programming guide. Apple Developer Documentation, 2014.
- [16] Jeff Bolz, Ian Farmer, Eitan Grinspun, and Peter Schröder. Sparse matrix solvers on the GPU: Conjugate gradients and multigrid. *ACM Transactions on Graphics*, 22(3):917–924, 2003.
- [17] Ian Buck, Tim Foley, Daniel Horn, Jeremy Sugerman, Kayvon Fatahalian, Mike Houston, and Pat Hanrahan. Brook for GPUs: Stream computing on graphics hardware. *ACM Transactions on Graphics*, 23(3):777–786, 2004.

- [18] John Burgess. RTX ON – the NVIDIA Turing GPU. Hot Chips 31, https://old.hotchips.org/hc31/Hc31_2.12_NVIDIA_final.pdf, 2019.
- [19] Chips and Cheese. Testing AMD’s giant MI300X. <https://chipsandcheese.com/p/testing-amds-giant-mi300x>, 2024.
- [20] Chips and Cheese. Nvidia’s B200: Keeping the CUDA juggernaut rolling. <https://chipsandcheese.com/p/nvidias-b200-keeping-the-cuda-juggernaut>, 2025.
- [21] Alpin Dale. Characterizing warp divergence from Pascal to Blackwell. arXiv:2607.23402, 2026.
- [22] Kayvon Fatahalian and Mike Houston. A closer look at GPUs. *Communications of the ACM*, 51(10):50–57, 2008.
- [23] Michael Garland and David B. Kirk. Understanding throughput-oriented architectures. *Communications of the ACM*, 53(11):58–66, 2010.
- [24] Mark Harris. Optimizing parallel reduction in CUDA. NVIDIA Developer Technology, <https://developer.download.nvidia.com/assets/cuda/files/reduction.pdf>, 2007.
- [25] Rodrigo Huerta, Mojtaba Abaie Shoushtary, José-Lorenzo Cruz, and Antonio González. Analyzing modern NVIDIA GPU cores. arXiv:2503.20481, 2025.
- [26] Intel Corporation. oneAPI specification. <https://oneapi.io>, 2020.
- [27] Aaron Jarmusch and Sunita Chandrasekaran. Microbenchmarking NVIDIA’s Blackwell architecture: An in-depth architectural analysis. arXiv:2512.02189, 2025.
- [28] Zhe Jia, Marco Maggioni, Benjamin Staiger, and Daniele P. Scarpazza. Dissecting the NVIDIA Volta GPU architecture via microbenchmarking. arXiv:1804.06826, 2018.
- [29] Zhe Jia, Marco Maggioni, Jeffrey Smith, and Daniele P. Scarpazza. Dissecting the NVidia Turing T4 GPU via microbenchmarking. arXiv:1903.07486, 2019.
- [30] Khronos OpenCL Working Group. The OpenCL specification, version 1.0. Khronos Group, 2008.
- [31] Khronos SYCL Working Group. SYCL specification, version 1.2. Khronos Group, 2015.
- [32] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. ImageNet classification with deep convolutional neural networks. In *Advances in Neural Information Processing Systems 25 (NIPS 2012)*, pages 1097–1105, 2012.
- [33] Jens Krüger and Rüdiger Westermann. Linear algebra operators for GPU implementation of numerical algorithms. *ACM Transactions on Graphics*, 22(3):908–916, 2003.
- [34] Chester Lam. GCN, AMD’s GPU architecture modernization. Chips and Cheese, <https://chipsandcheese.com/p/gcn-amds-gpu-architecture-modernization>, 2023.
- [35] Michael Larabel. AMD previews Instinct MI400 series & Helios AI rack. Phoronix, 2025.
- [36] E. Scott Larsen and David McAllister. Fast matrix multiplies using graphics hardware. In *Proc. ACM/IEEE Conference on Supercomputing (SC ’01)*, 2001.

- [37] Erik Lindholm, Mark J. Kilgard, and Henry Moreton. A user-programmable vertex engine. In *Proc. 28th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH '01)*, pages 149–158, 2001.
- [38] Erik Lindholm, John Nickolls, Stuart Oberman, and John Montrym. NVIDIA Tesla: A unified graphics and computing architecture. *IEEE Micro*, 28(2):39–55, 2008.
- [39] LLVM Project. AMDGPU target description (AMDGPU.td): FeatureMaxWavesPerEU8 for gfx942 and gfx950. <https://github.com/llvm/llvm-project/blob/main/llvm/lib/Target/AMDGPU/AMDGPU.td>, 2025.
- [40] LLVM Project. AMDGPUPassPipeline.cpp: AMDGPU pass pipeline (FixIrreducible, StructureCFG). <https://github.com/llvm/llvm-project/blob/main/llvm/lib/Target/AMDGPU/AMDGPUPassPipeline.cpp>, 2025.
- [41] LLVM Project. SIOptimizeControlFlow.cpp: lowering of divergent control flow for AMDGPU. <https://github.com/llvm/llvm-project/blob/main/llvm/lib/Target/AMDGPU/SIOptimizeControlFlow.cpp>, 2025.
- [42] William R. Mark, R. Steven Glanville, Kurt Akeley, and Mark J. Kilgard. Cg: A system for programming graphics hardware in a C-like language. *ACM Transactions on Graphics*, 22(3): 896–907, 2003.
- [43] Michael McCool, Stefanus Du Toit, Tiberiu Popa, Bryan Chan, and Kevin Moule. Shader algebra. *ACM Transactions on Graphics*, 23(3):787–795, 2004.
- [44] Wen mei W. Hwu, David B. Kirk, and Izzat El Hajj. *Programming Massively Parallel Processors: A Hands-on Approach*. Morgan Kaufmann, 4th edition, 2022.
- [45] John Nickolls, Ian Buck, Michael Garland, and Kevin Skadron. Scalable parallel programming with CUDA. *ACM Queue*, 6(2):40–53, 2008.
- [46] NVIDIA. Volta tuning guide. <https://docs.nvidia.com/cuda/volta-tuning-guide/index.html>, 2017.
- [47] NVIDIA. NVIDIA Turing GPU architecture. Whitepaper WP-09183-001, 2018.
- [48] NVIDIA. NVIDIA Hopper architecture in-depth. NVIDIA Technical Blog, <https://developer.nvidia.com/blog/nvidia-hopper-architecture-in-depth/>, 2022.
- [49] NVIDIA. CUDA binary utilities, release 12.4. https://docs.nvidia.com/cuda/archive/12.4.0/pdf/CUDA_Binary_Uilities.pdf, 2024.
- [50] NVIDIA. CUDA C++ best practices guide. CUDA Toolkit Documentation, <https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/>, 2025.
- [51] NVIDIA. Blackwell SM100 GEMMs (CUTLASS documentation). https://docs.nvidia.com/cutlass/4.6.0/media/docs/cpp/blackwell_functionality.html, 2025.
- [52] NVIDIA. Simplify GPU programming with NVIDIA CUDA tile in Python. NVIDIA Technical Blog, <https://developer.nvidia.com/blog/simplify-gpu-programming-with-nvidia-cuda-tile-in-python>, December 2025.

- [53] NVIDIA. NVIDIA Hopper tuning guide. CUDA Toolkit Documentation, <https://docs.nvidia.com/cuda/hopper-tuning-guide/>, 2025.
- [54] NVIDIA. Introducing NVFP4 for efficient and accurate low-precision inference. NVIDIA Technical Blog, <https://developer.nvidia.com/blog/introducing-nvfp4-for-efficient-and-accurate-low-precision-inference/>, 2025.
- [55] NVIDIA. tcgen05 MMA programming (CUTLASS documentation). https://docs.nvidia.com/cutlass/4.6.0/media/docs/pythonDSL/mma_docs/tcgen05_programming.html, 2025.
- [56] NVIDIA. WGMMMA programming (CUTLASS documentation). https://docs.nvidia.com/cutlass/4.6.0/media/docs/pythonDSL/mma_docs/wgmma_programming.html, 2025.
- [57] NVIDIA. NVIDIA Vera Rubin ramps into full production. Press release, <https://investor.nvidia.com/news/press-release-details/2026/NVIDIA-Vera-Rubin-Ramps-Into-Full-Production-to-Power-Agentic-AI-Factories-Worldwide/default.aspx>, 2026.
- [58] NVIDIA Corporation. NVIDIA launches the world’s first graphics processing unit: GeForce 256. Press release, August 1999.
- [59] NVIDIA Corporation. NVIDIA Tesla V100 GPU architecture. White paper WP-08608-001, 2017.
- [60] NVIDIA Corporation. NVIDIA A100 tensor core GPU architecture. White paper, <https://images.nvidia.com/aem-dam/en-zz/Solutions/data-center/nvidia-ampere-architecture-whitepaper.pdf>, 2020.
- [61] NVIDIA Corporation. NVIDIA H100 tensor core GPU architecture. White paper, 2022.
- [62] NVIDIA Corporation. NVIDIA H100 tensor core GPU. Data sheet, 2023.
- [63] NVIDIA Corporation. NVIDIA Blackwell architecture technical brief. Technical brief, 2024.
- [64] NVIDIA Corporation. CUDA C++ programming guide. <https://docs.nvidia.com/cuda/cuda-c-programming-guide/>, 2025.
- [65] NVIDIA Corporation. Parallel thread execution ISA. <https://docs.nvidia.com/cuda/parallel-thread-execution/>, 2025.
- [66] John D. Owens, David Luebke, Naga Govindaraju, Mark Harris, Jens Krüger, Aaron E. Lefohn, and Timothy J. Purcell. A survey of general-purpose computation on graphics hardware. *Computer Graphics Forum*, 26(1):80–113, 2007.
- [67] Timothy J. Purcell, Ian Buck, William R. Mark, and Pat Hanrahan. Ray tracing on programmable graphics hardware. *ACM Transactions on Graphics*, 21(3):703–712, 2002.
- [68] Mahesha S. Branch divergence in SIMT machines: From control-flow graph theory to compiler divergence analysis in LLVM. Technical survey, <https://hsmahesha.github.io/papers/divergence-analysis.pdf>, 2026.
- [69] Mahesha S. Modern low-precision numerical formats for AI hardware. Technical survey, <https://hsmahesha.github.io/papers/low-precision-types.pdf>, 2026.

- [70] Mahesha S. Systolic arrays: From 1970s VLSI to modern AI matrix engines. Technical survey, <https://hsmahesha.github.io/papers/systolic-arrays.pdf>, 2026.
- [71] ServeTheHome. NVIDIA launches next-generation Rubin AI compute platform at CES 2026. <https://www.servethehome.com/nvidia-launches-next-generation-rubin-ai-compute-platform-at-ces-2026/>, 2026.
- [72] Tolga Soyata. *GPU Parallel Program Development Using CUDA*. CRC Press, 2018.
- [73] Philippe Tillet. Introducing Triton: Open-source GPU programming for neural networks. OpenAI blog, <https://openai.com/index/triton/>, 2021.
- [74] Philippe Tillet, H. T. Kung, and David Cox. Triton: An intermediate language and compiler for tiled neural network computations. In *Proc. 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages (MAPL)*, pages 10–19, 2019.
- [75] Vasily Volkov. Better performance at lower occupancy. GPU Technology Conference (GTC), 2010.
- [76] David Wang. The new AMD compute DNA (CDNA) architecture. LinkedIn article, <https://www.linkedin.com/pulse/new-amd-compute-dna-cdna-architecture-david-wang-%E7%8E%8B%E5%95%9F%E5%B0%9A-/>, 2020.
- [77] Samuel Williams, Andrew Waterman, and David Patterson. Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009.

A Glossary

Every technical term used in the paper, in alphabetical order, with the section where it is explained.

ALU (arithmetic logic unit)

The circuit that actually performs additions, multiplications and comparisons (Section 2).

Arithmetic intensity

Number of arithmetic operations a program performs per byte it moves to or from main memory; decides whether it is limited by arithmetic or by memory (Section 12.3).

Bandwidth

How much data memory can deliver per second (Section 2).

Bank conflict

Two or more threads of a warp needing different words from the same shared-memory bank, so that they must wait in turn (Section 12.2).

Barrier

A point in a kernel that no thread of a block passes until all of them have reached it (`__syncthreads()`) (Section 9).

Block (thread block)

A group of up to 1,024 threads that run on the same SM, use the same shared memory and wait for each other at a barrier (Section 9). AMD: work-group.

Branch divergence

Threads of one warp taking different paths at an `if`, so that the threads of one path are switched off while the other path runs (Sections 11.3 and 14.3).

Cache

A small, fast memory close to the ALUs that keeps copies of recently used data (Section 2). GPUs have an L1 cache in each SM and a large L2 cache shared by all SMs.

Coalescing

Combining the memory accesses of a warp into a few large transfers when the threads access neighbouring addresses (Section 12.2).

Compute unit (CU)

AMD's name for a GPU core, the equivalent of NVIDIA's SM (Section 14.2).

CUDA

NVIDIA's extension of C/C++ for writing GPU programs (Sections 7 and 9).

Device

The GPU, as seen from the program running on the CPU (Section 9).

Die

One piece of silicon; large modern GPUs combine several dies in one package (Section 10).

Execution mask

One bit per thread of a warp, saying which threads take part in the current instruction (Section 11). On AMD GPUs it is a register called EXEC (Section 14.3).

Fixed function

Hardware that performs a built-in calculation with a few settings but cannot be programmed (Section 4).

FLOP/s

Floating-point operations per second, the usual measure of arithmetic speed (Section 2).

Fragment

A candidate pixel produced when a triangle is rasterised, waiting to be coloured (Section 3).

Frame

One complete image; games draw 60 or more per second (Section 3).

GPGPU

General-purpose computing on GPUs: using a GPU for work other than graphics (Section 6).

Grid

All the blocks launched by one kernel launch (Section 9).

HBM (high-bandwidth memory)

Stacks of memory chips placed right beside the GPU in the same package, giving terabytes per second of bandwidth (Section 10).

Host

The CPU and its program, which prepares data and launches kernels on the GPU (Section 9).

Kernel

A function, written for one thread, that runs on the GPU once for every thread launched (Section 9).

Latency

The time one request takes to complete, for example one memory read (Section 2).

Matrix core

AMD's unit for multiplying small matrices, one in each SIMD unit of a compute unit; the counterpart of NVIDIA's tensor core (Section 14.5).

Occupancy

The number of warps present on an SM as a fraction of the maximum number of warps the SM can hold (Section 12.1).

Pipelining

Splitting a calculation into steps done by a row of units, so that different items are at different steps at the same time, like cars on an assembly line (Section 4).

Pixel

One dot of the screen; a 4K screen has about 8.3 million (Section 3).

Predicate

A one-bit register that each thread has for itself; an instruction guarded by it is carried out only by the threads in which it is true (Section 14.3).

Rasteriser

The hardware that works out which pixels each triangle covers (Section 3).

Reconvergence

The point where the threads of a split warp join up again after an `if` (Section 11).

Reduction

Combining all elements of an array into one value, such as their sum (Section 12.4).

Register

A tiny, very fast storage slot inside a core (Section 2); a GPU SM has 65,536 of them.

Scalar unit

AMD's unit that computes a value once for a whole wavefront and keeps it in scalar registers (Section 14.4).

Shader

A small program run by the graphics hardware once for every vertex or fragment (Section 3).

Shared memory

A small, fast memory inside each SM, shared by the threads of one block (Section 9). AMD: local data share (LDS).

SIMD

Single instruction, multiple data: one instruction applied to several values at once by several ALUs (Sections 2 and 8).

SIMT

Single instruction, multiple threads: NVIDIA's model in which the 32 threads of a warp execute together under a mask (Section 11).

SPMD

Single program, multiple data: one program run separately on many data items (Section 3).

Streaming multiprocessor (SM)

NVIDIA's name for a GPU core (Section 10).

Tensor core

A unit inside each SM that multiplies small matrices, used heavily in AI (Sections 13 and 14.5). AMD: matrix core.

Texture

An image glued onto 3D surfaces; also a way of reading GPU memory (Section 3).

Thread

One independent sequence of instructions with its own registers (Section 2); on a GPU, one instance of a kernel.

Throughput

Total work completed per second, for a GPU the number of threads finished per second (Section 8.6).

Uniform datapath

NVIDIA's unit, added in 2018, that computes a value once for a whole warp (Sections 10.5 and 14.4).

Vector register

On AMD GPUs, a register that holds one 32-bit value for each of the 64 threads of a wavefront (Section 14.2).

Vertex

A corner point of a triangle, with its position and other data (Section 3).

Warp

A group of 32 consecutive threads that the hardware executes together (Sections [10](#) and [11](#)). AMD: wavefront (64 threads).

Wavefront

AMD's group of 64 threads that execute together; the counterpart of NVIDIA's warp (Section [14.2](#)).

Work-group

AMD's name for a thread block (Section [14](#)).